

CHAPTER 5

ADVERSARIAL SEARCH

INTRODUCTION TO ARTIFICIAL INTELLIGENT (AI)



REV: 06.05.2024

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Adversarial Search Problems (Games)

In this chapter, we assume that there are multi-agents in the environment. Since the agents' goals are in conflict, we investigate **adversarial search problems**—often known as **games**.

Games are problems with multi-agent environments.

We will consider games with two players, whom we call **MAX** and **MIN**. (Our object is to search for a strategy on behalf of the player **MAX**.)

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Adversial Search Problems (Games)

Here, we assume that the enviroment is

- deterministic,
- fully observable
- in which two agents act alternately and
- in which the utility values at the end of the game are always equal and opposite (**zero-sum games**). For example, if one player wins a game of chess, the other player necessarily loses.

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Definition of a Game

A game can be formally defined as a kind of search problem with the following elements:

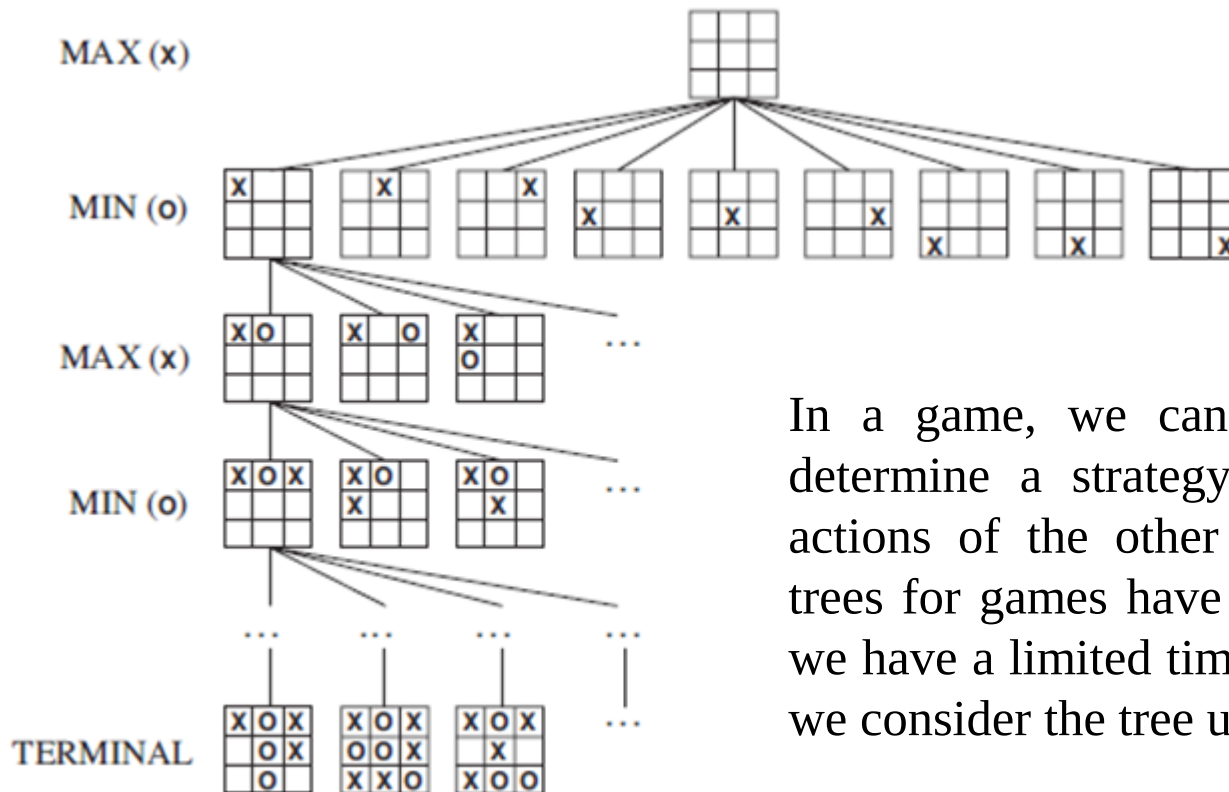
- S_0 : The initial state, which specifies how the game is set up at the start.
- **PLAYER**(s): Defines which player (**MAX** or **MIN**) will move in a state s .
- **ACTIONS**(s): Returns the set of legal moves (actions) in a state s .
- **RESULT**(s, a): The transition model, which defines the result of a move.
- **TERMINAL-TEST**(s): A terminal test, which is true when the game is over (or a leaf node is being searched) and false otherwise. States where their terminal test are true are called **terminal states**.
- **UTILITY**(s, p): A utility function (an objective function), defines the final numeric value for a game that ends in terminal state s for a player p .

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The Tic-Tac-Toe Game



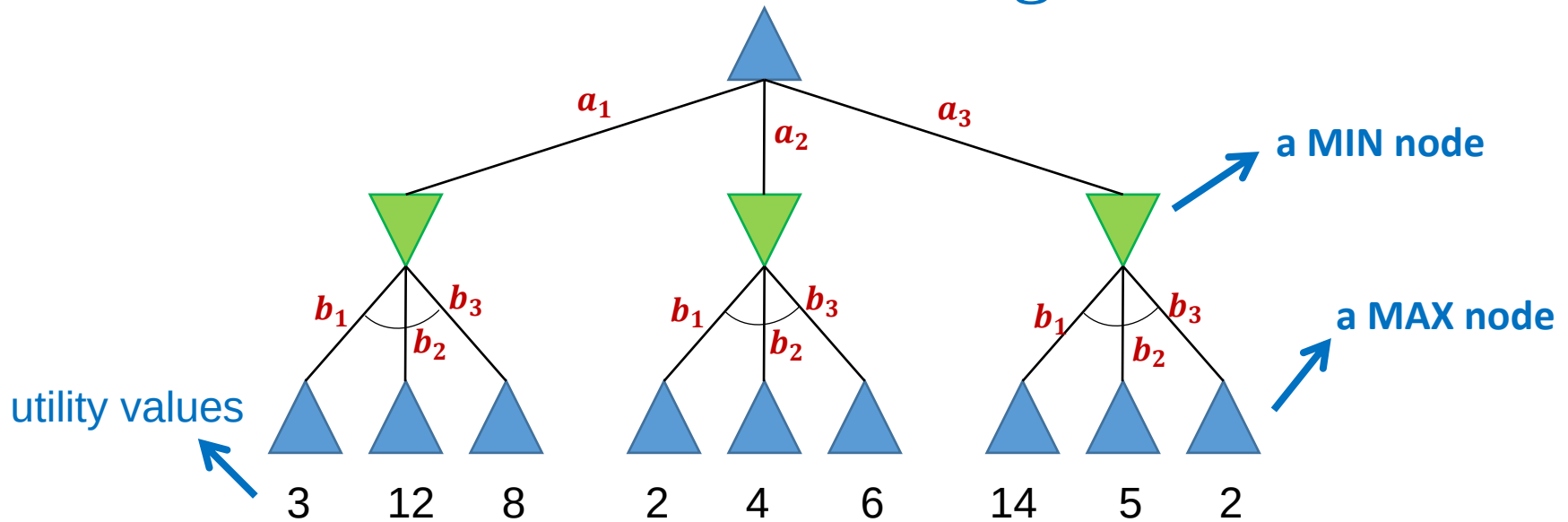
In a game, we can make a **search tree** to determine a strategy that also considers the actions of the other agent. Generally, search trees for games have very high depth and also we have a limited time to choose an action. So, we consider the tree up to **a specific depth**.

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The Minimax Algorithm



Consider a simple game that ends after one move each by **MAX** and **MIN**. We say that this tree is **one move deep**, consisting of **two half-moves**, each of which is called a **ply**. Given a game tree, the optimal strategy can be determined from the **minimax value** of each node. The minimax value of a node is determined by the **utility function** for the **MAX**. Here, player **MAX** assumes that player **MIN** chooses optimal actions.

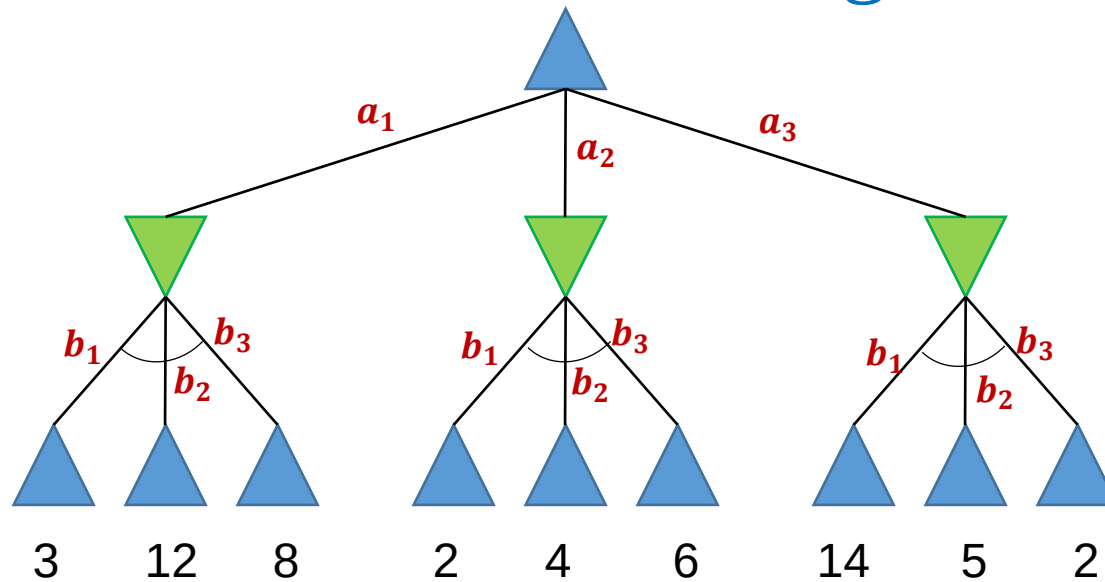
The player **MAX**  to reach the node with max value
The player **MIN**  to reach the node with min value

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The Minimax Algorithm

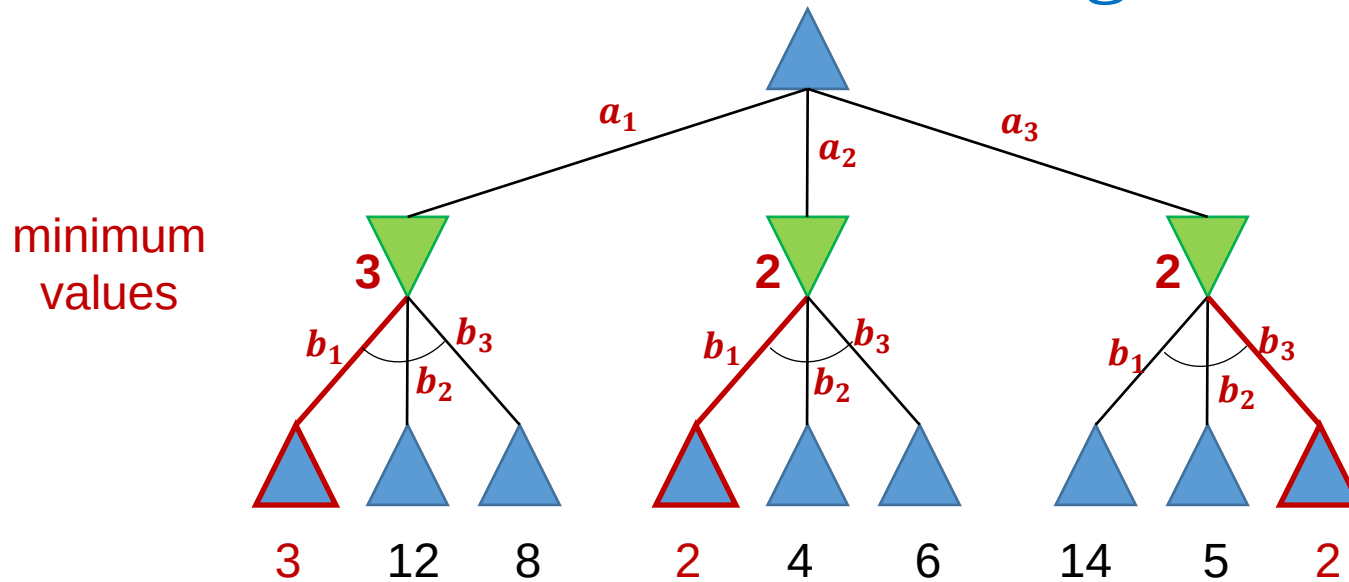


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The Minimax Algorithm

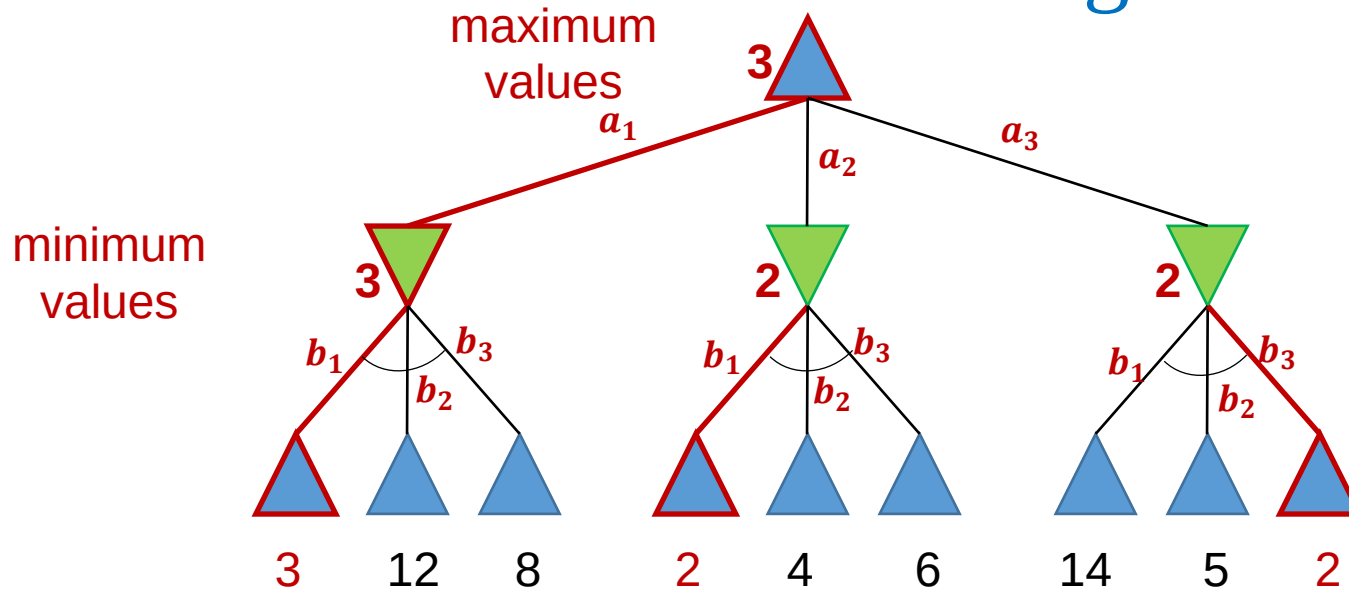


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



The Minimax Algorithm



Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The **Minimax algorithm** is given below.

```
function MINIMAX-DECISION(state) returns an action
- return argmaxa ∈ ACTIONS(s) MAX_VALUE(RERESULT(state, a))
```

```
function MAX_VALUE (state) returns a utility value
- if TERMINAL-TEST(state) then return UTILITY(state)
- v ← -∞
- for each action a in ACTIONS(state) do
  - v ← MAX(v, MIN_VALUE(RERESULT(s, a)))
- return v
```

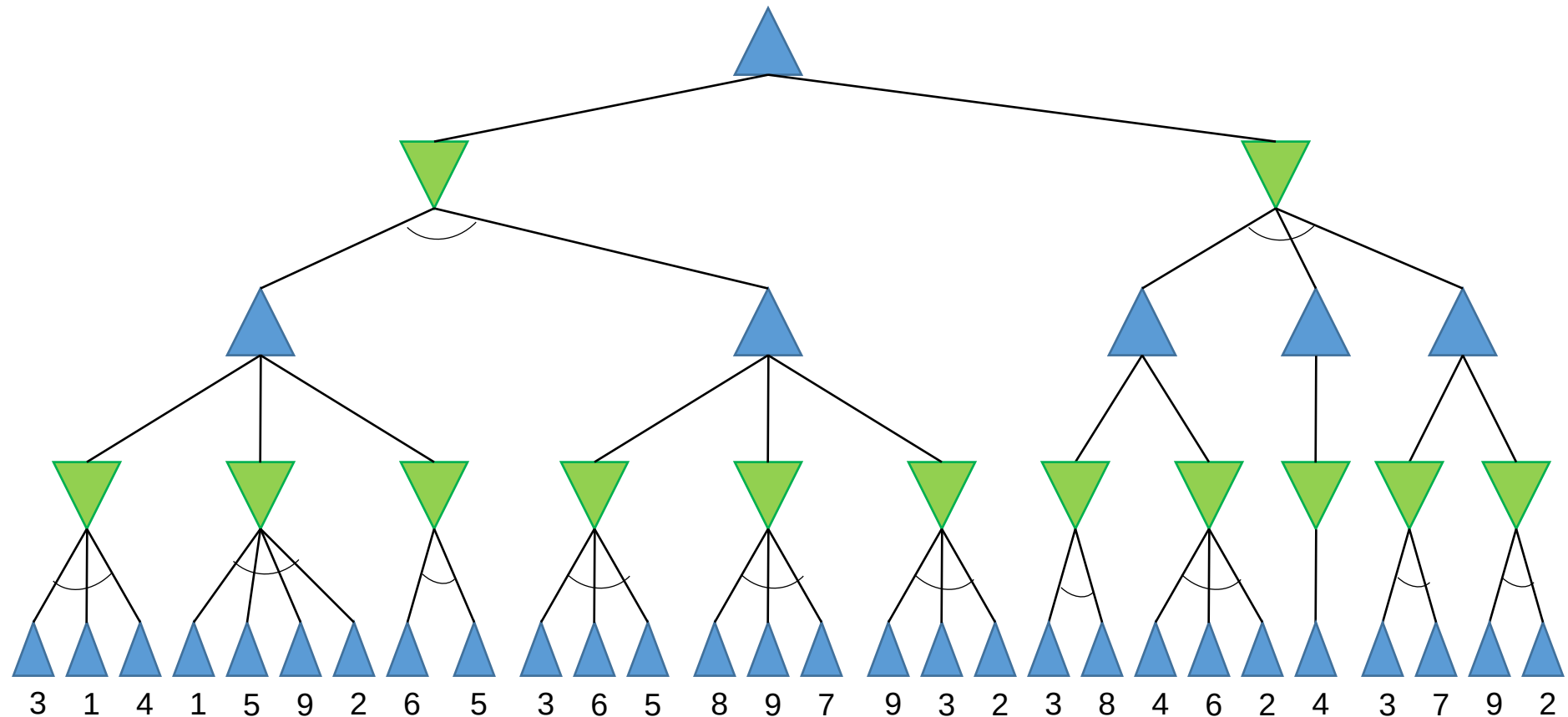
```
function MIN_VALUE (state) returns a utility value
- if TERMINAL-TEST(state) then return UTILITY(state)
- v ← ∞
- for each action a in ACTIONS(state) do
  - v ← MIN(v, MAX_VALUE(RERESULT(s, a)))
- return v
```

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Tree for two-moves deep

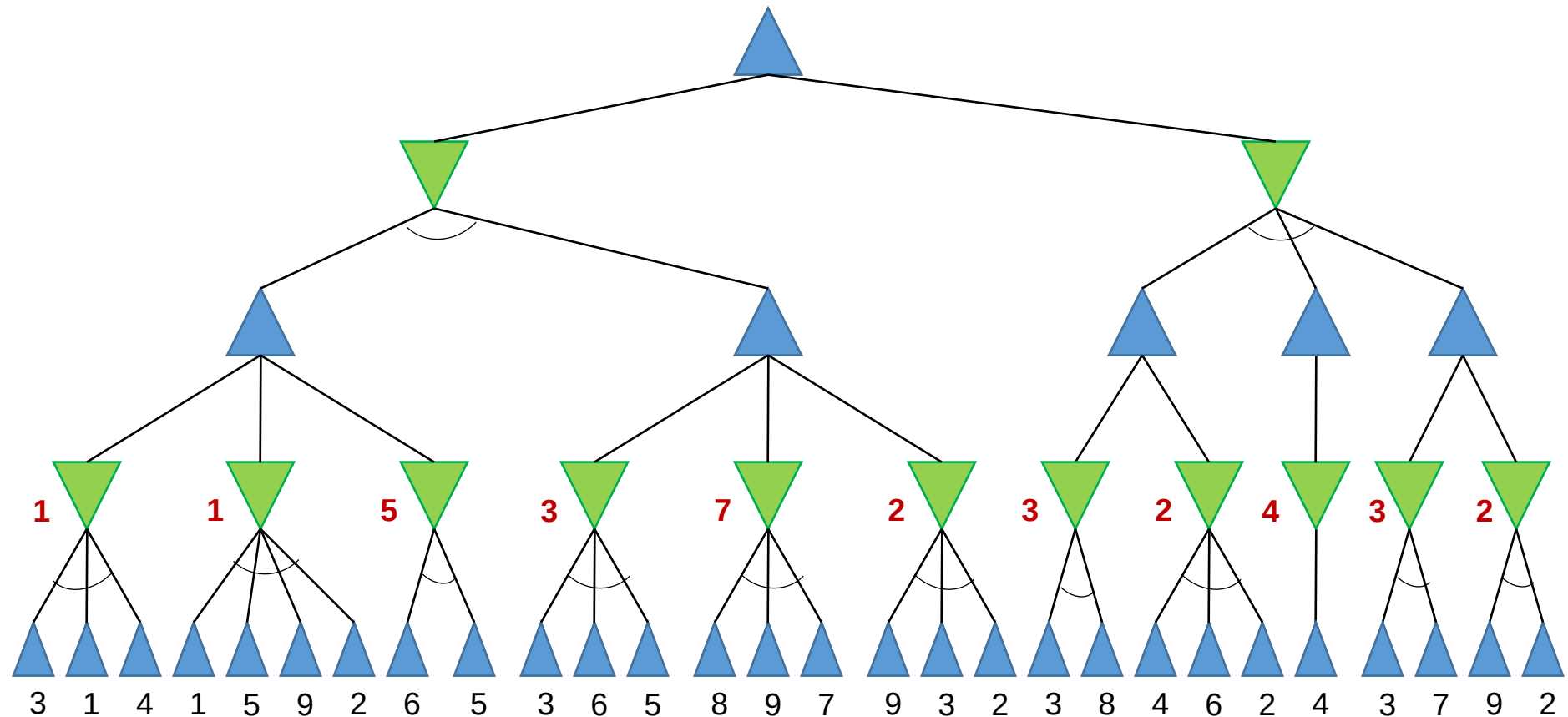


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Tree for two-moves deep

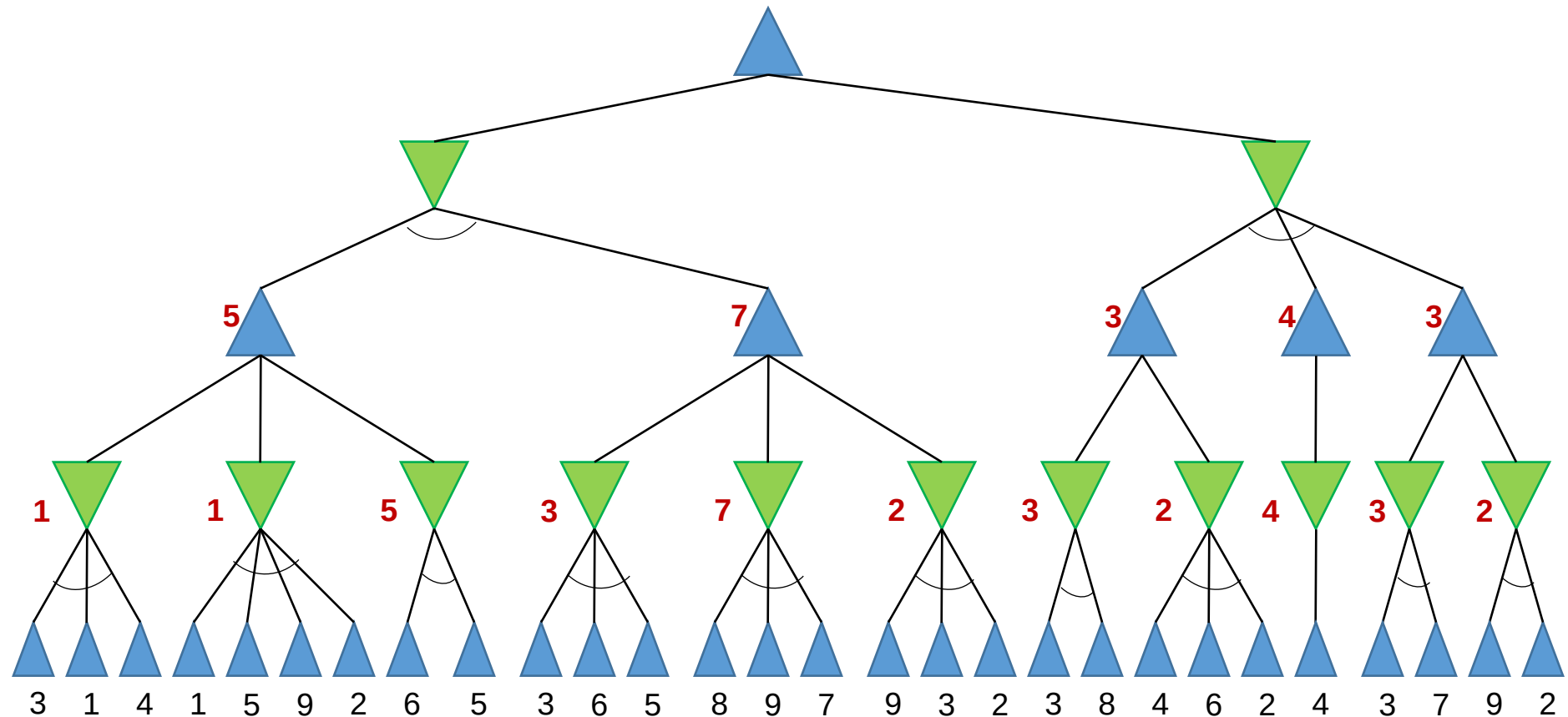


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Tree for two-moves deep

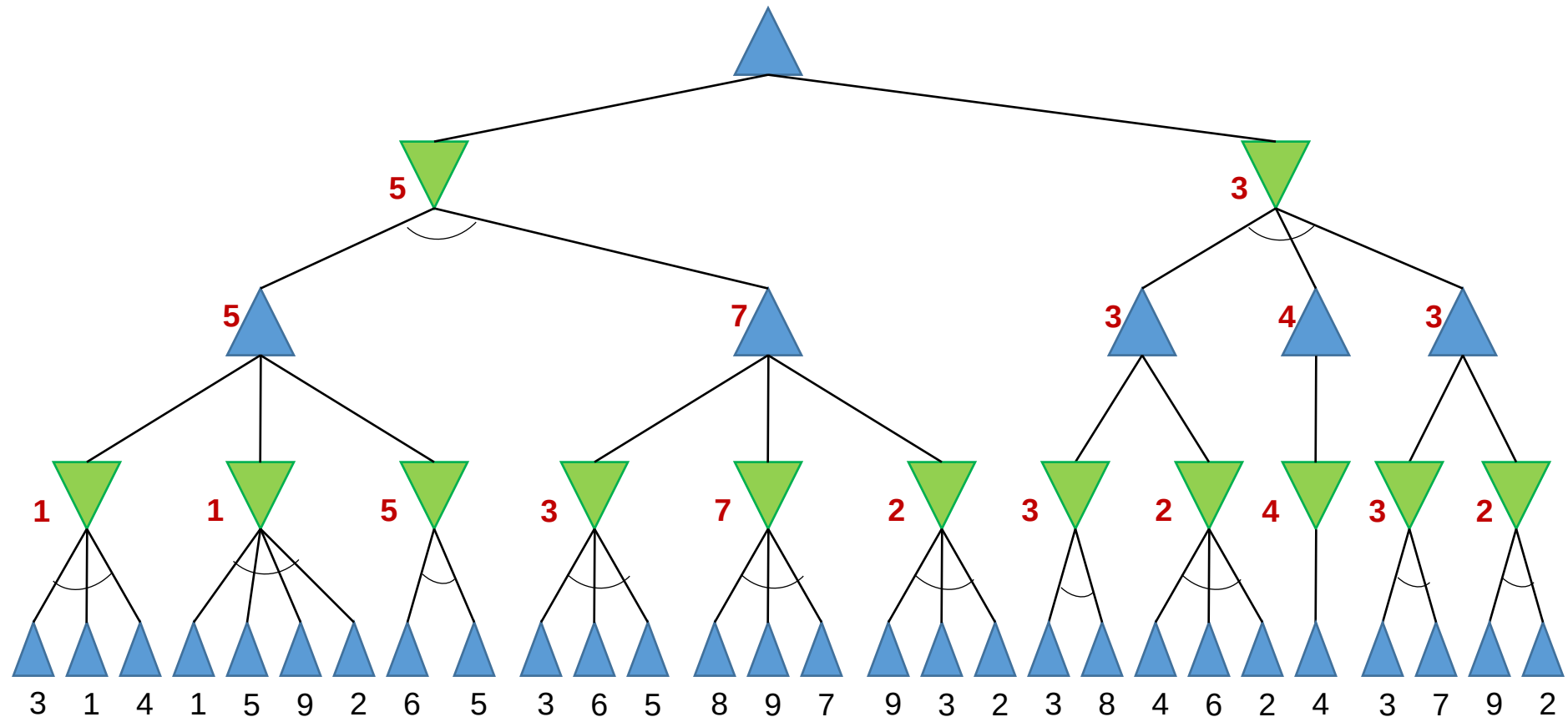


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Tree for two-moves deep

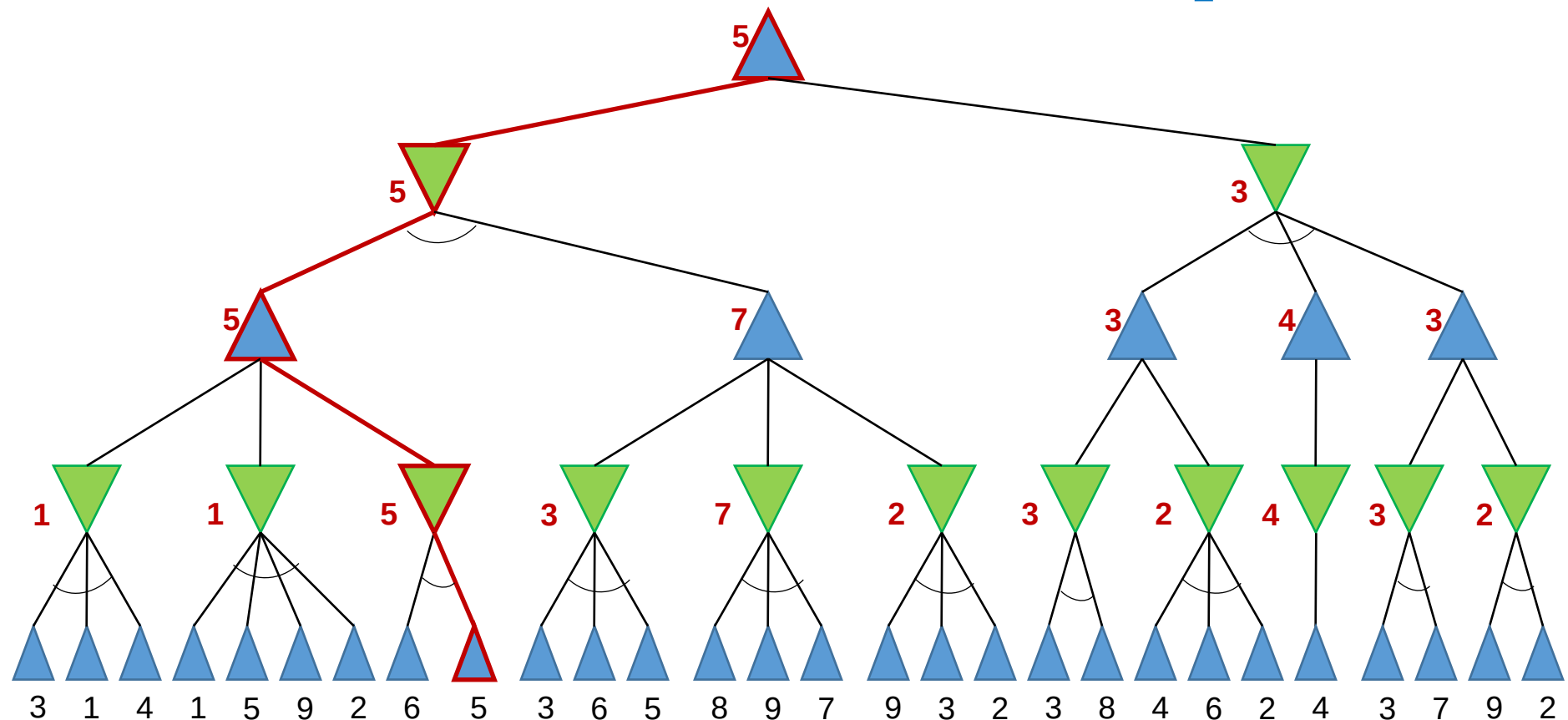


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Tree for two-moves deep



Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Ex: Tic-Tac-Toe

Consider the tic-tac-toe game.

- Define an appropriate **utility function**.
- Construct its **search tree with one move** (two plies) deep.
- Search for the optimal solution by using the **minimax algorithm**.

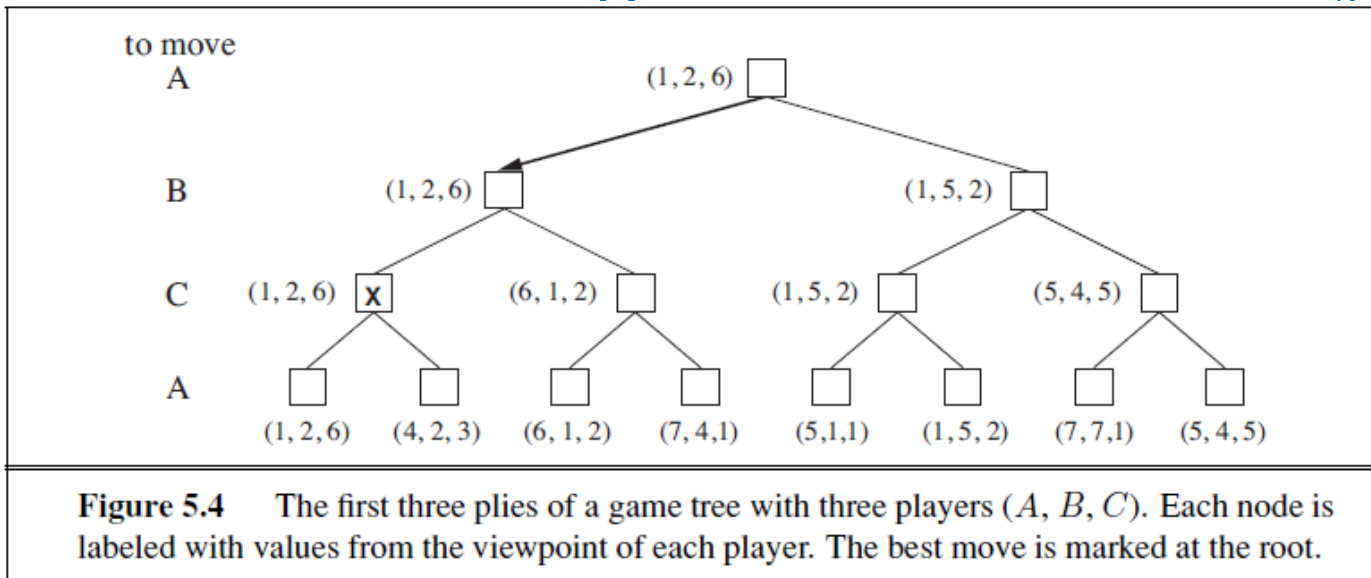
Ignore the symmetric states having the same minimax value.

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The Minimax Algorithm for Multi-Players



We need to replace the single value for each **node with a vector** of values. The elements of the vector represent the utility value of each player and each player wants to maximize its utility function.

Multiplayer games usually involve **alliances** among the players. Alliances are made and broken as the game proceeds. For example, suppose A and B are in weak positions and C is in a stronger position. Then it is often optimal for both A and B to attack C rather than each other. Thus, the minimax algorithm should be improved for such cases.

Adversarial Search

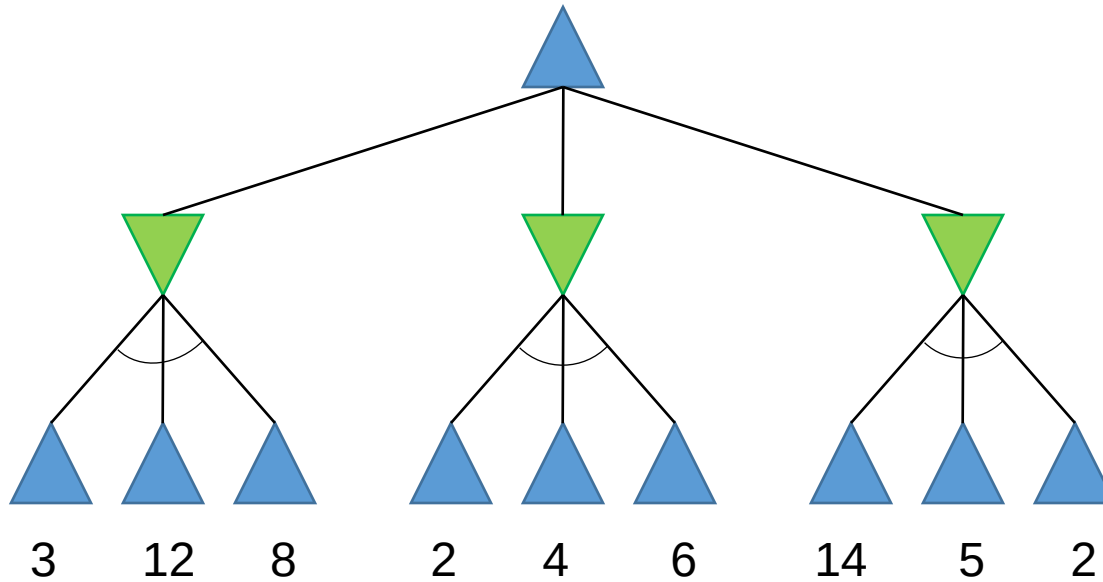
INTRODUCTION TO ARTIFICIAL INTELLIGENT



Alpha-Beta Pruning

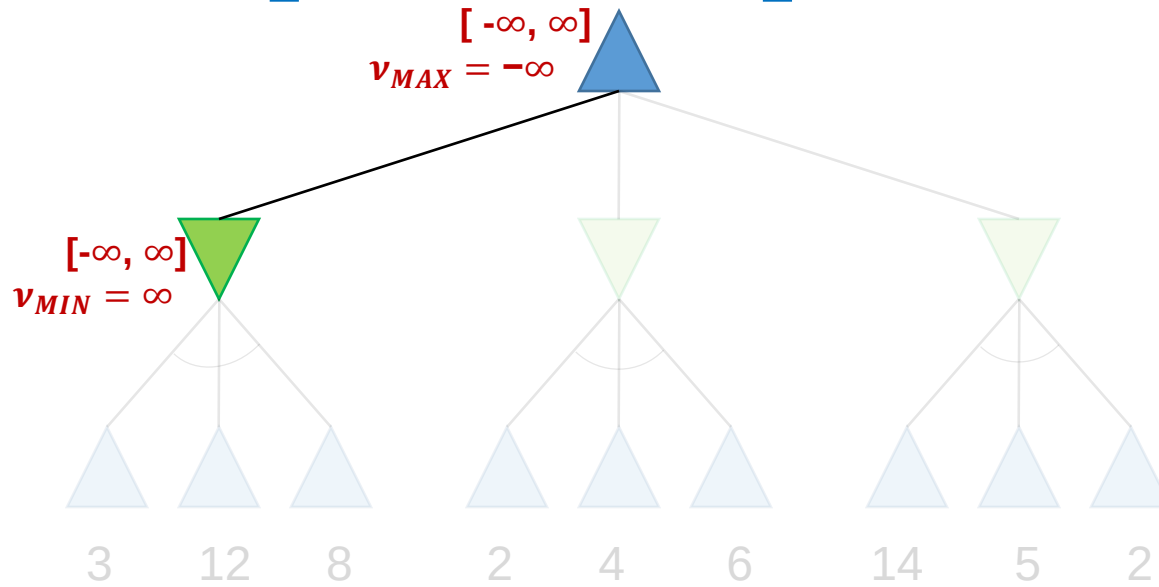
In minimax algorithm, we can use pruning, which is called **Alpha-Beta pruning**, without changing the final result. Thus, we can decrease the number of the produced nodes.

alpha-beta pruning

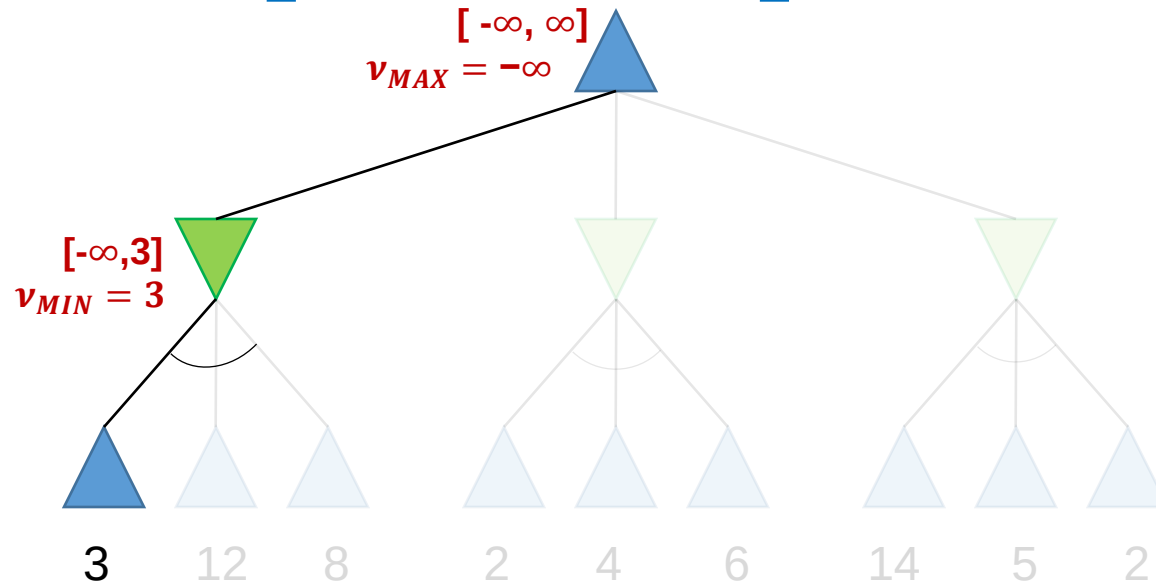


We use the deepening-first search for the minimax trees.

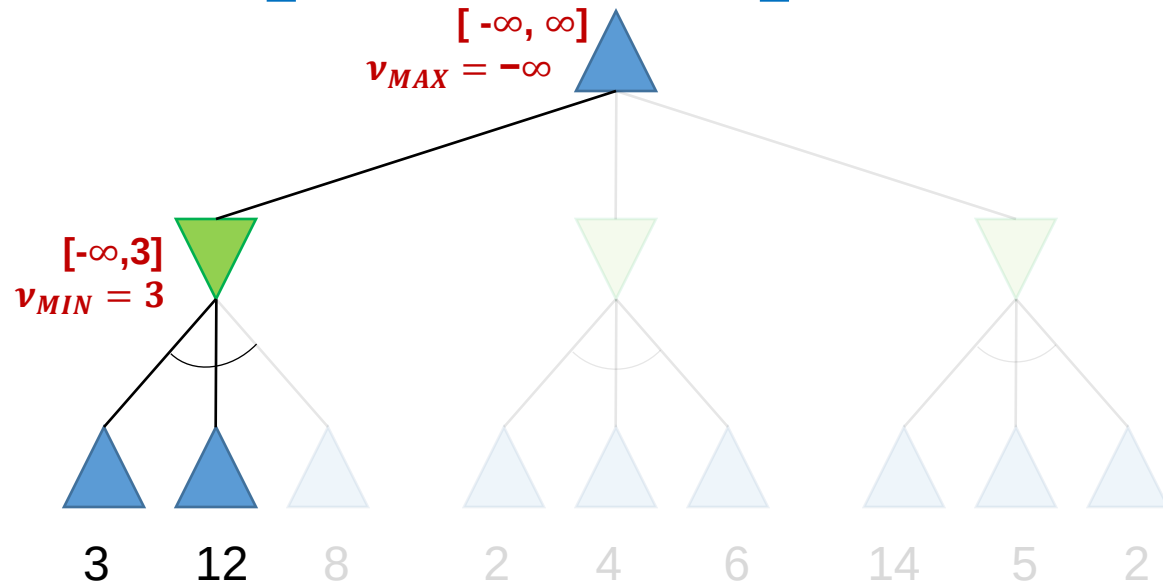
alpha-beta pruning



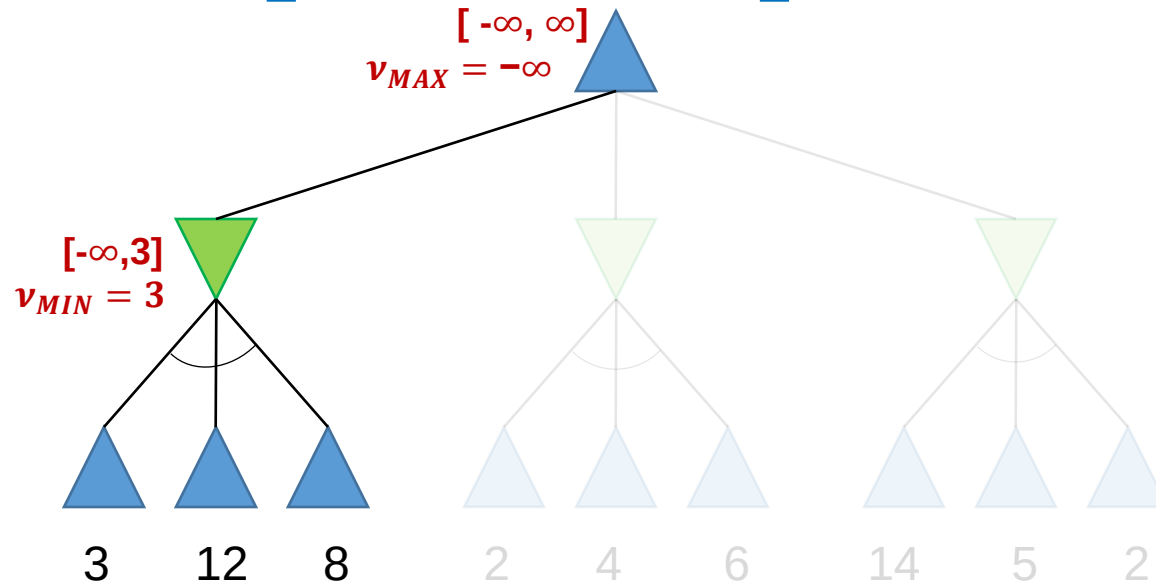
alpha-beta pruning



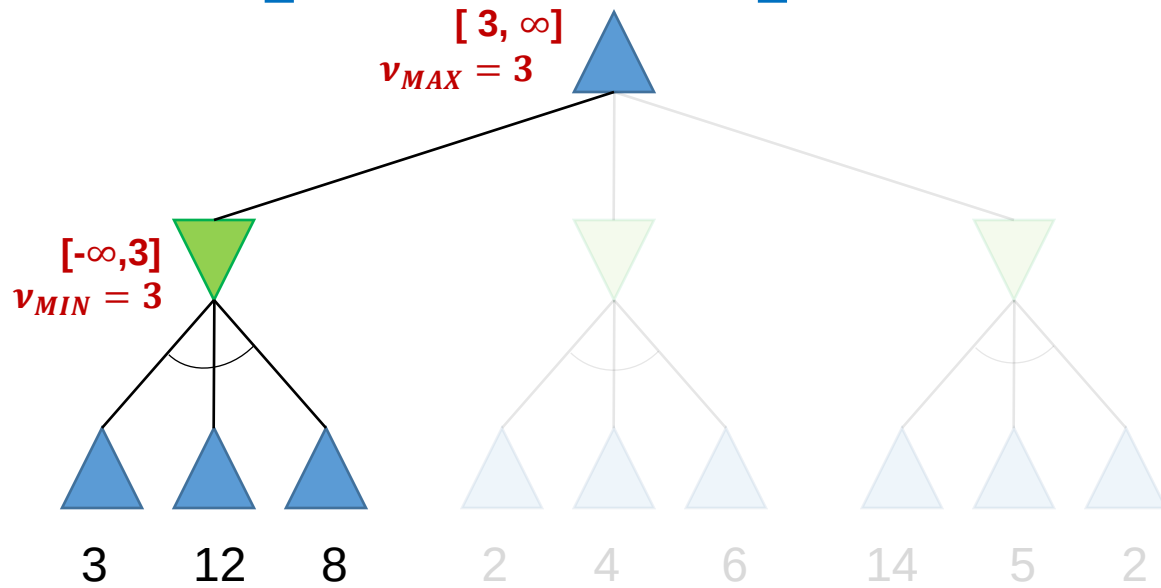
alpha-beta pruning



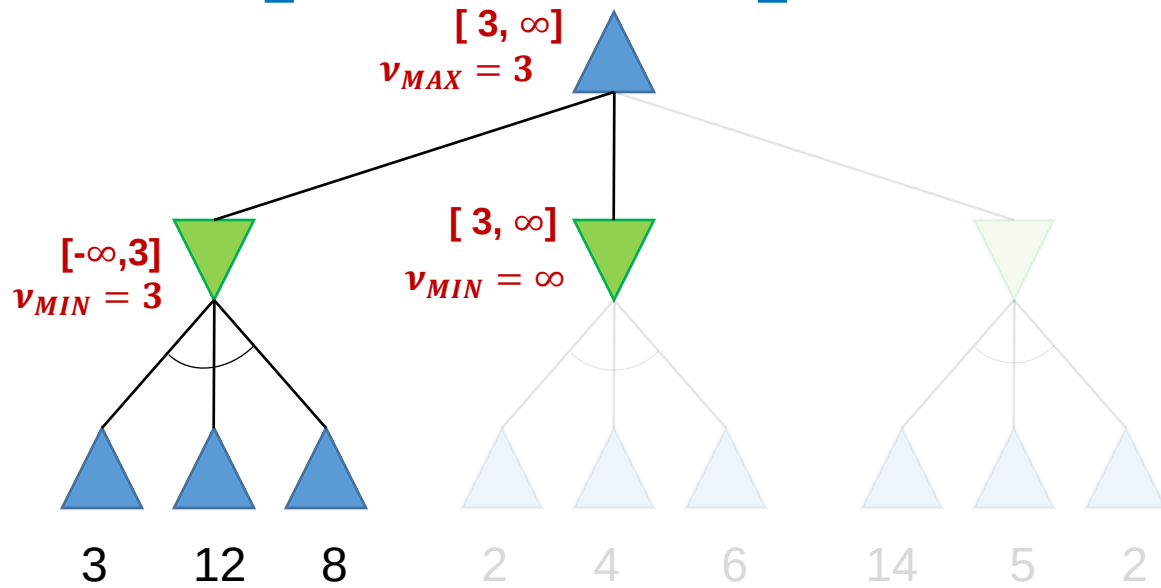
alpha-beta pruning



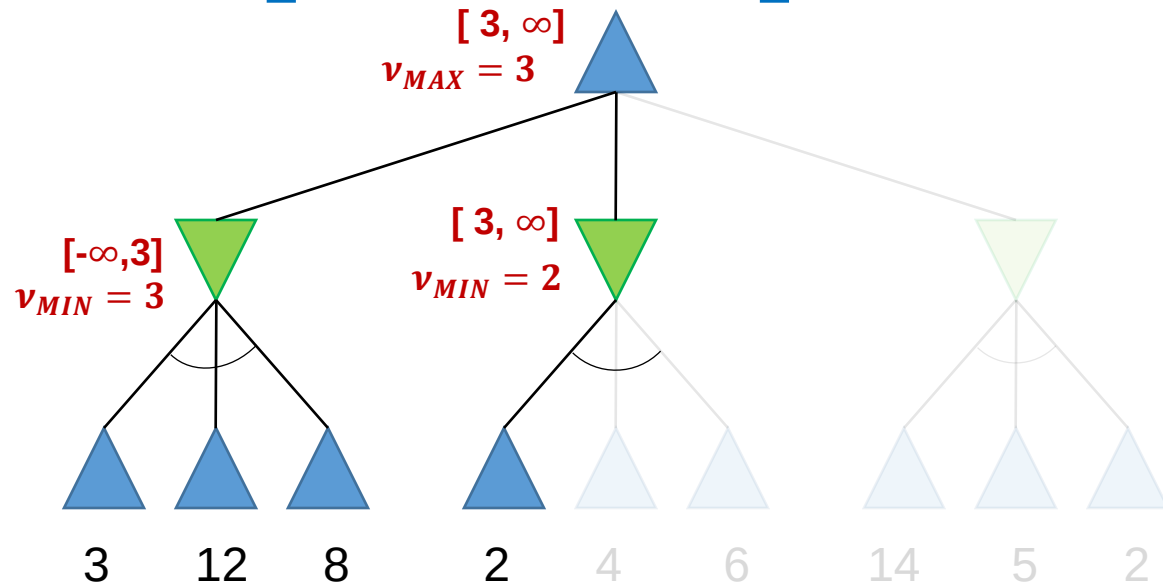
alpha-beta pruning



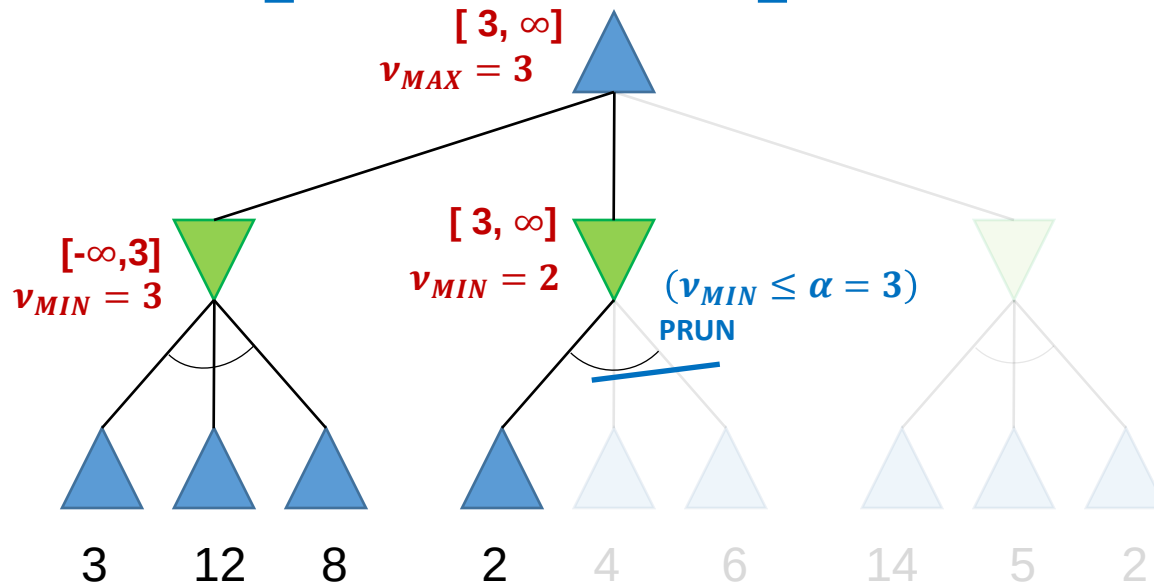
alpha-beta pruning



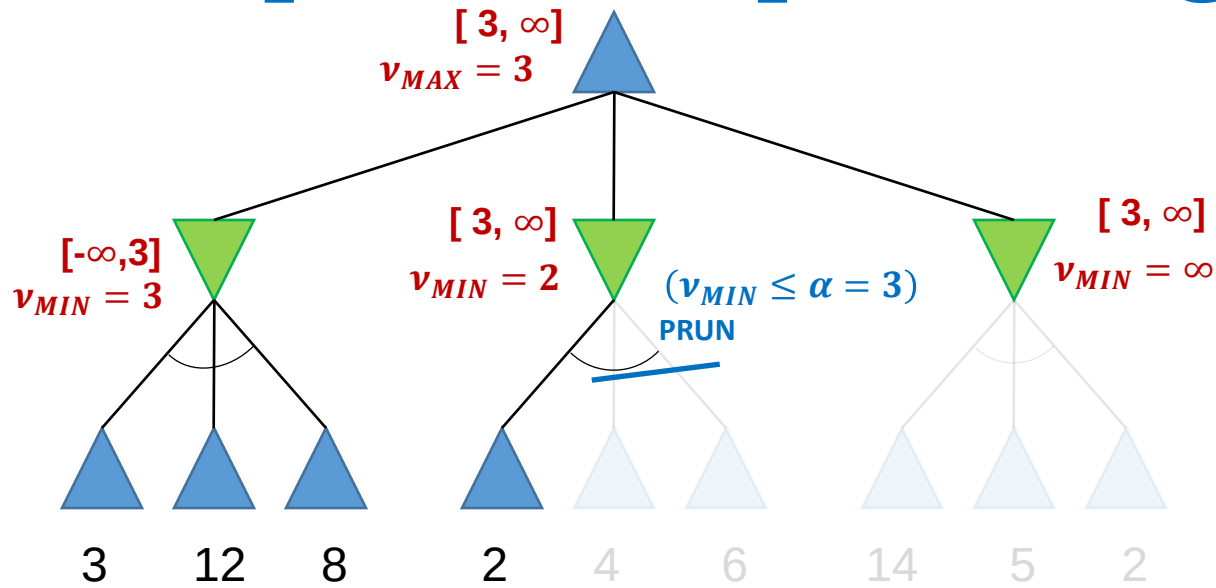
alpha-beta pruning



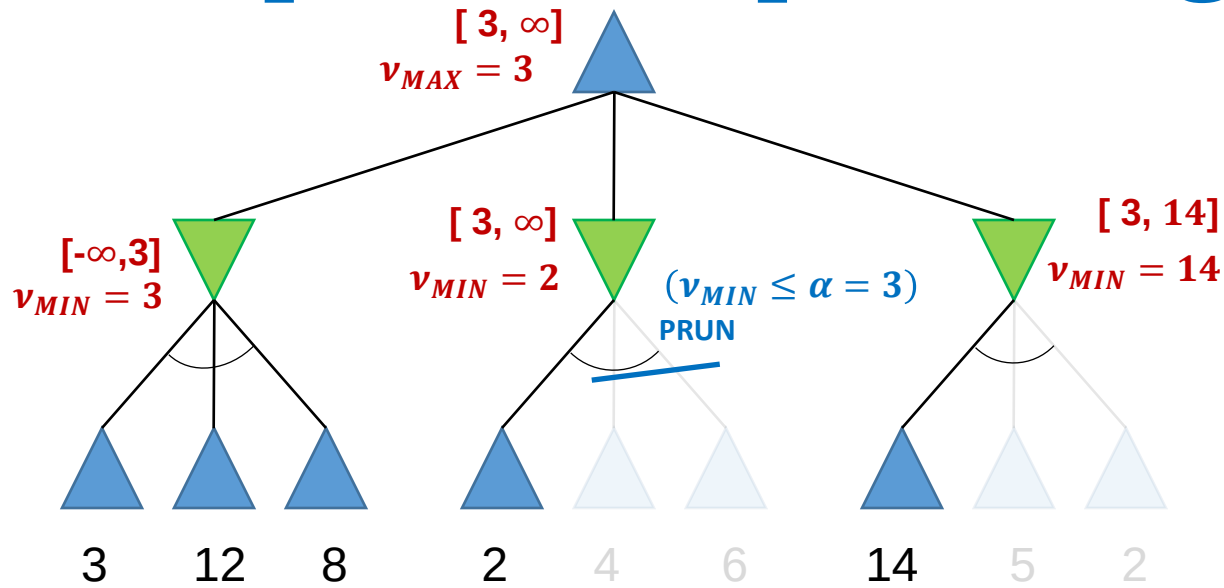
alpha-beta pruning



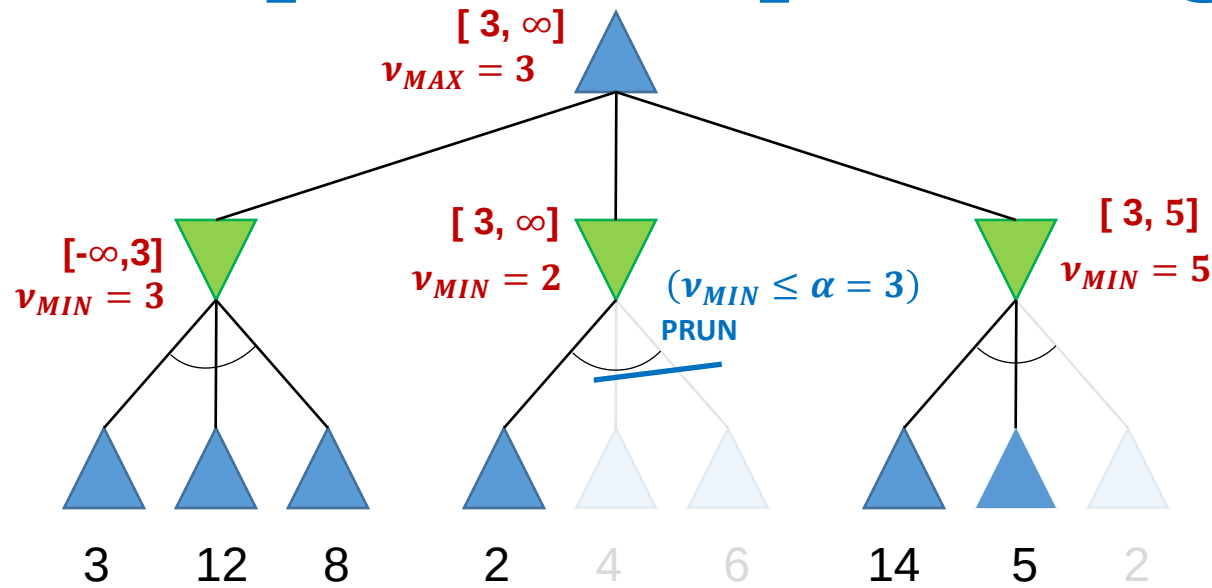
alpha-beta pruning



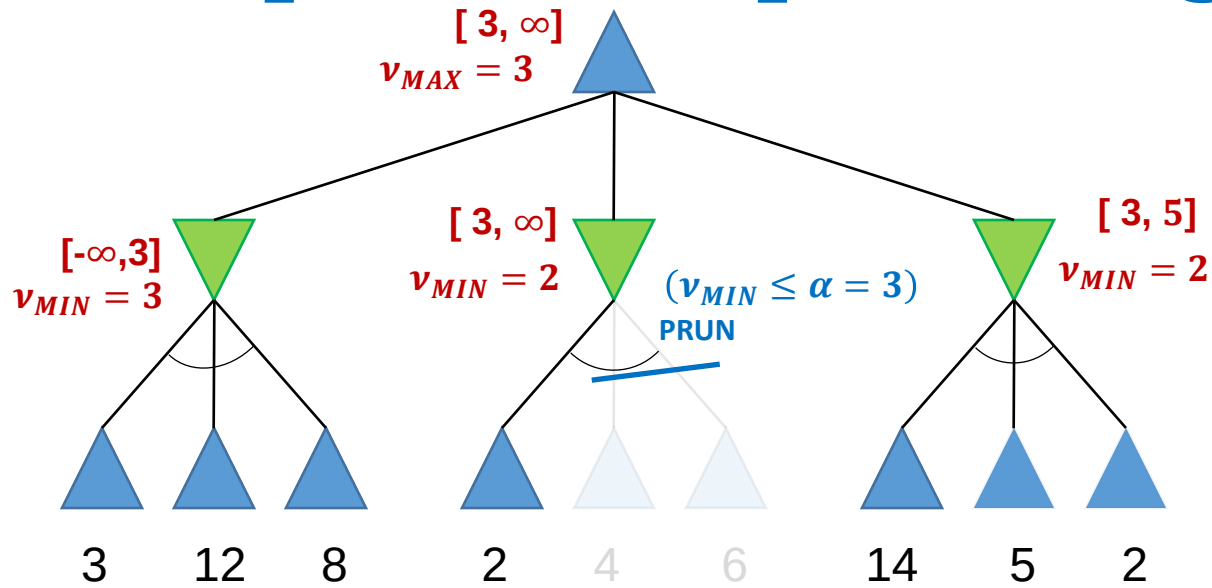
alpha-beta pruning



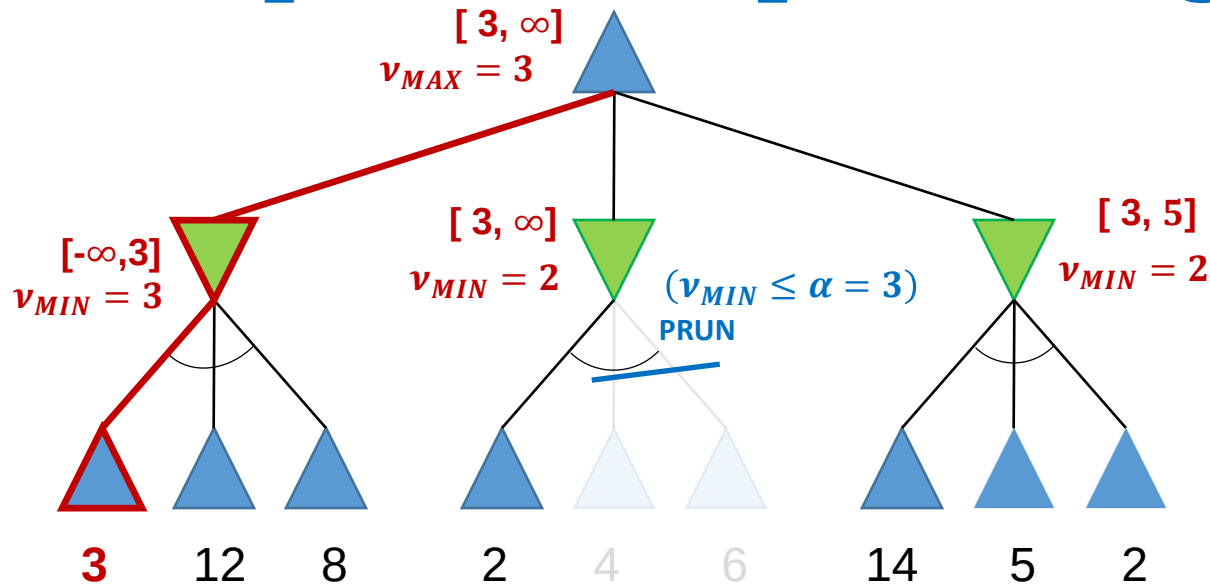
alpha-beta pruning



alpha-beta pruning



alpha-beta pruning



Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



The minimax algorithm with alpha-beta pruning is given below.

```
function ALPHA-BETA-SEARCH (state) returns an action
```

- $v \leftarrow \text{MAX_VALUE}(\text{state}, -\infty, \infty)$
- return the action in $\text{ACTIONS}(\text{state})$ with value v

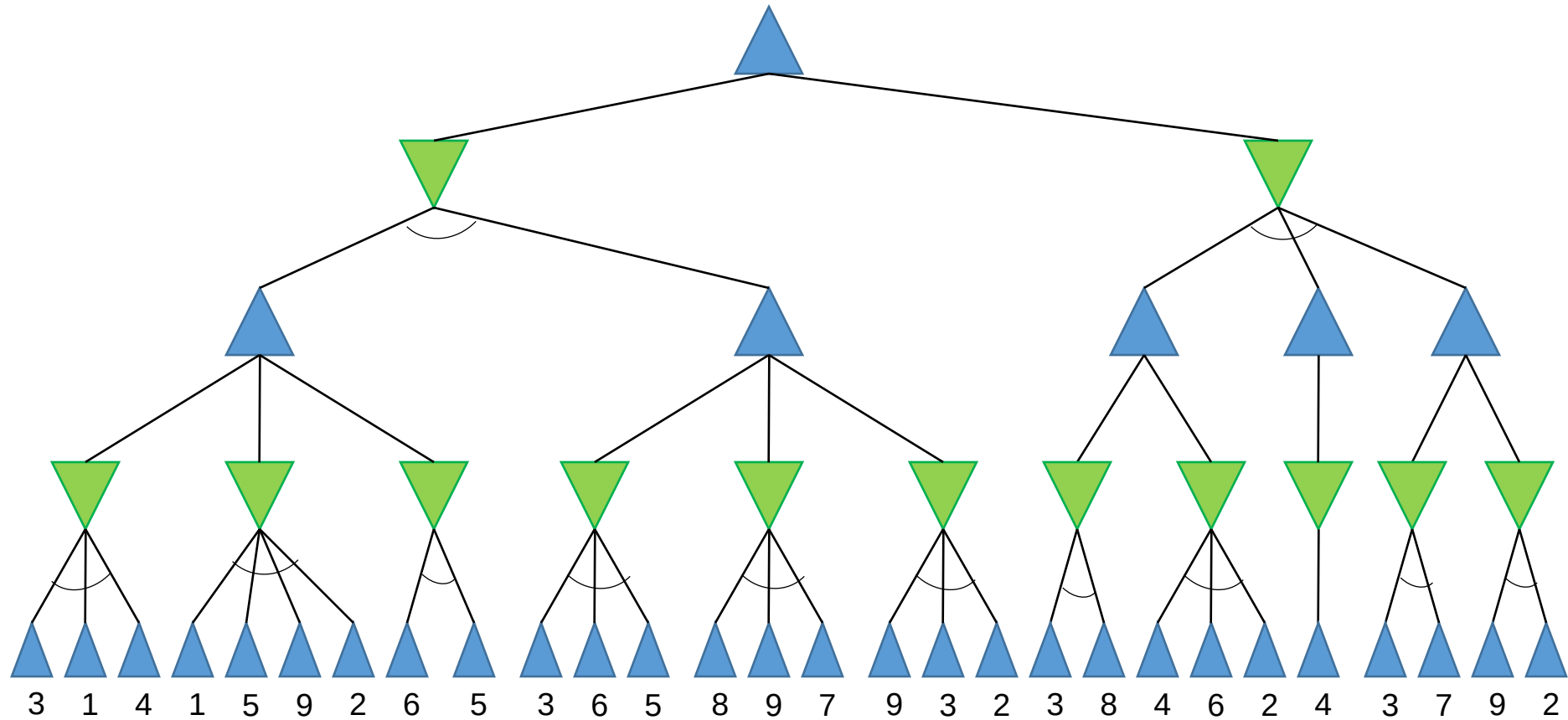
```
function MAX_VALUE (state,  $\alpha$ ,  $\beta$ ) returns a utility value
```

- if $\text{TERMINAL-TEST}(\text{state})$ then return $\text{UTILITY}(\text{state})$
- $v \leftarrow -\infty$
- for each action a in $\text{ACTIONS}(\text{state})$ do
 - $v \leftarrow \text{MAX}(v, \text{MIN_VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 - if $v \geq \beta$ then return v // Pruning
 - $\alpha \leftarrow \text{MAX}(\alpha, v)$
- return v

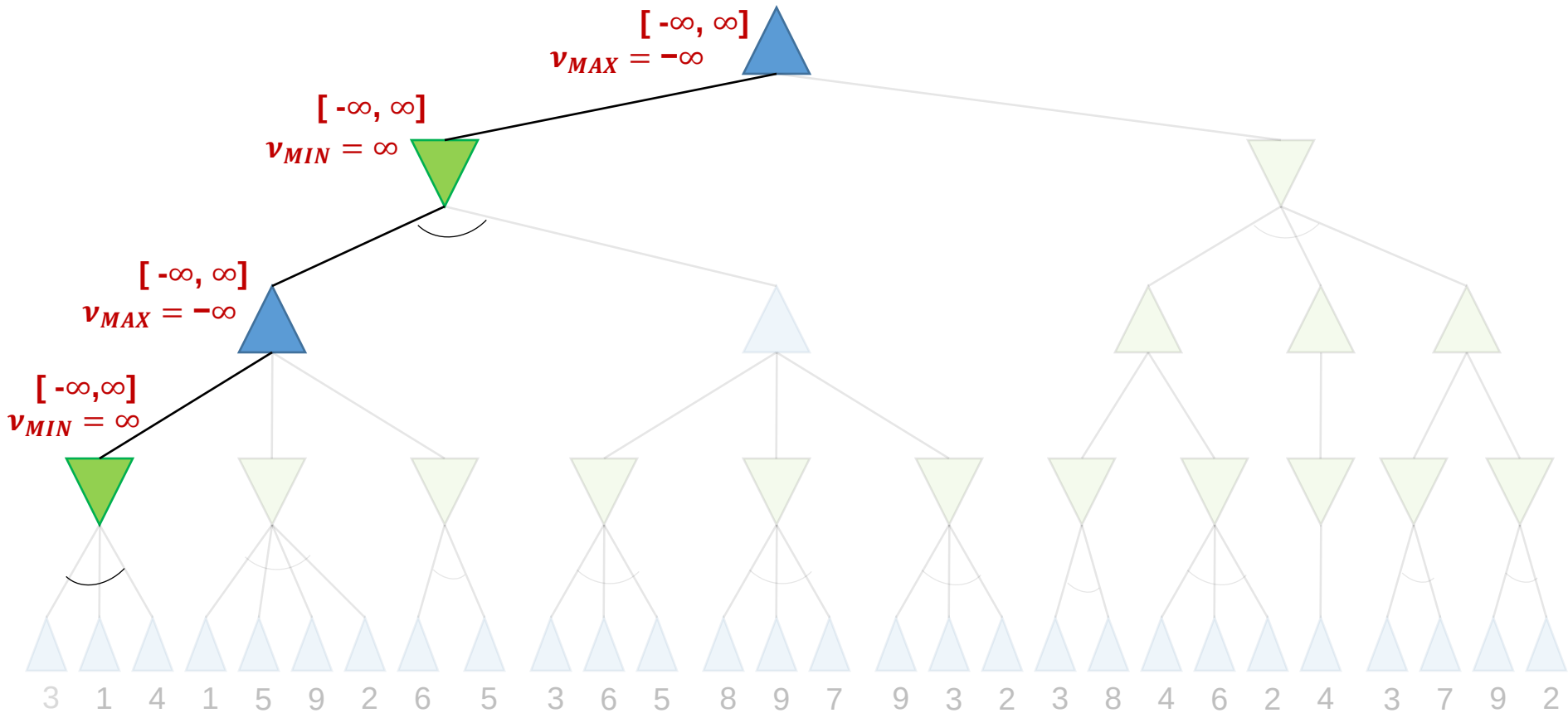
```
function MIN_VALUE (state,  $\alpha$ ,  $\beta$ ) returns a utility value
```

- if $\text{TERMINAL-TEST}(\text{state})$ then return $\text{UTILITY}(\text{state})$
- $v \leftarrow +\infty$
- for each action a in $\text{ACTIONS}(\text{state})$ do
 - $v \leftarrow \text{MIN}(v, \text{MAX_VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 - if $v \leq \alpha$ then return v // Pruning
 - $\beta \leftarrow \text{MIN}(\beta, v)$
- return v

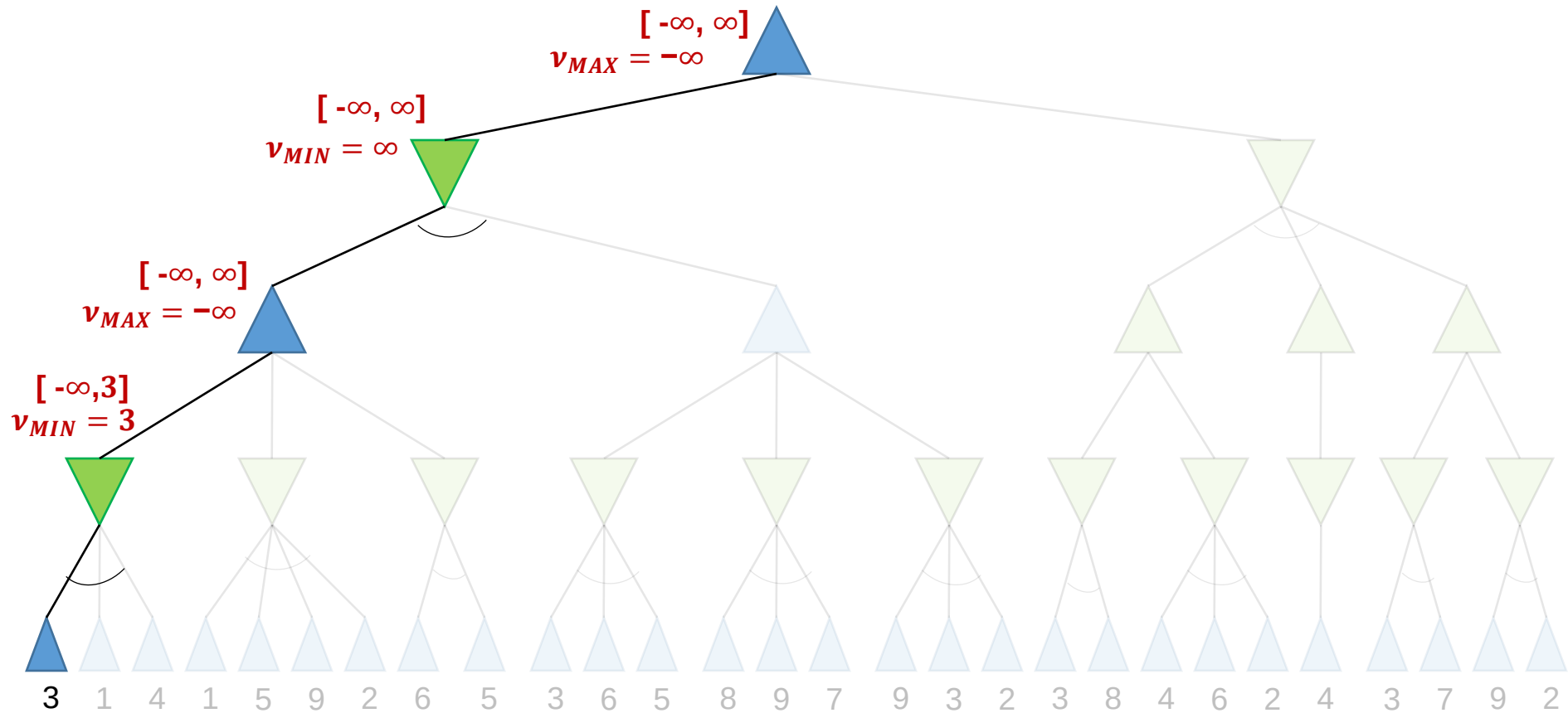
Ex: alpha-beta pruning



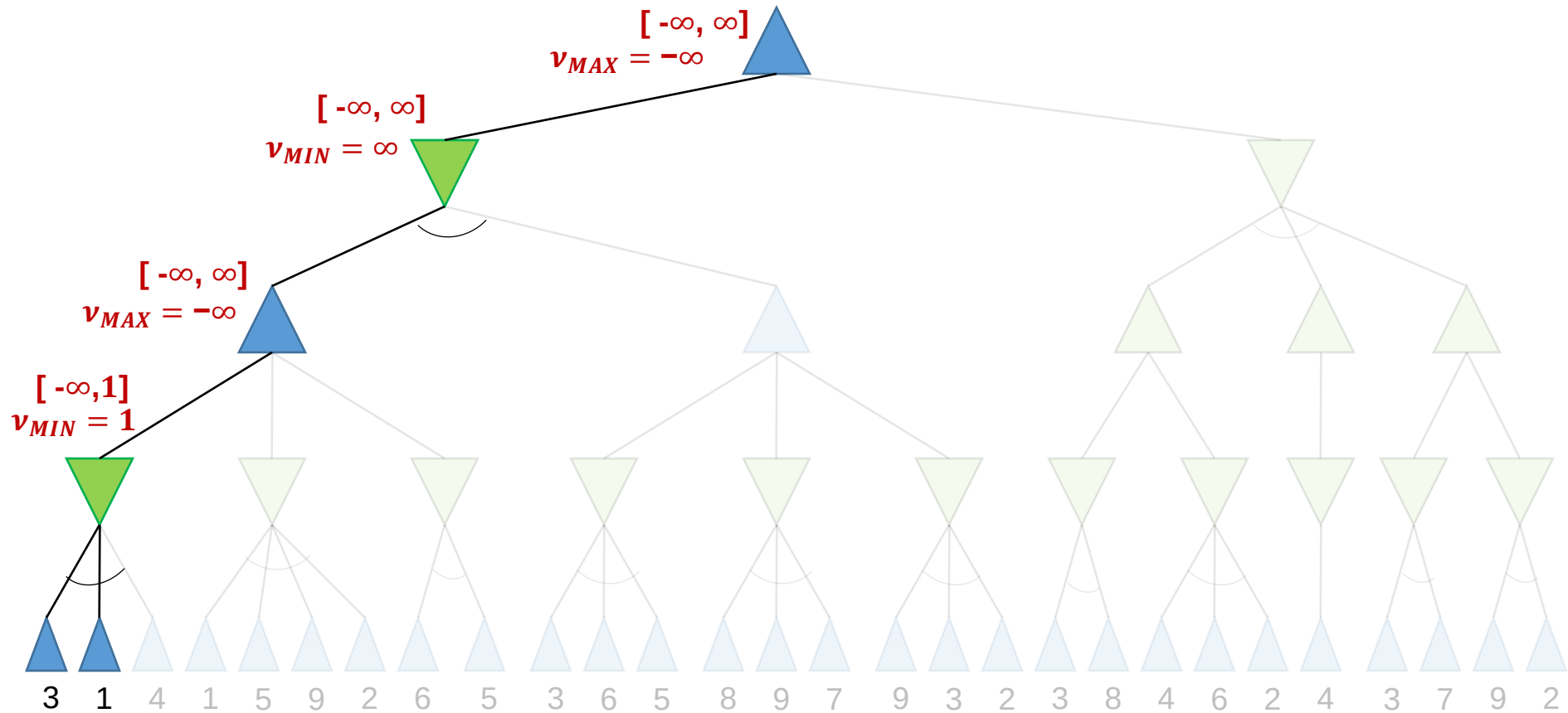
Ex: alpha-beta pruning



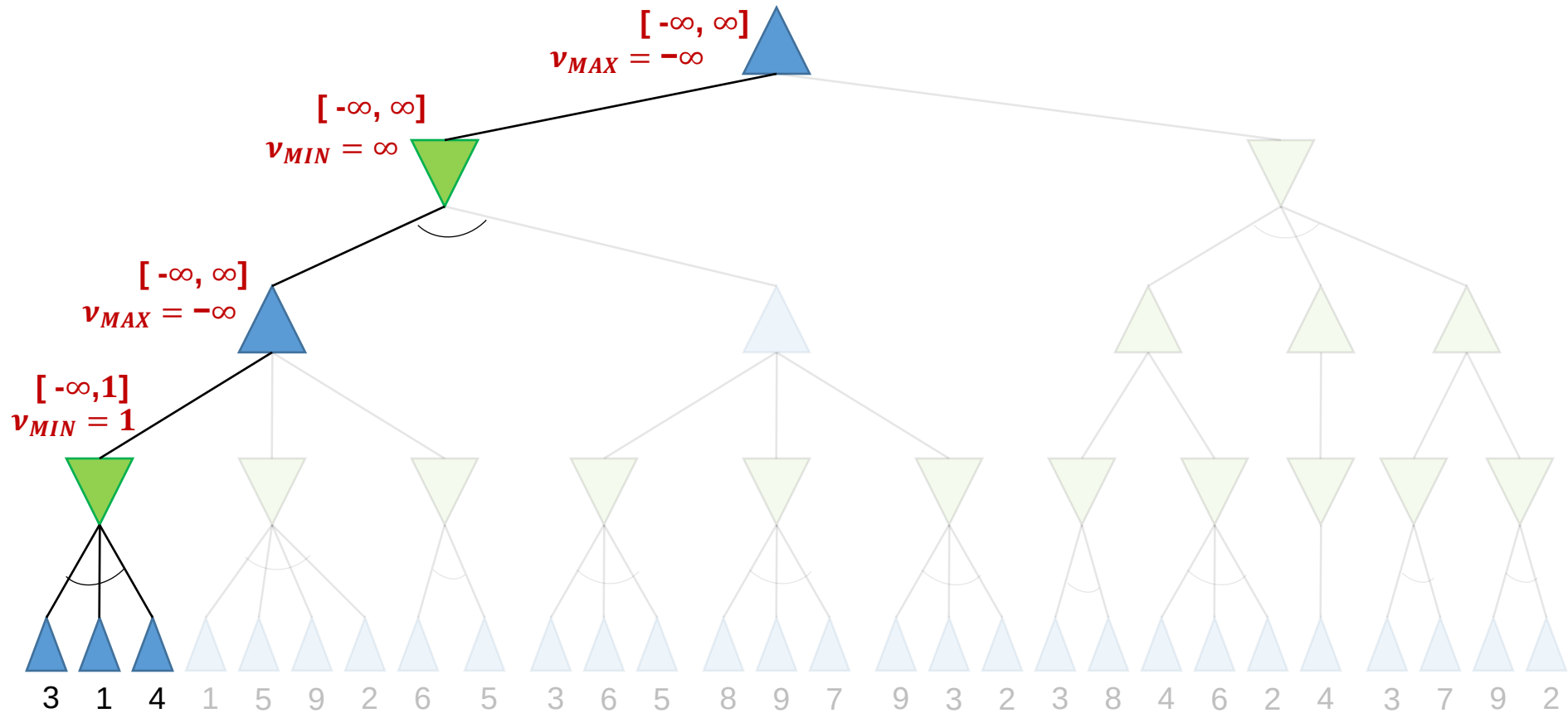
Ex: alpha-beta pruning



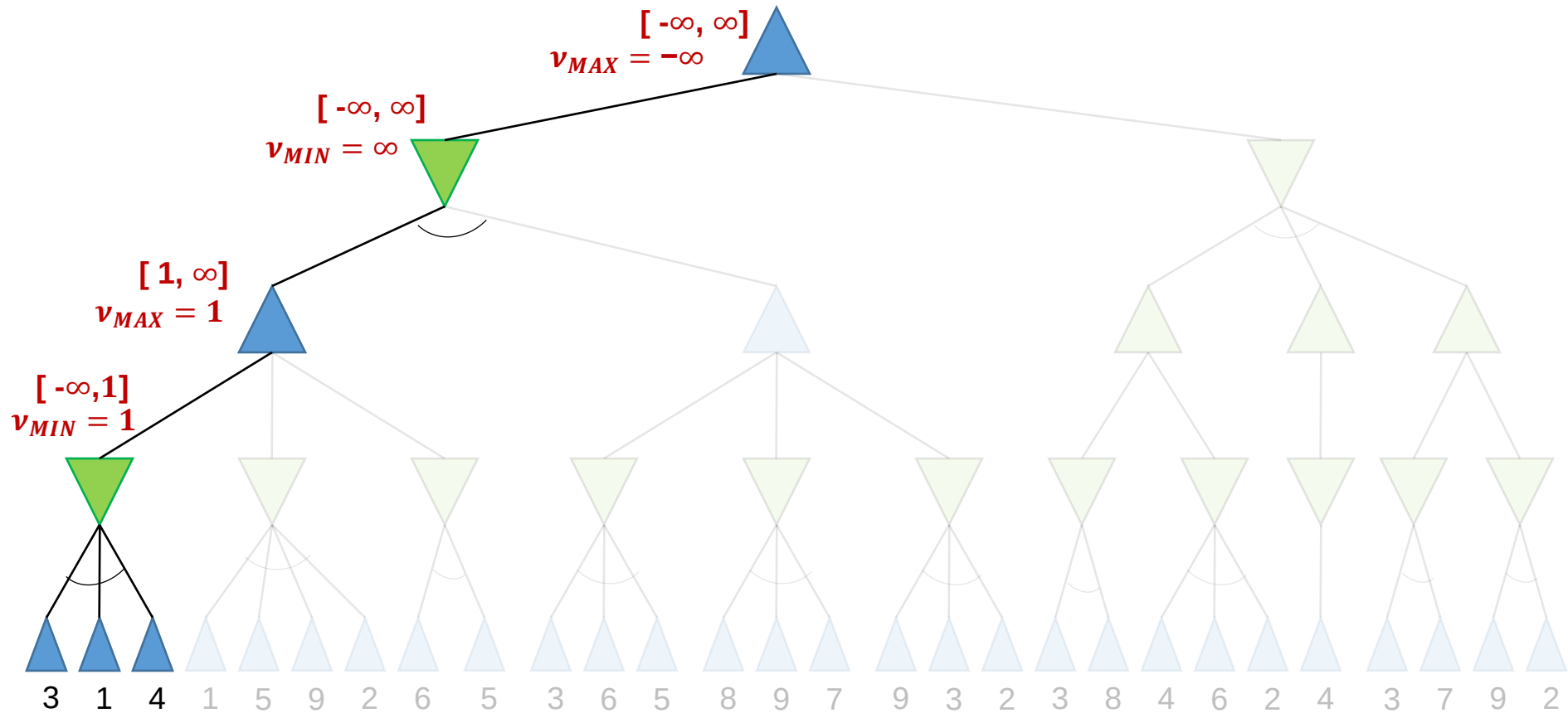
Ex: alpha-beta pruning



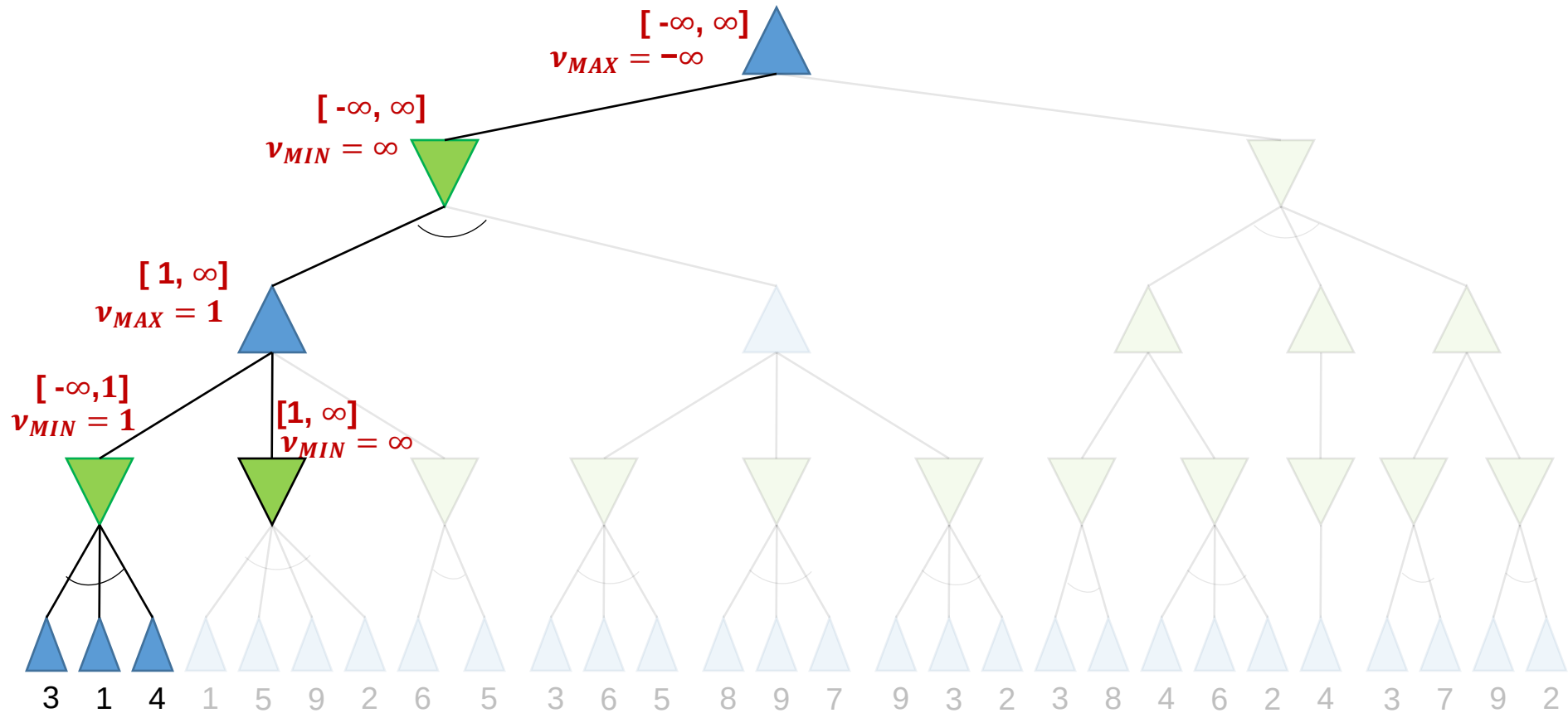
Ex: alpha-beta pruning



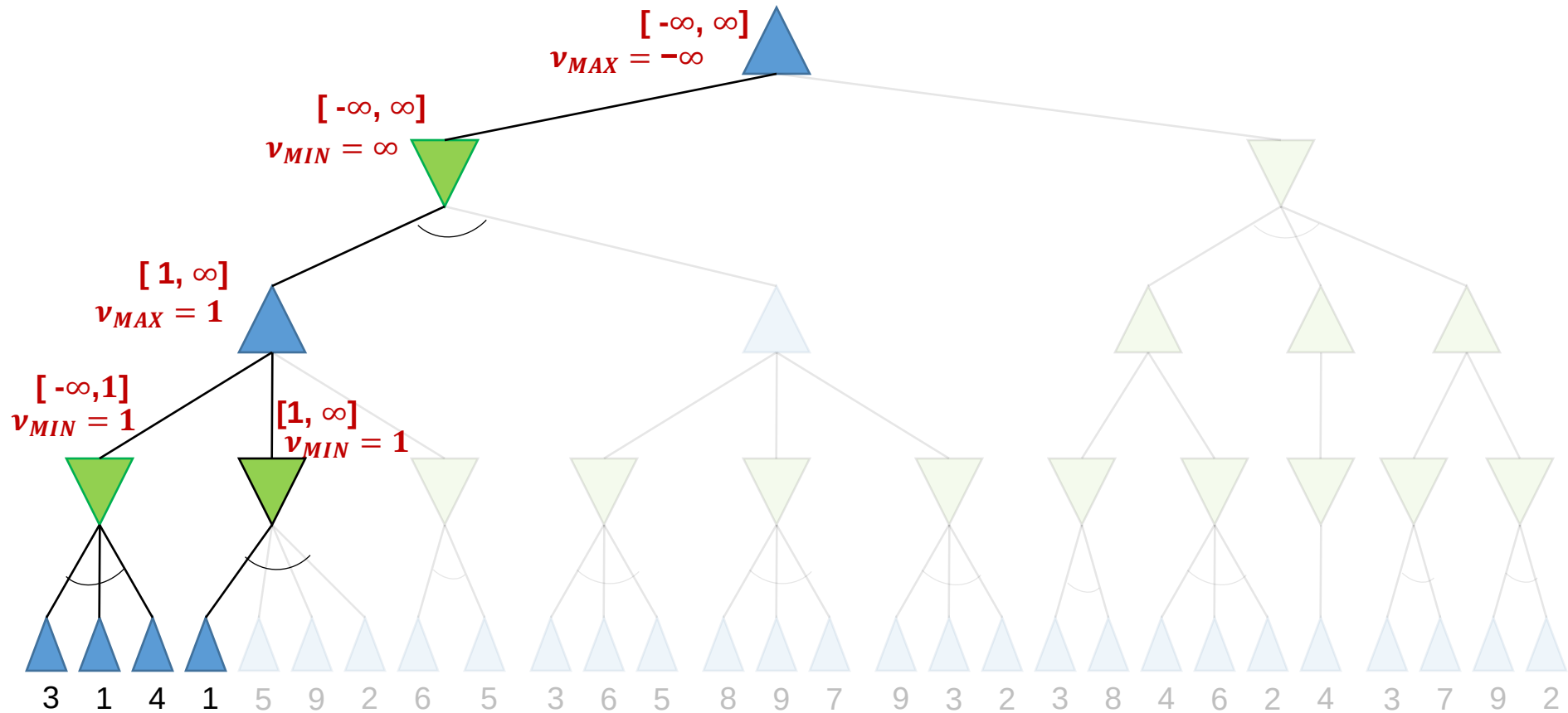
Ex: alpha-beta pruning



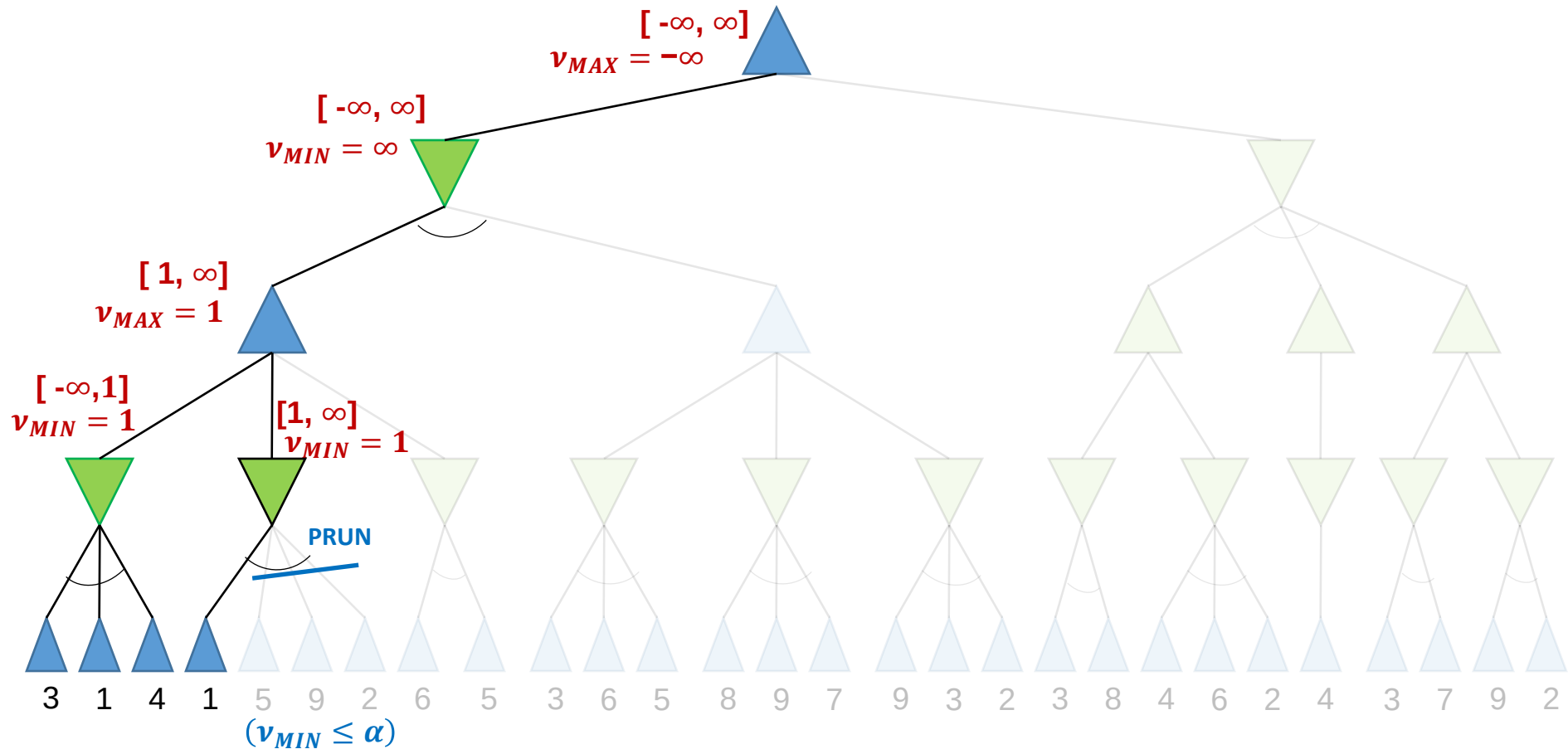
Ex: alpha-beta pruning



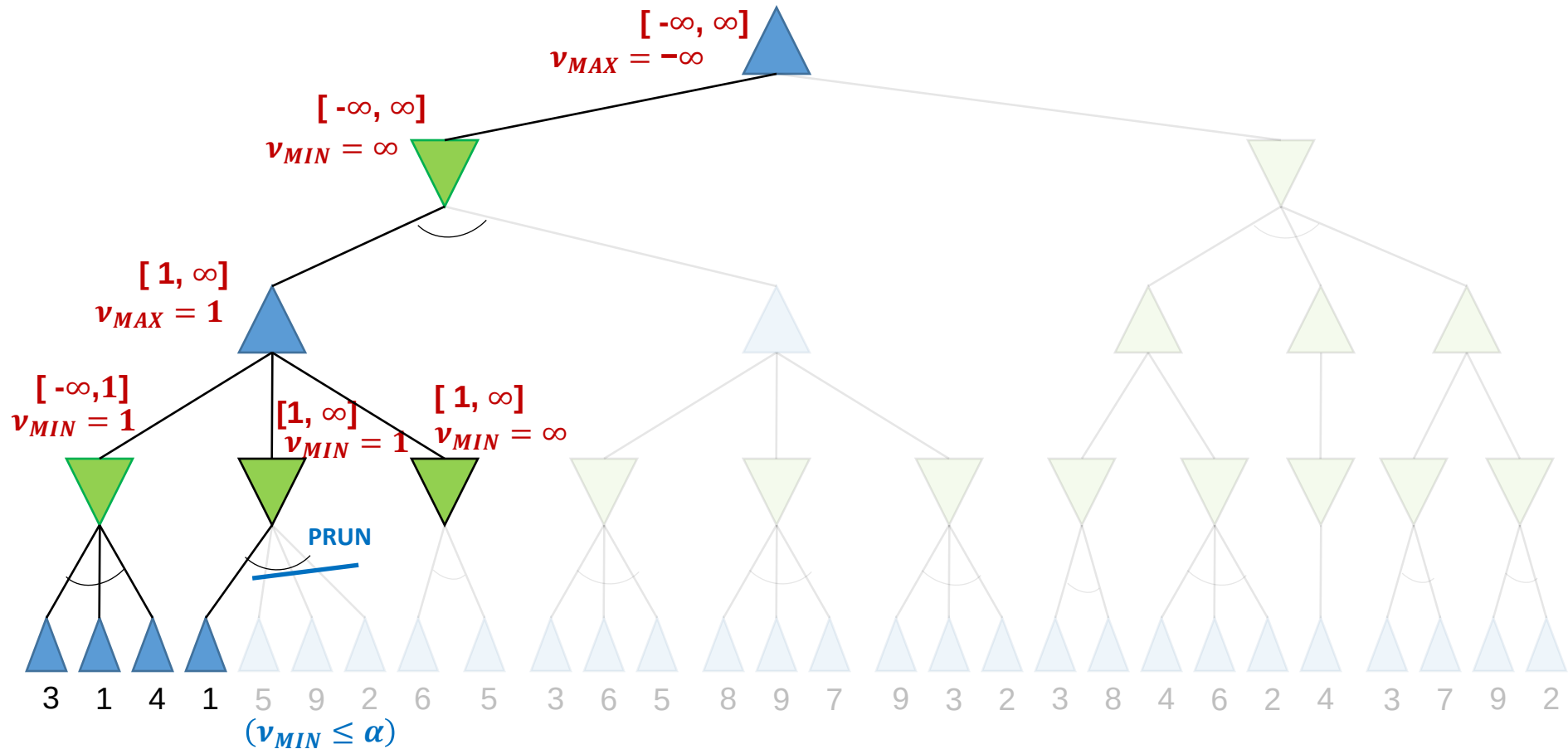
Ex: alpha-beta pruning



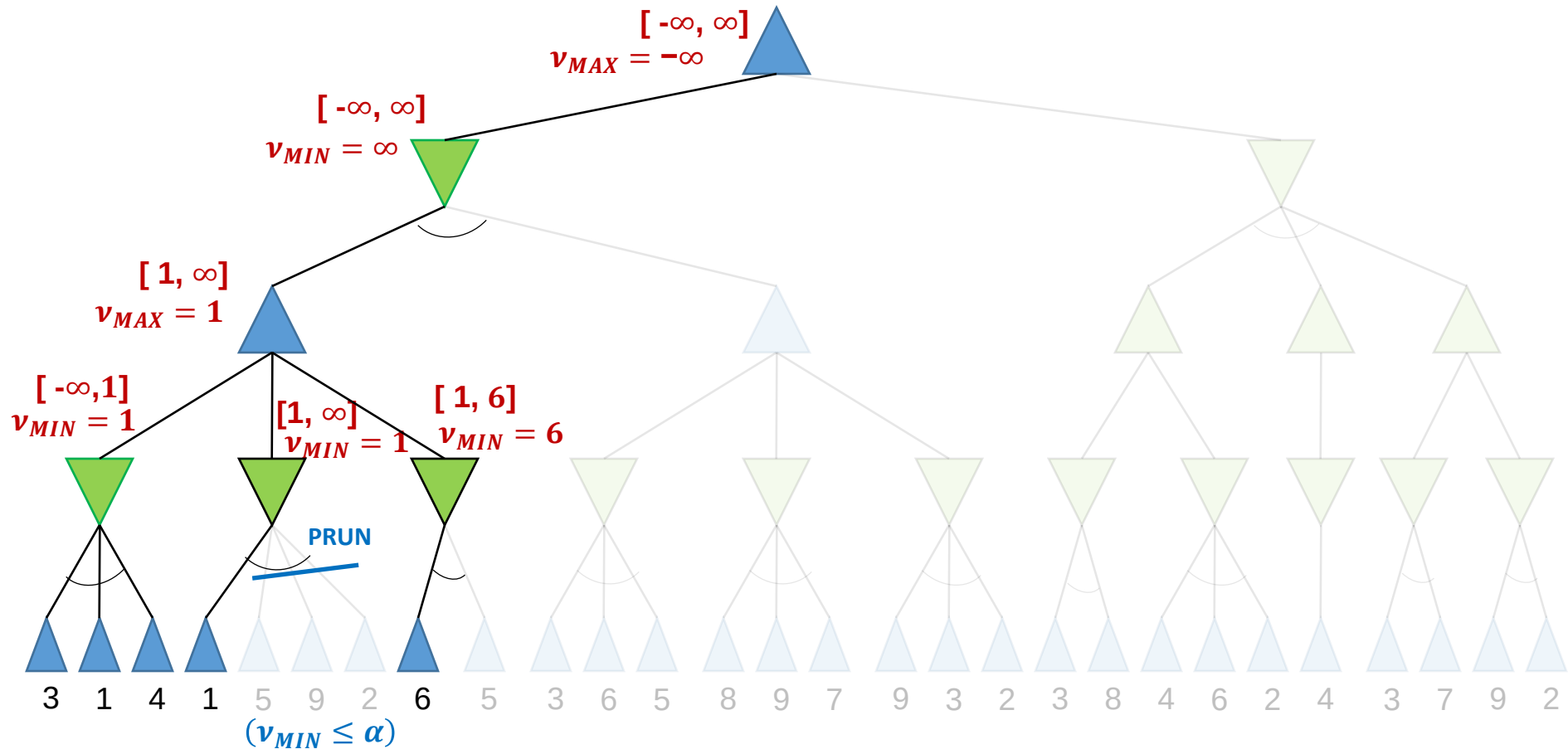
Ex: alpha-beta pruning



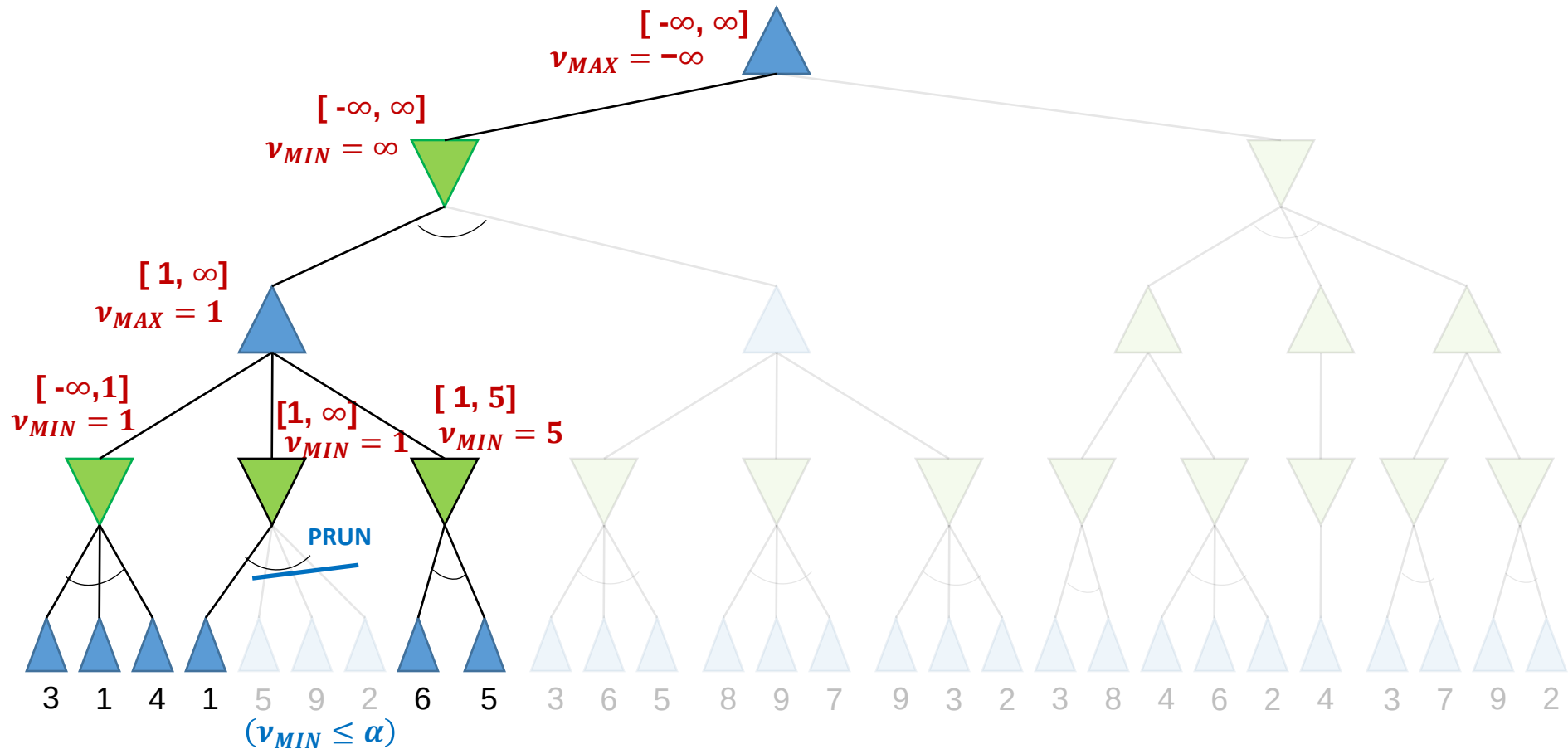
Ex: alpha-beta pruning



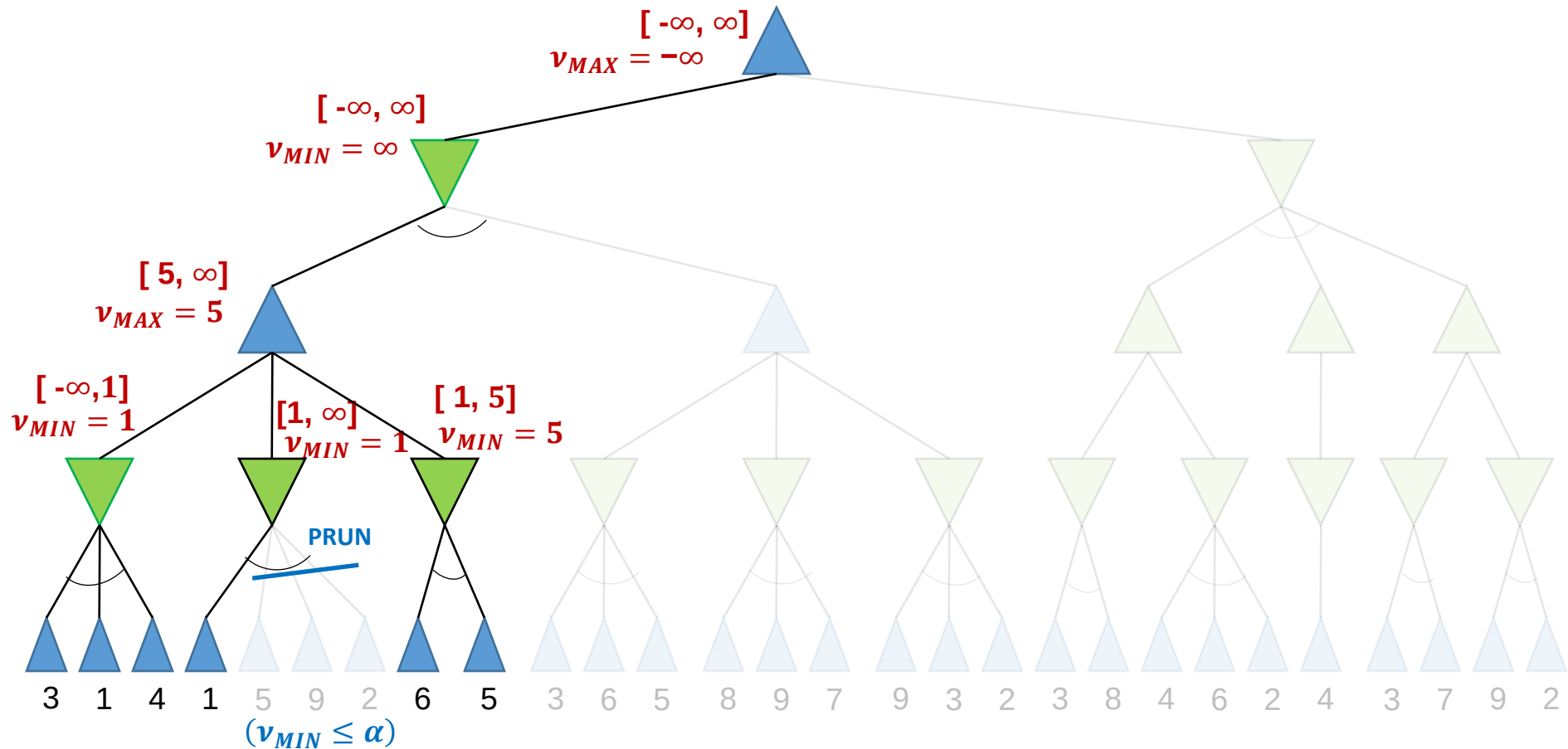
Ex: alpha-beta pruning



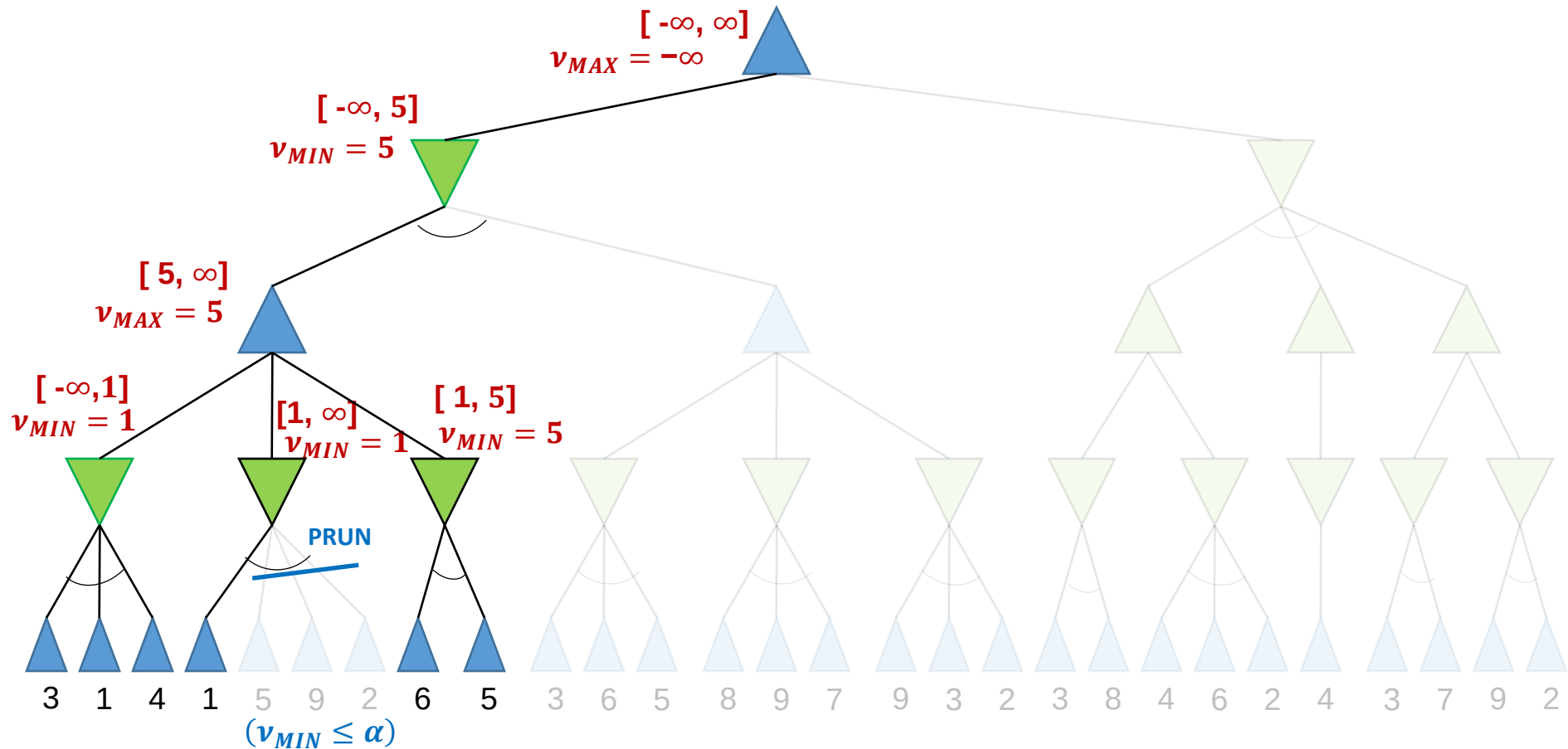
Ex: alpha-beta pruning



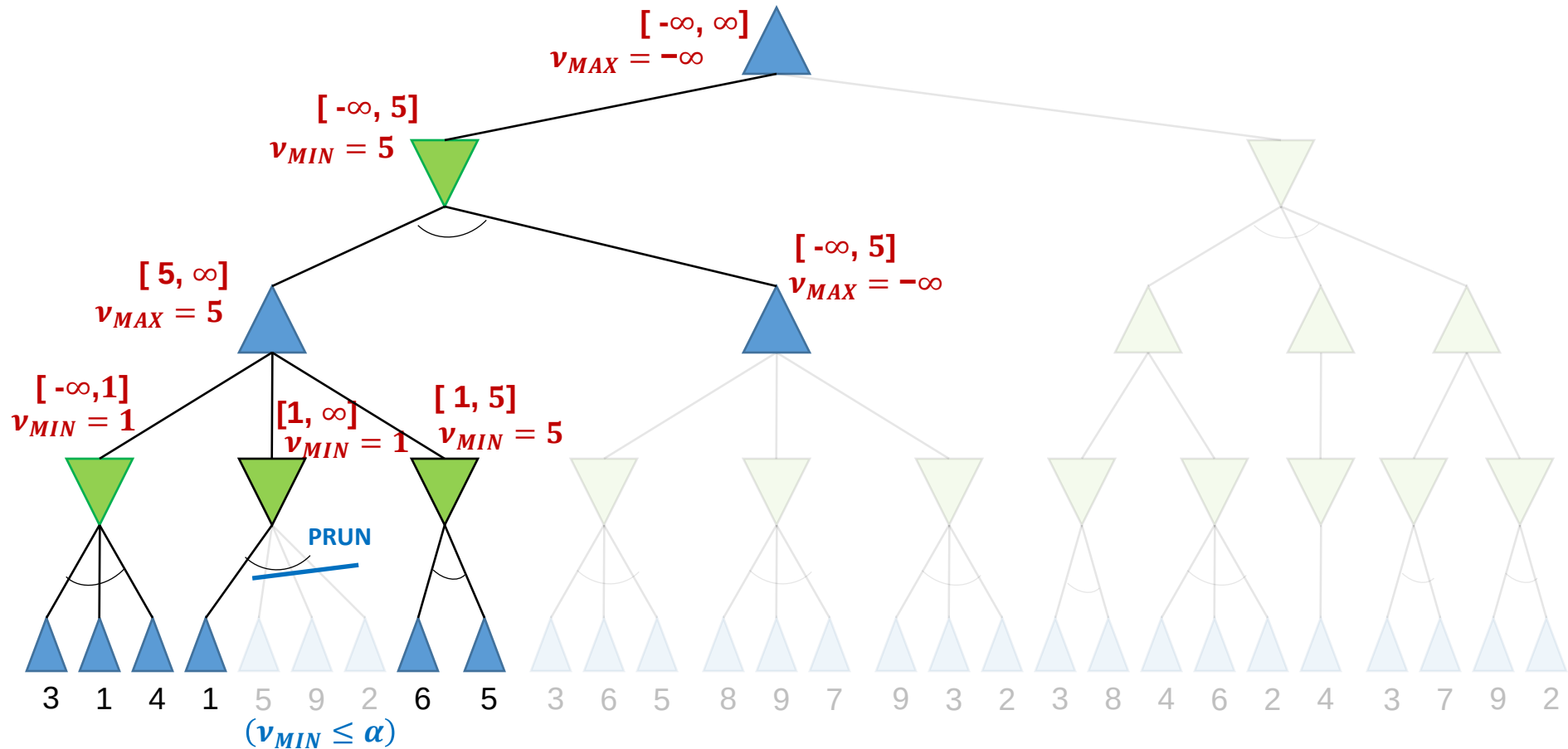
Ex: alpha-beta pruning



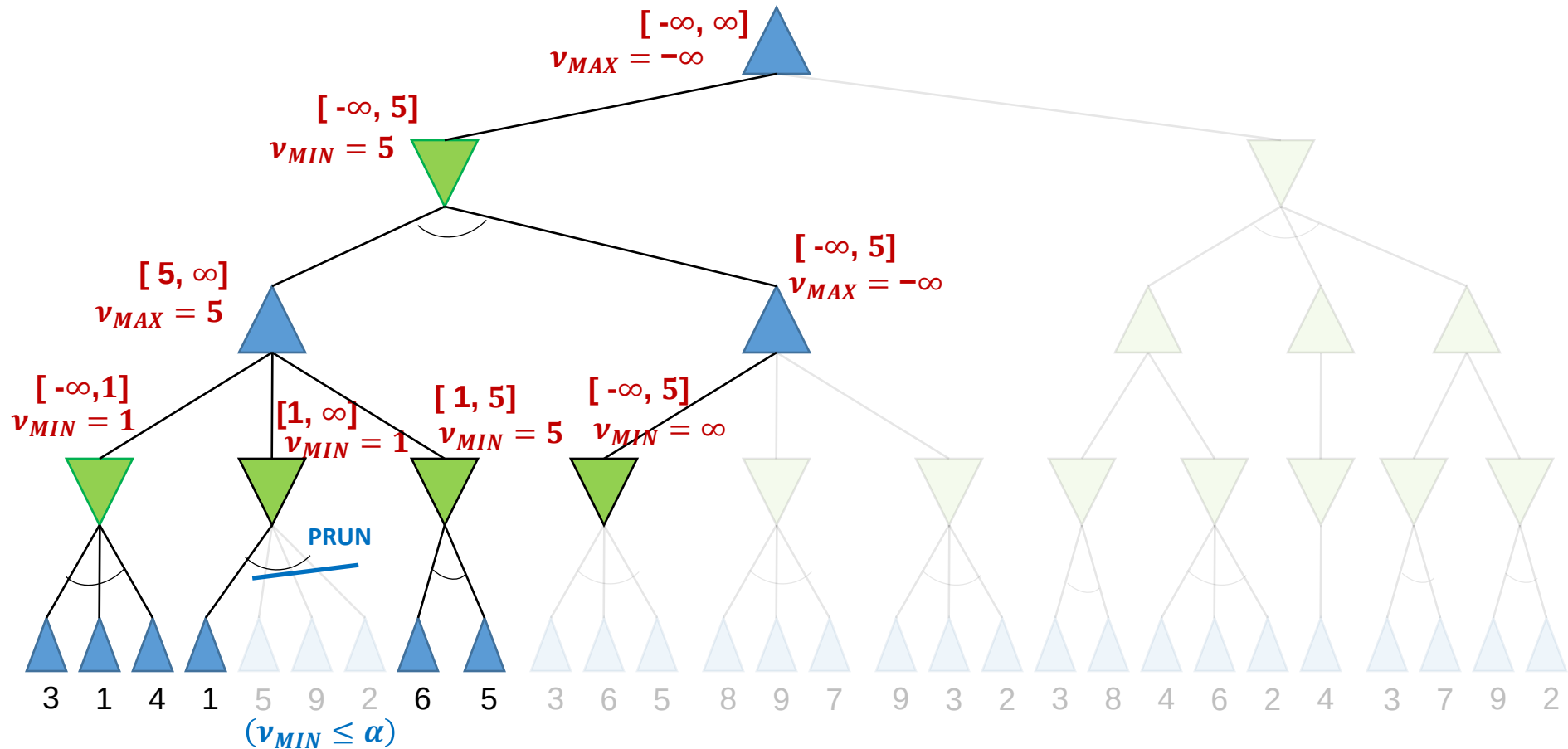
Ex: alpha-beta pruning



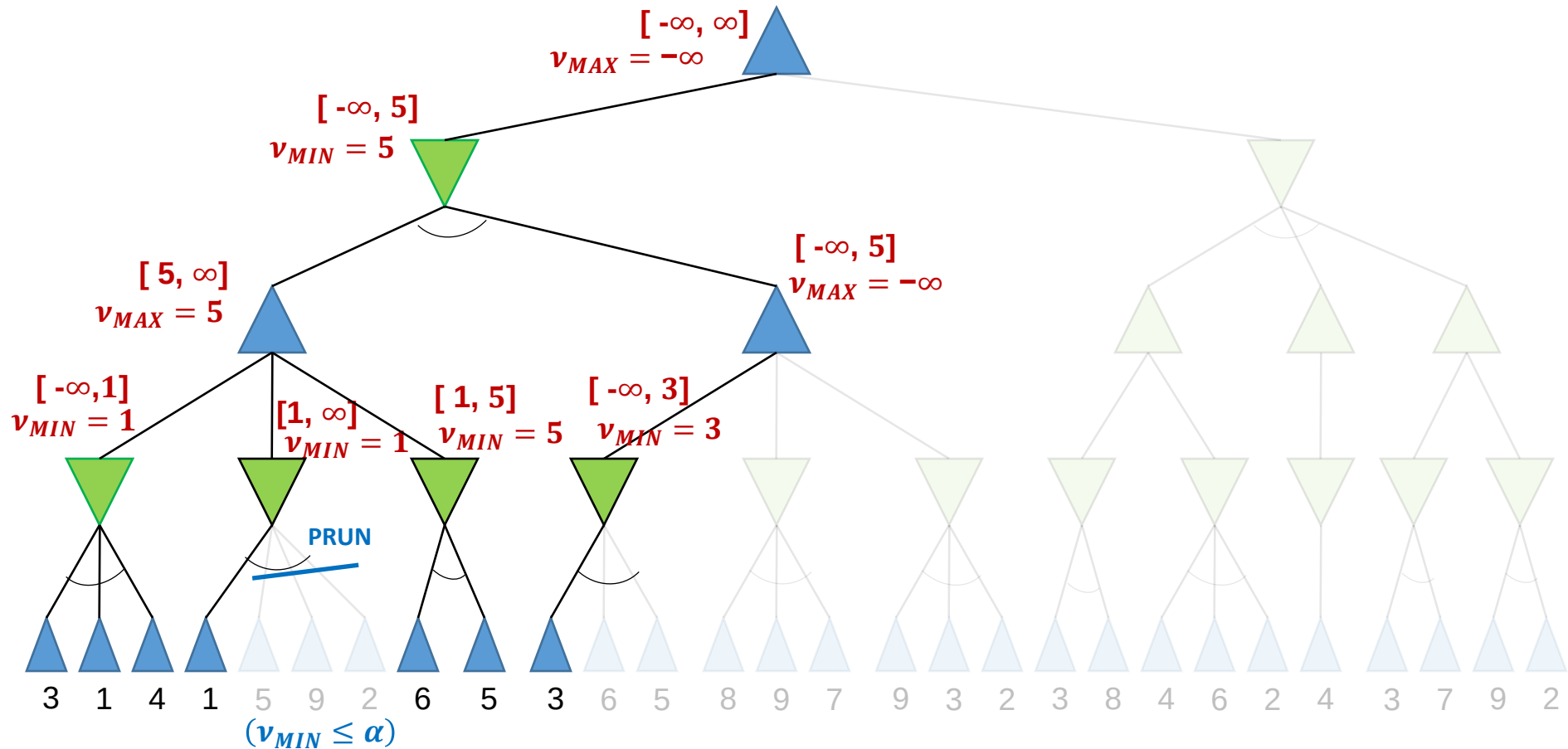
Ex: alpha-beta pruning



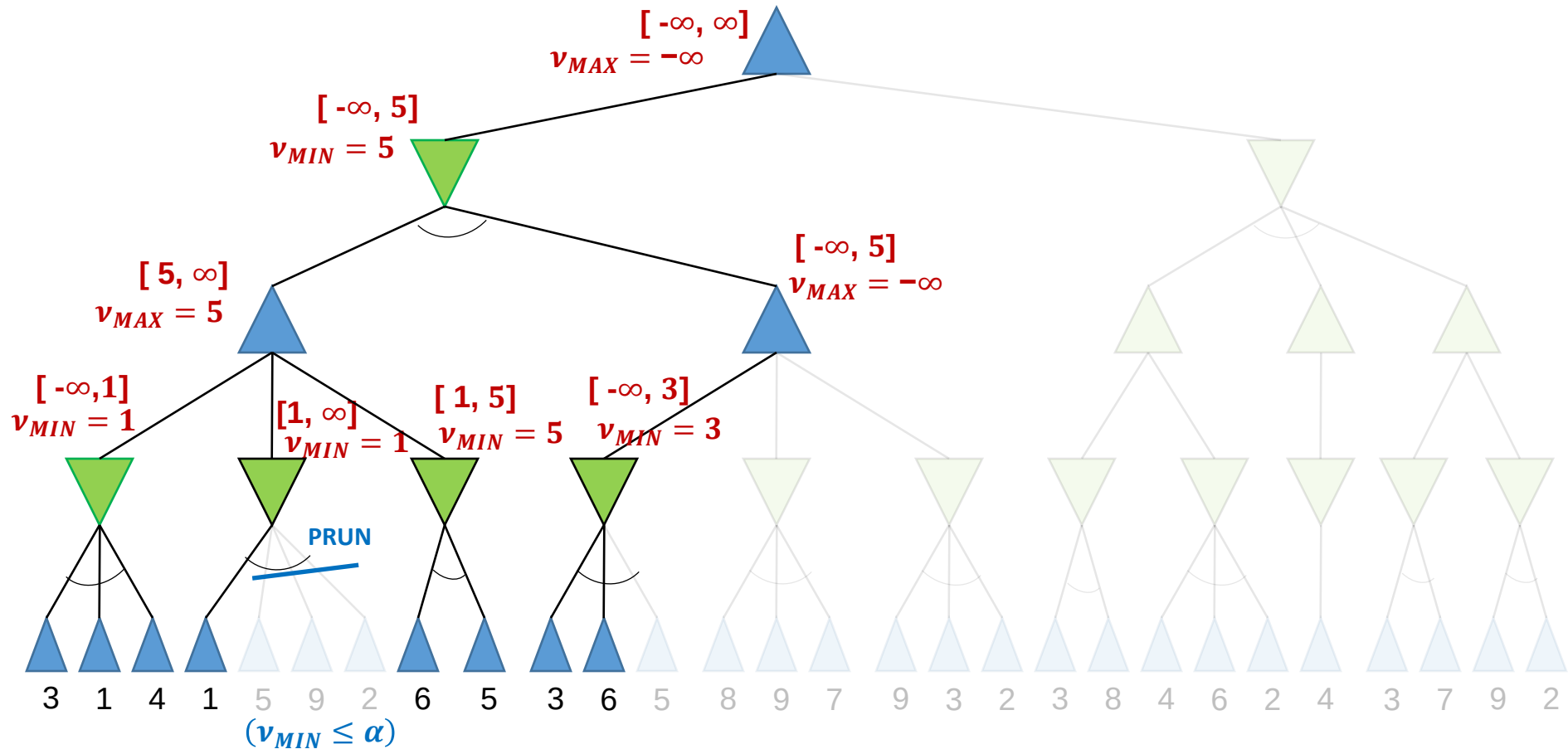
Ex: alpha-beta pruning



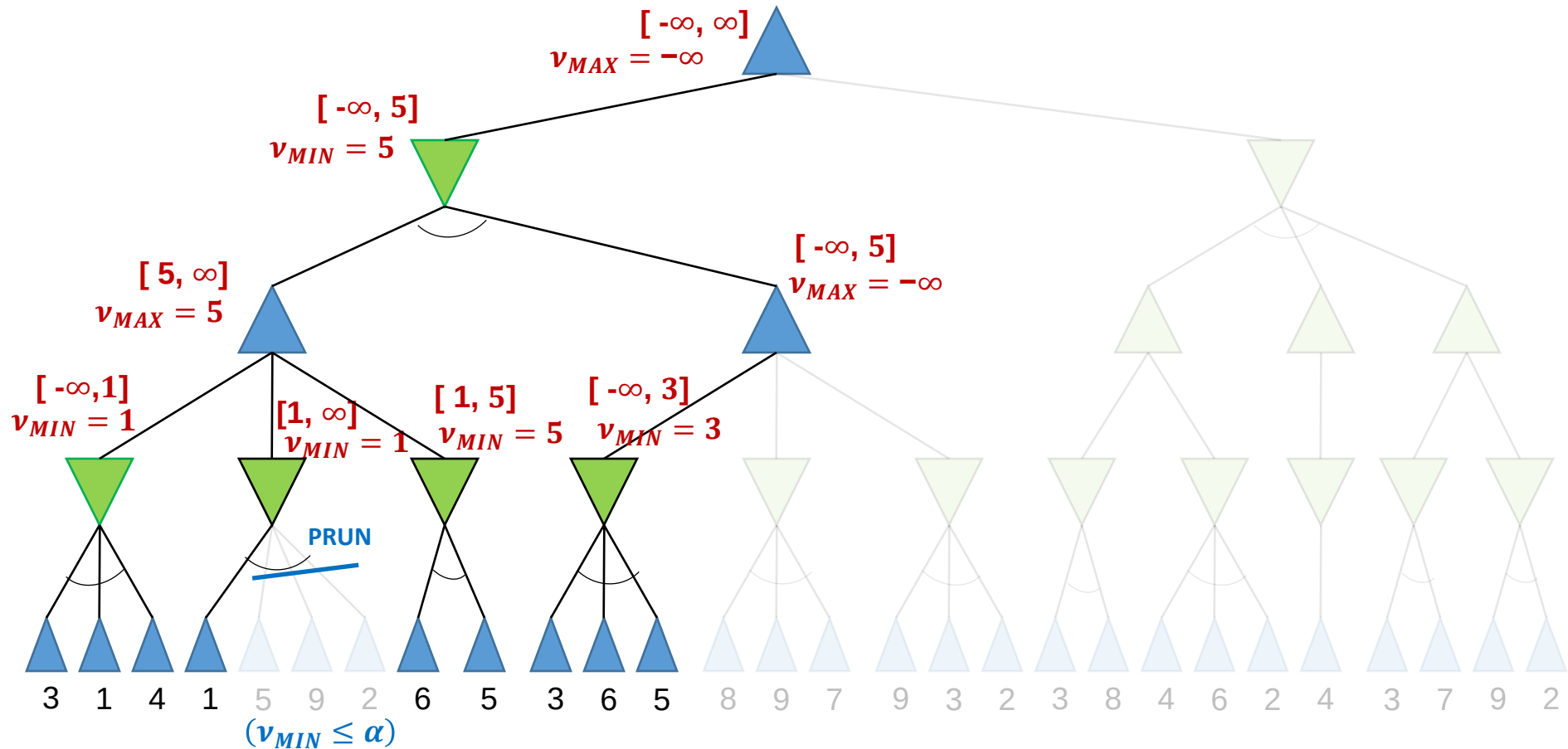
Ex: alpha-beta pruning



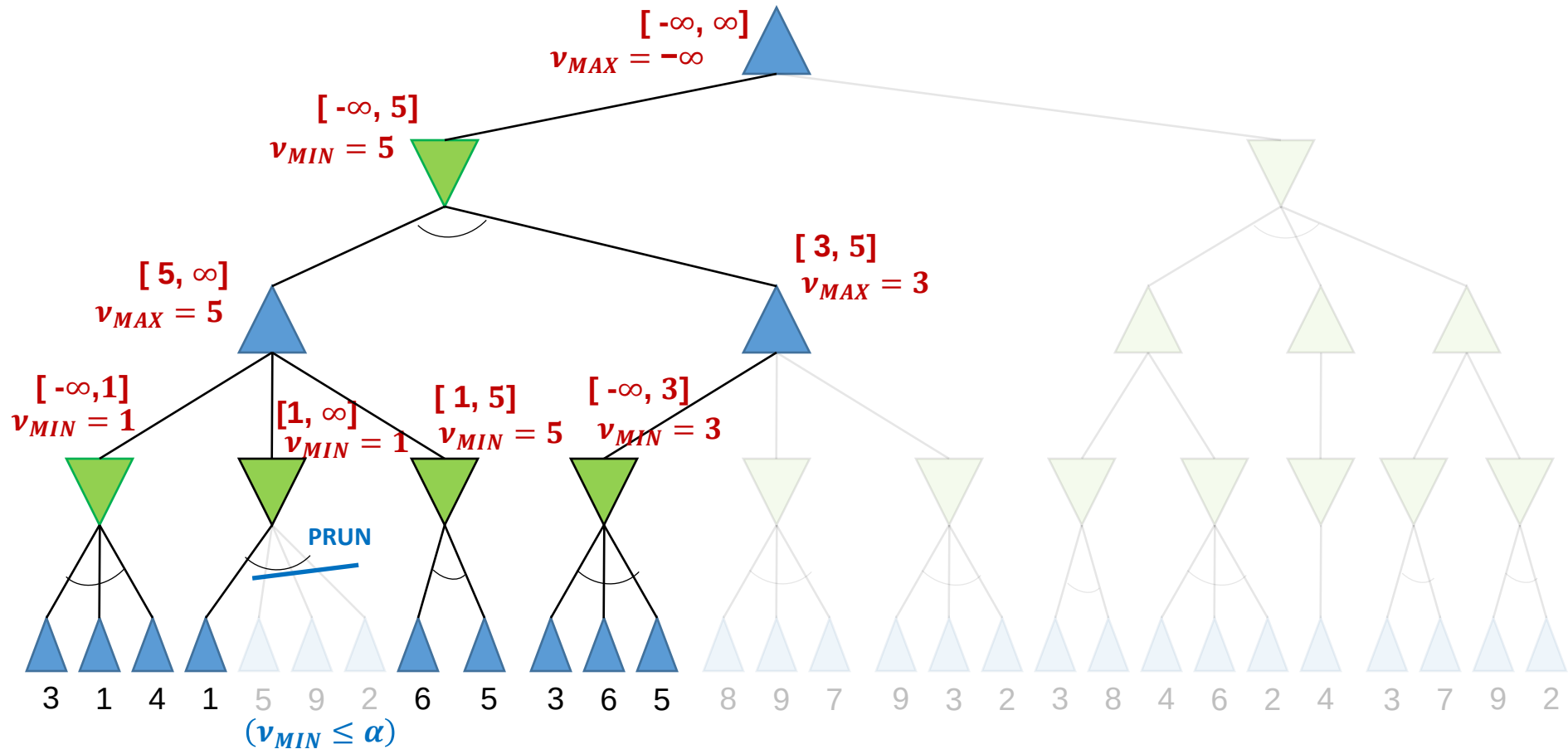
Ex: alpha-beta pruning



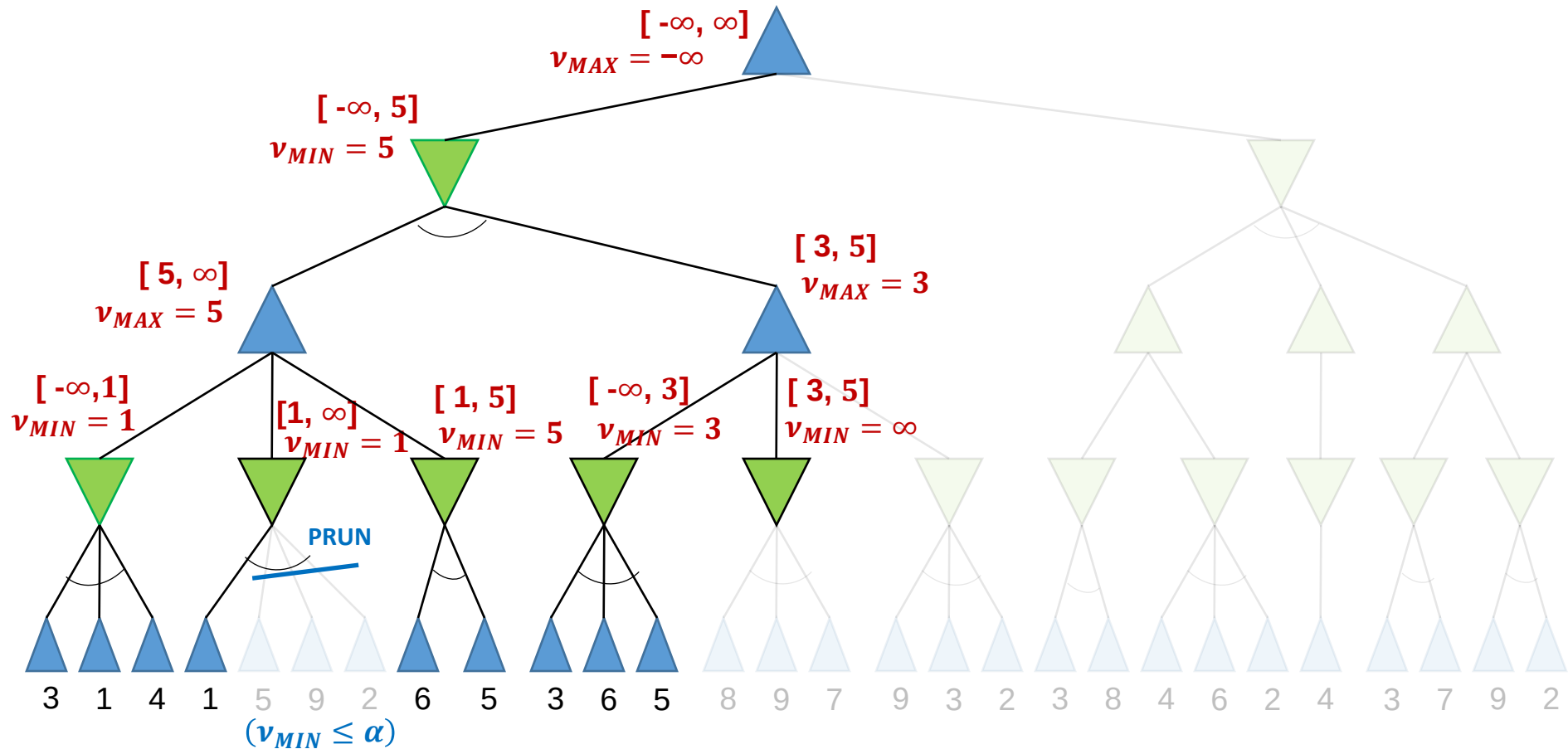
Ex: alpha-beta pruning



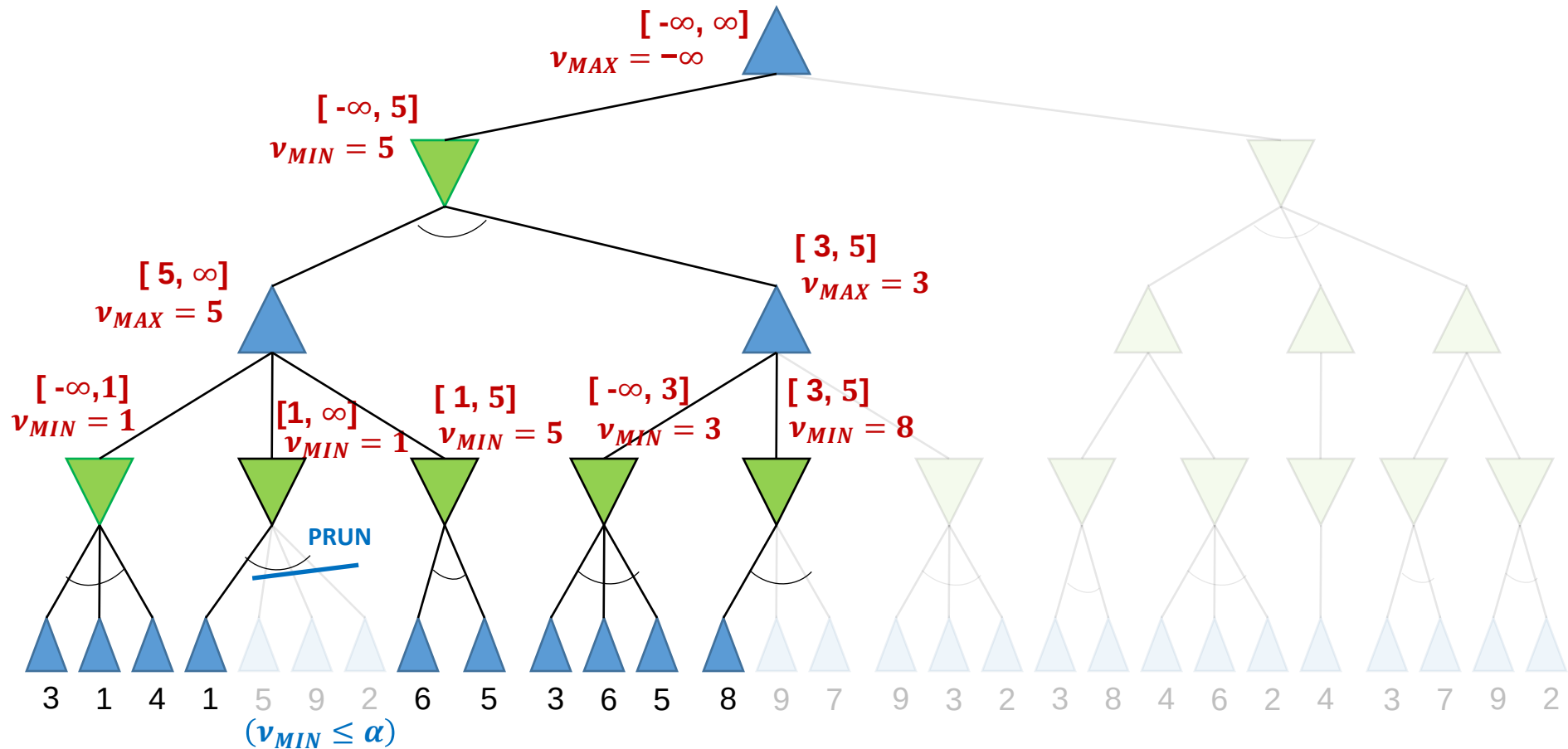
Ex: alpha-beta pruning



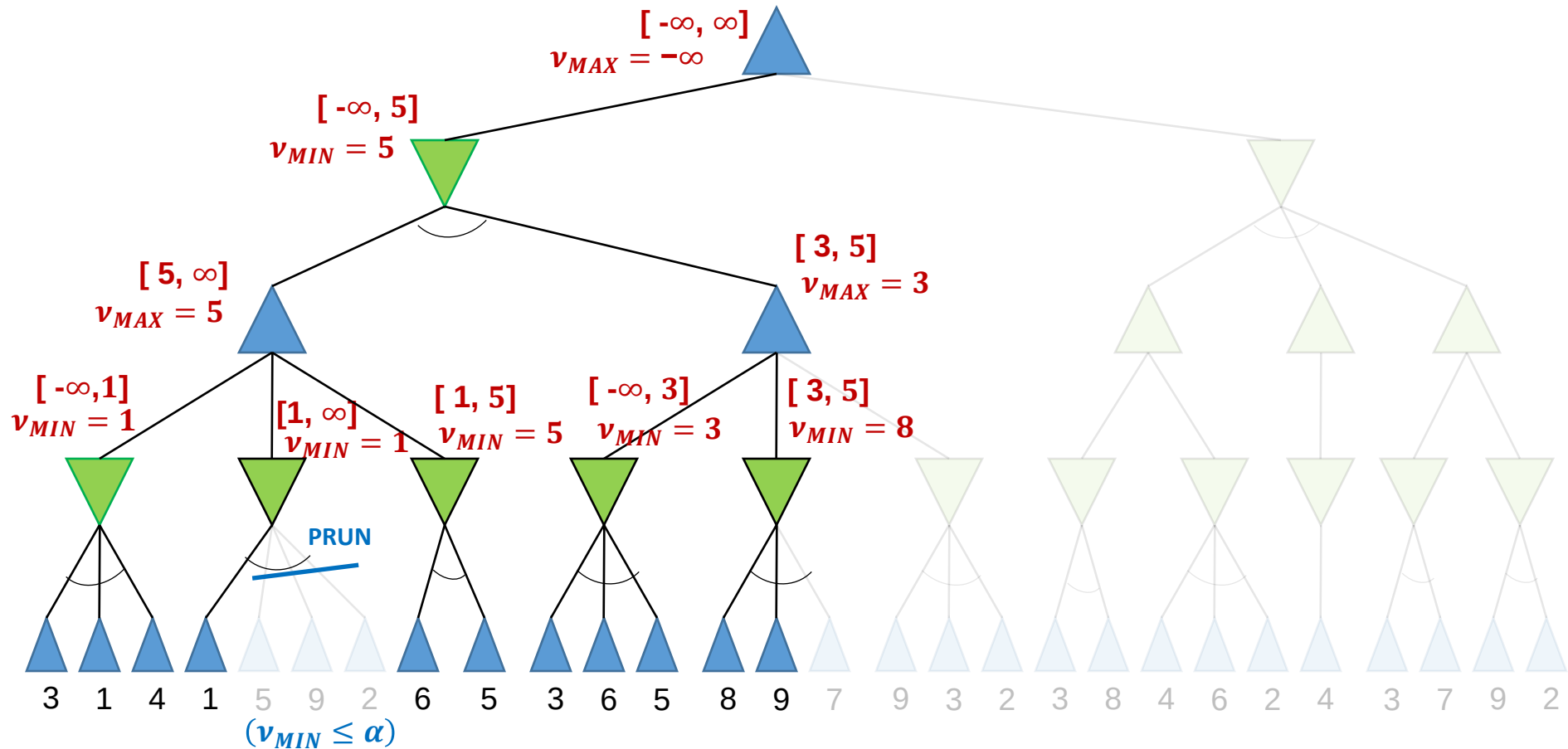
Ex: alpha-beta pruning



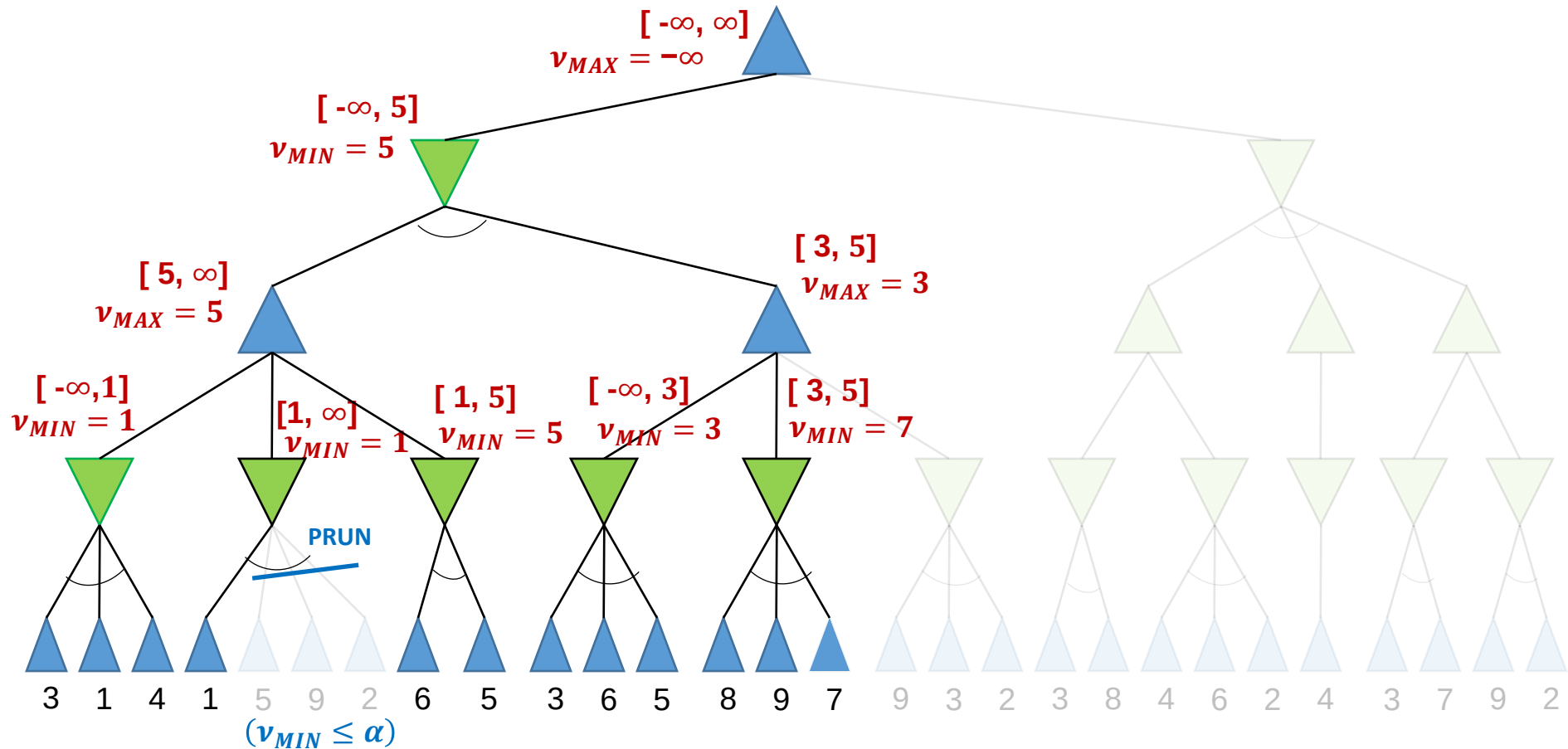
Ex: alpha-beta pruning



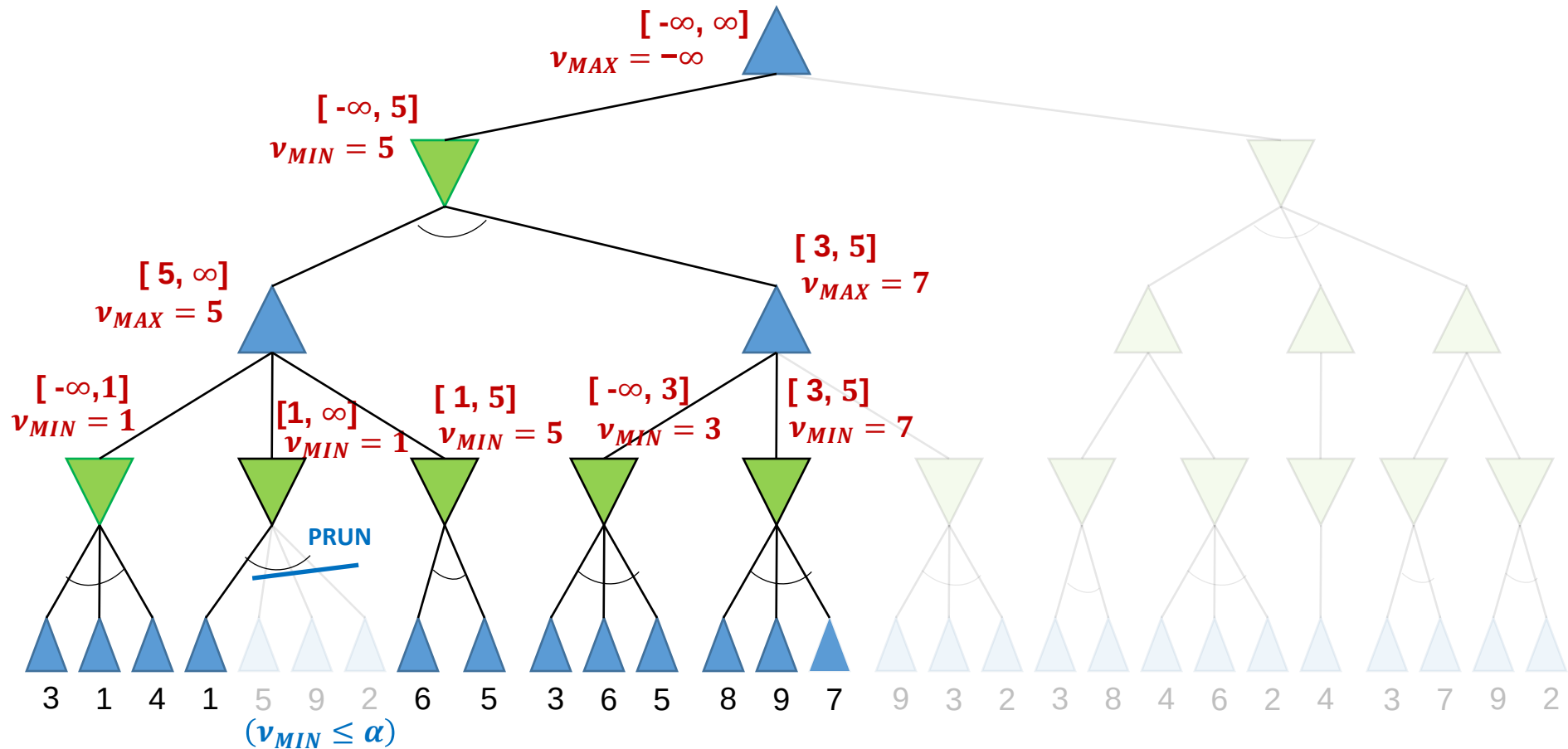
Ex: alpha-beta pruning



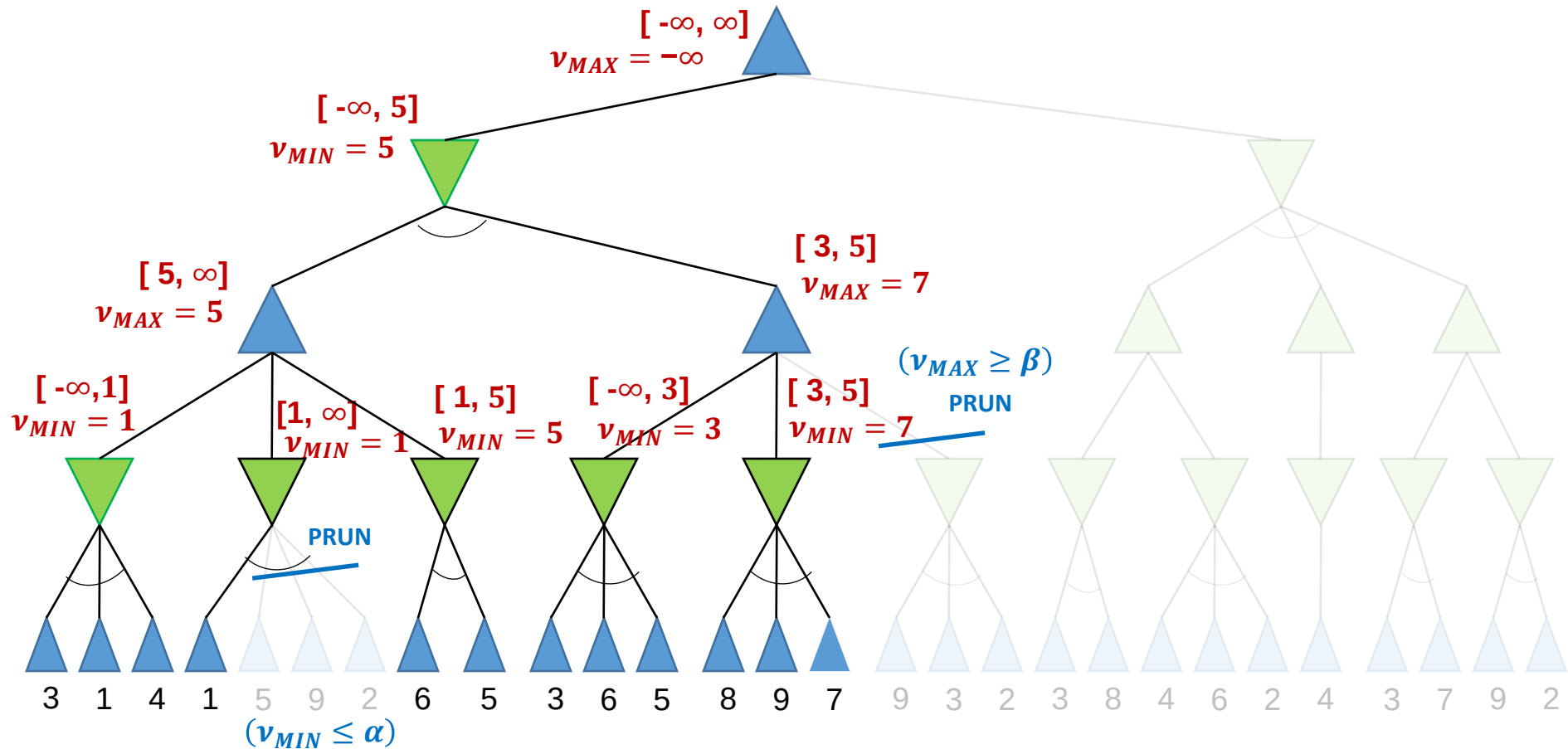
Ex: alpha-beta pruning



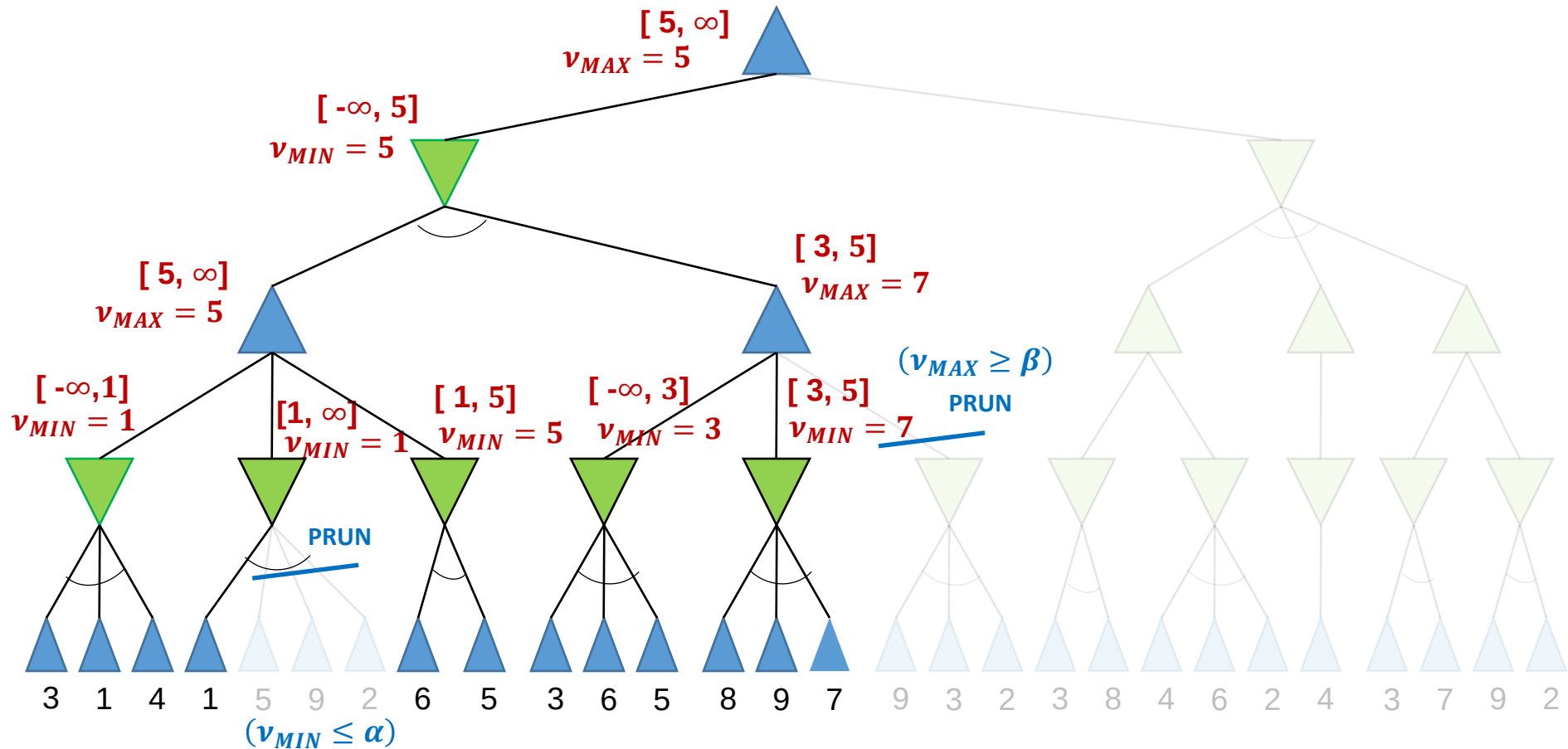
Ex: alpha-beta pruning



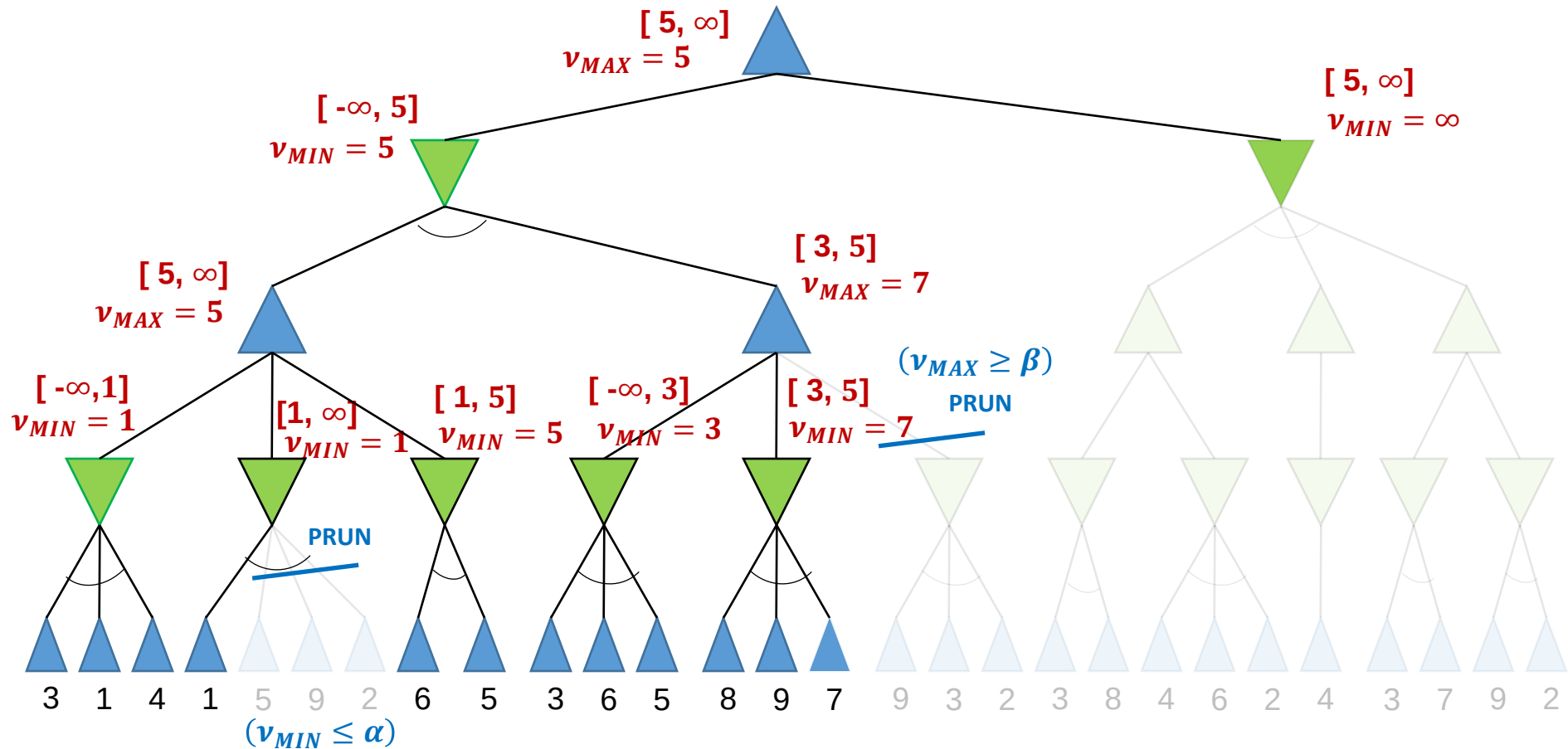
Ex: alpha-beta pruning



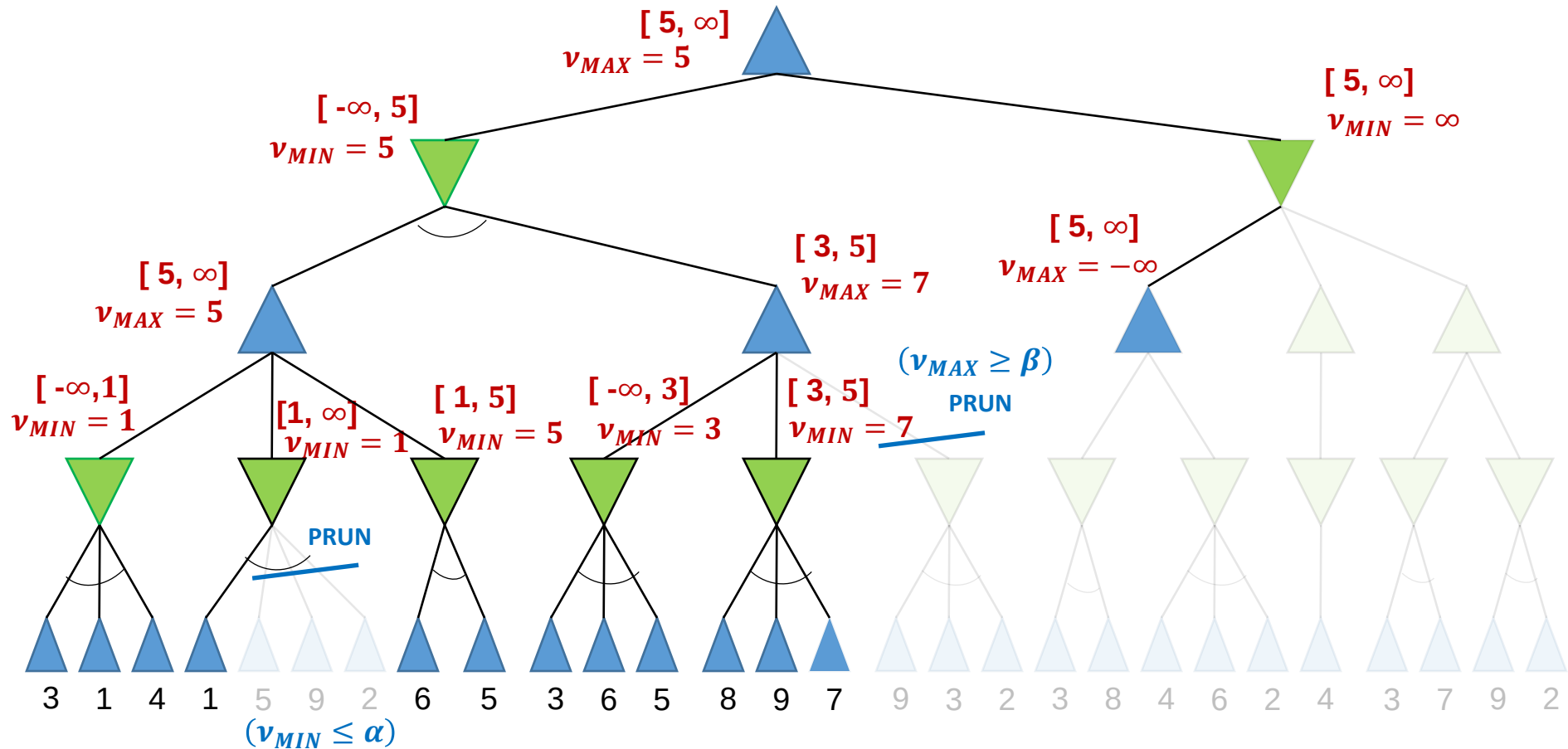
Ex: alpha-beta pruning



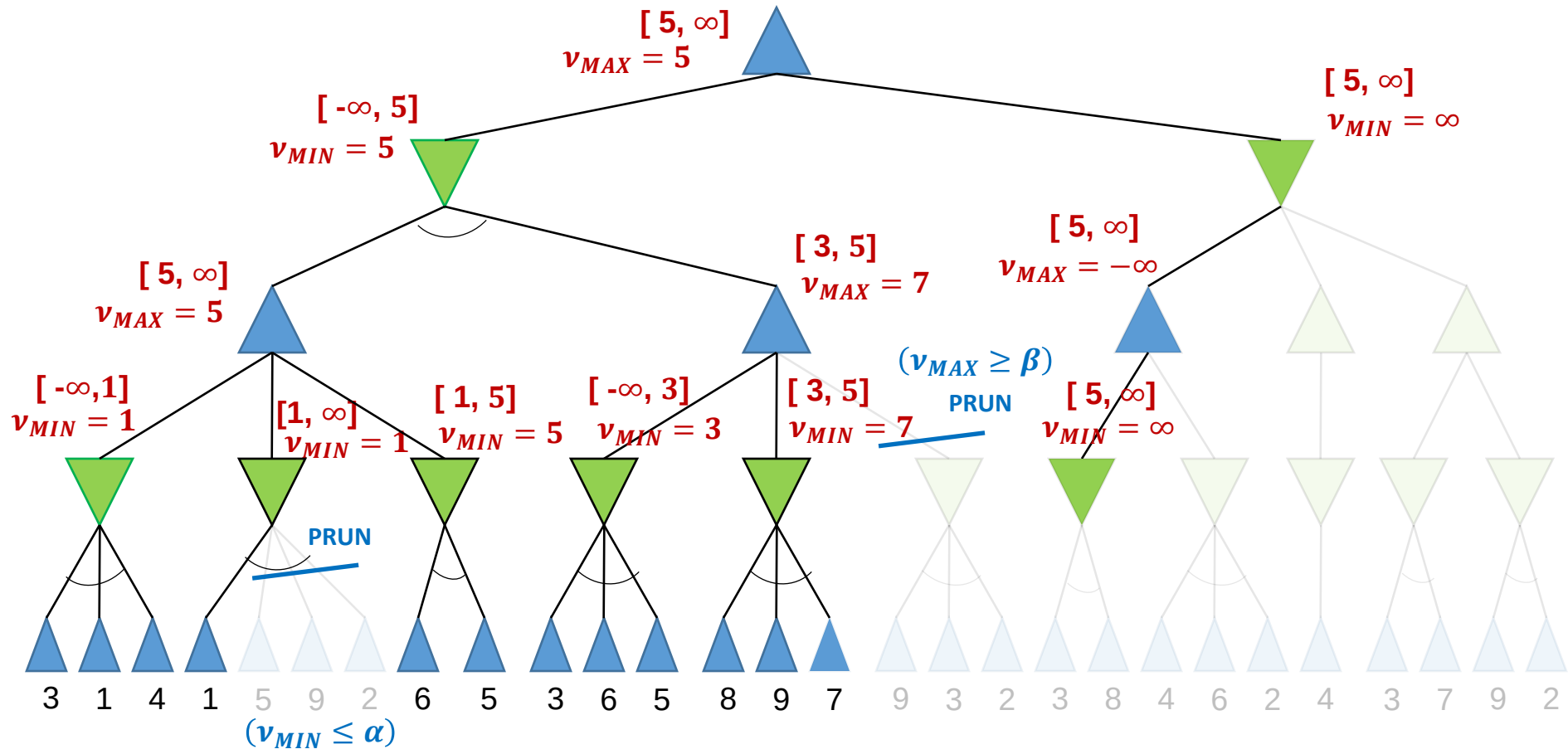
Ex: alpha-beta pruning



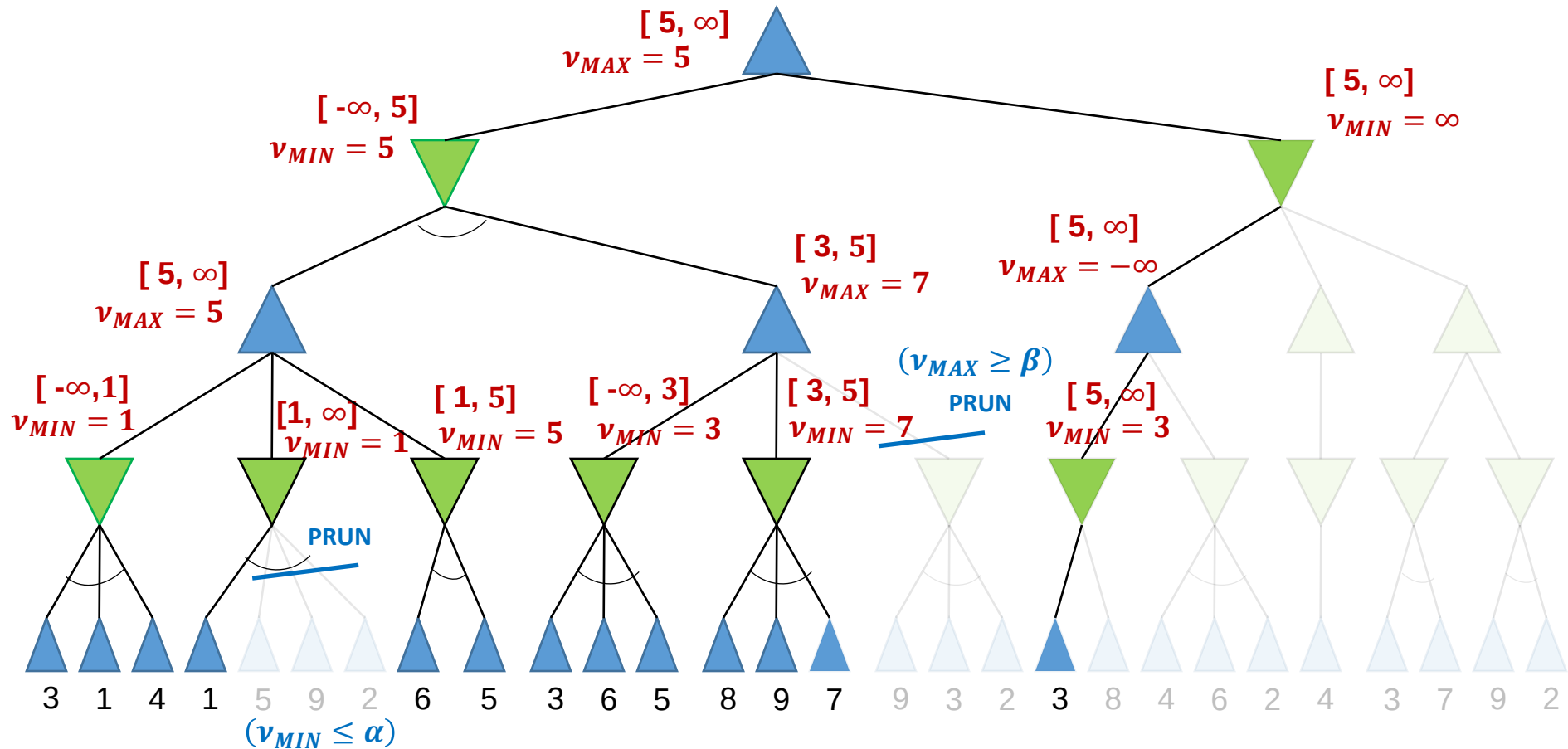
Ex: alpha-beta pruning



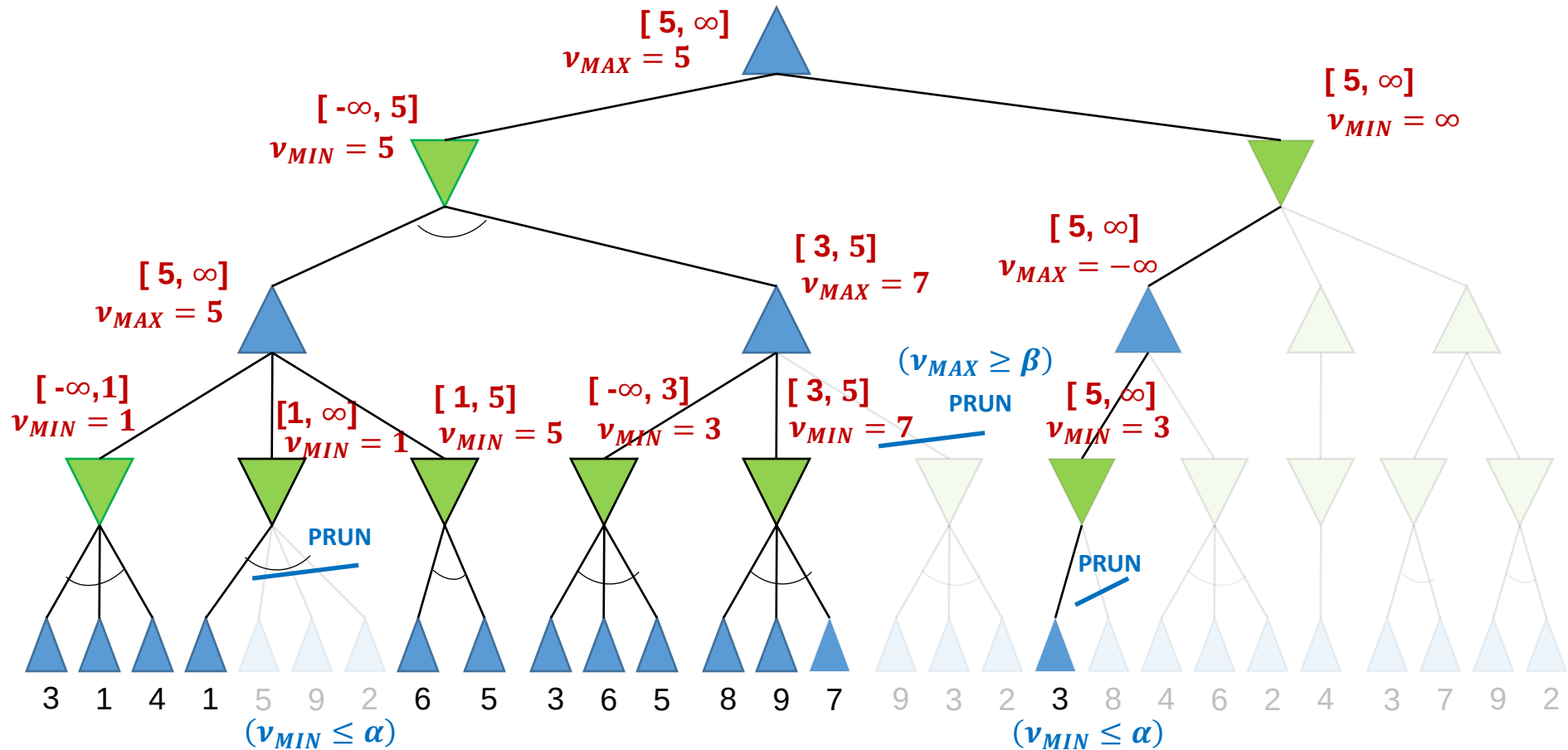
Ex: alpha-beta pruning



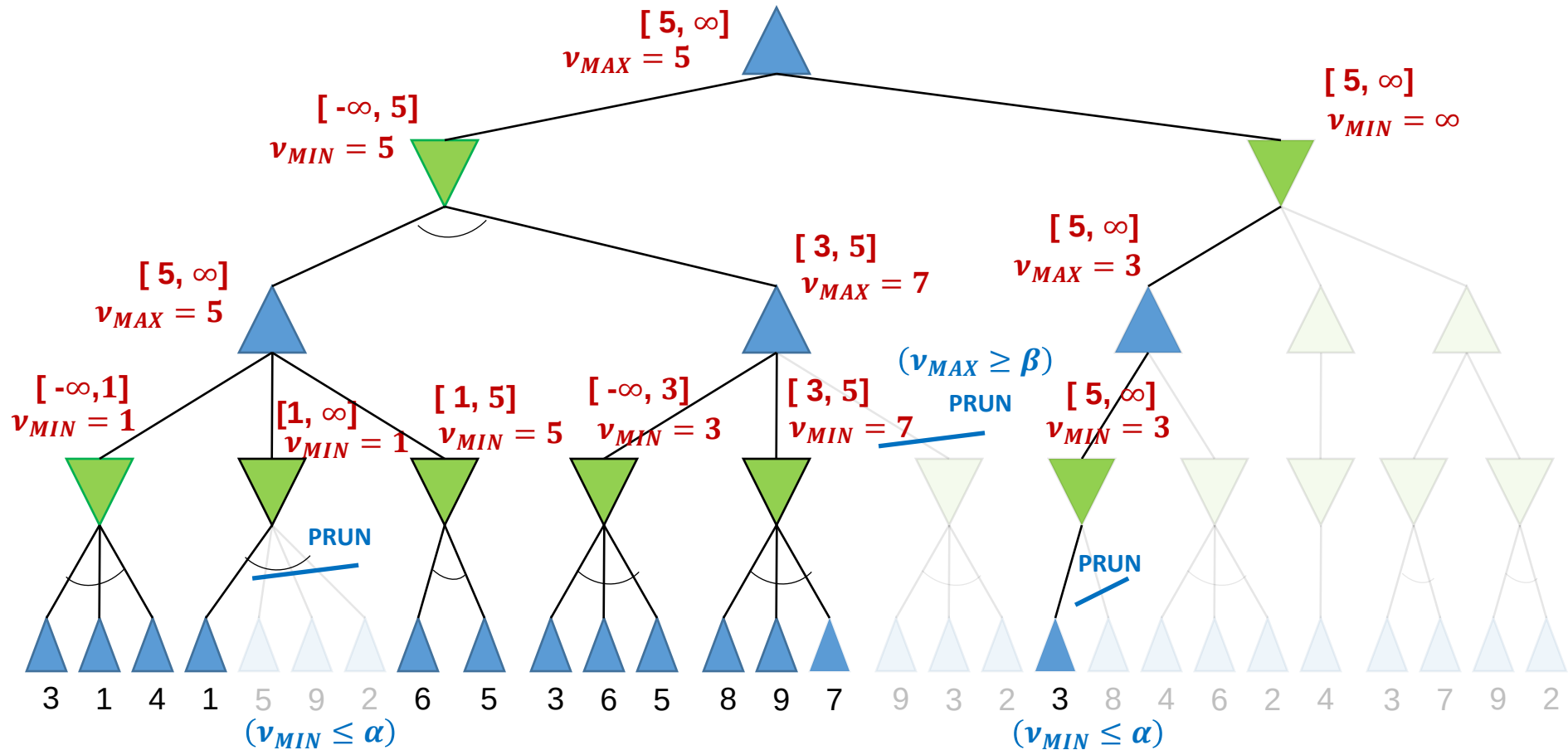
Ex: alpha-beta pruning



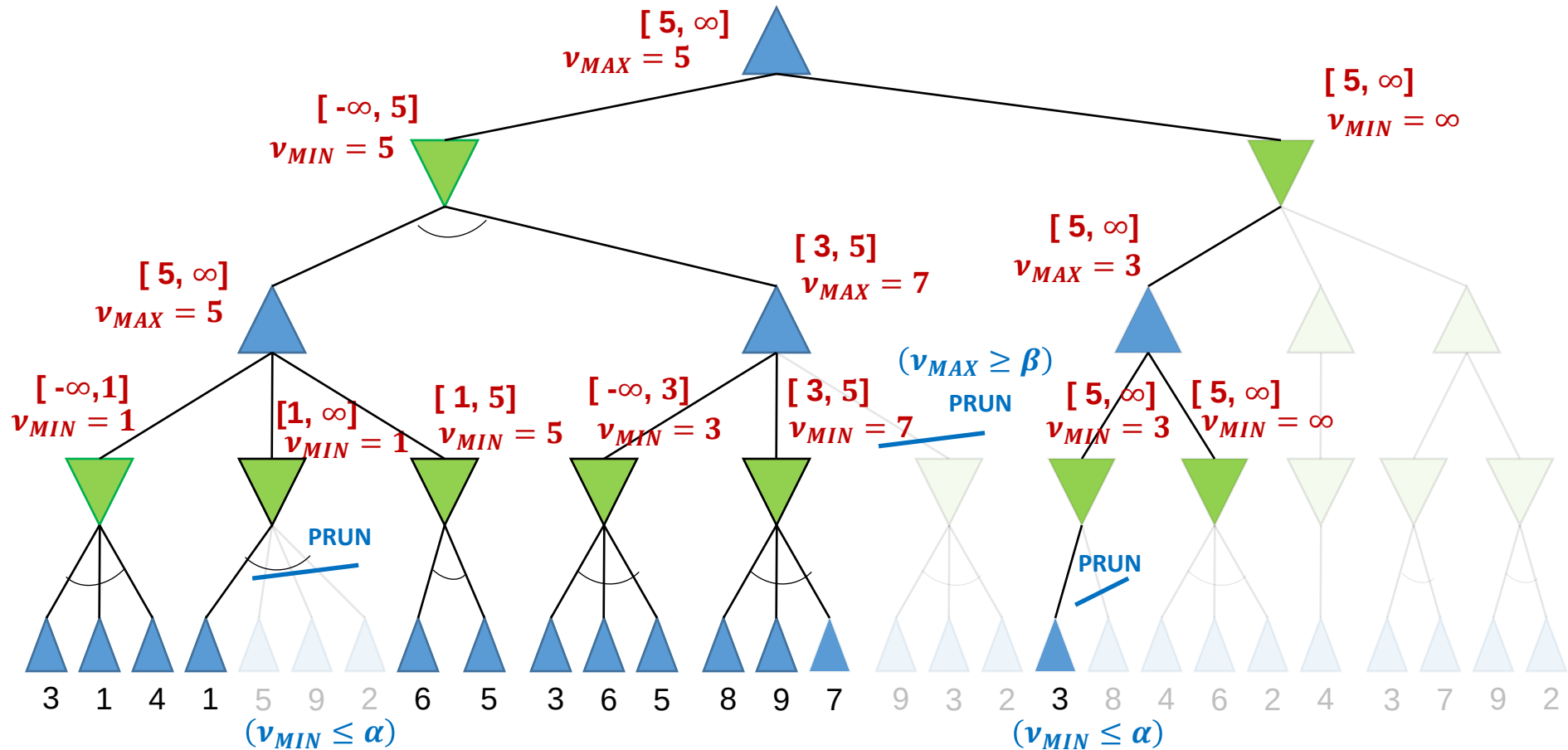
Ex: alpha-beta pruning



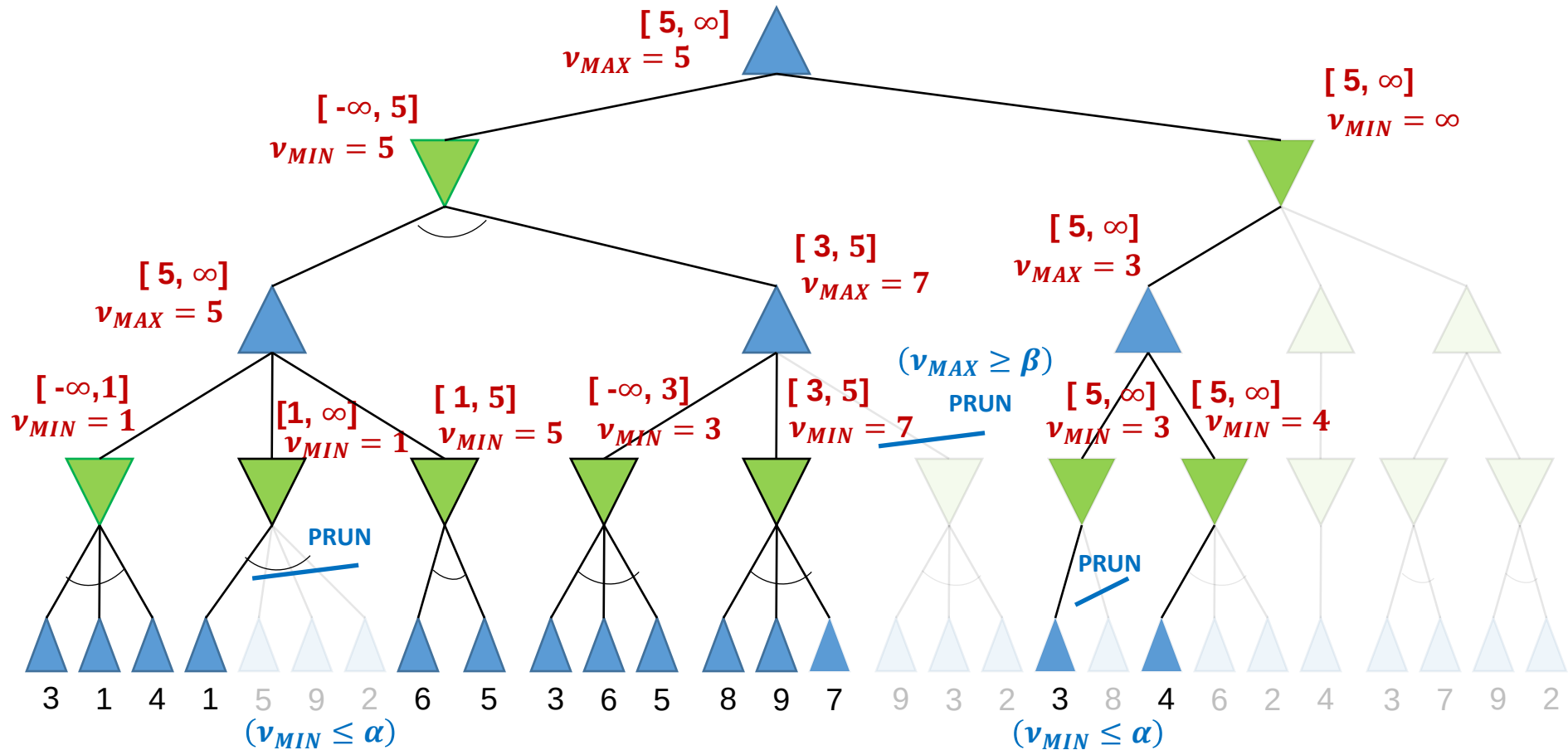
Ex: alpha-beta pruning



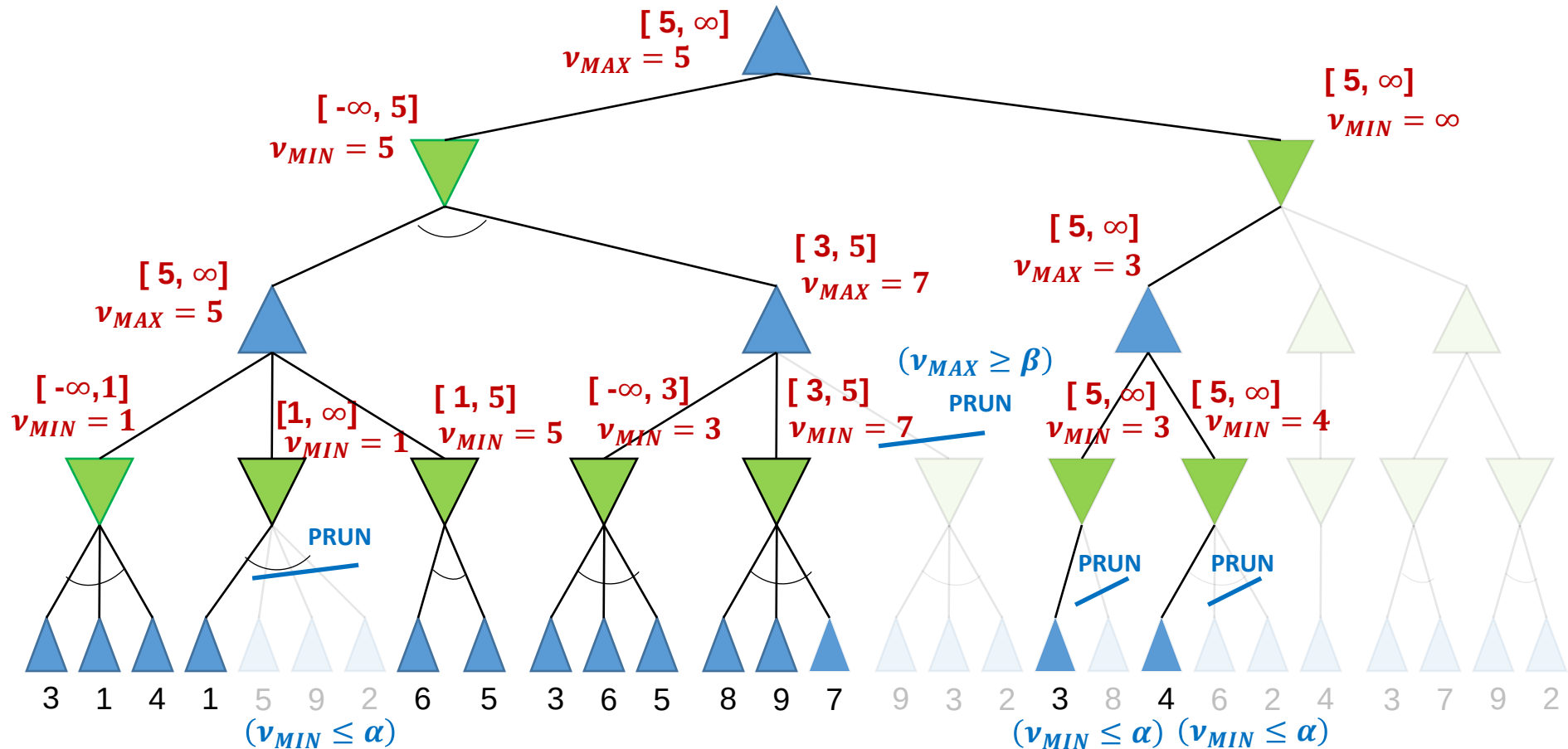
Ex: alpha-beta pruning



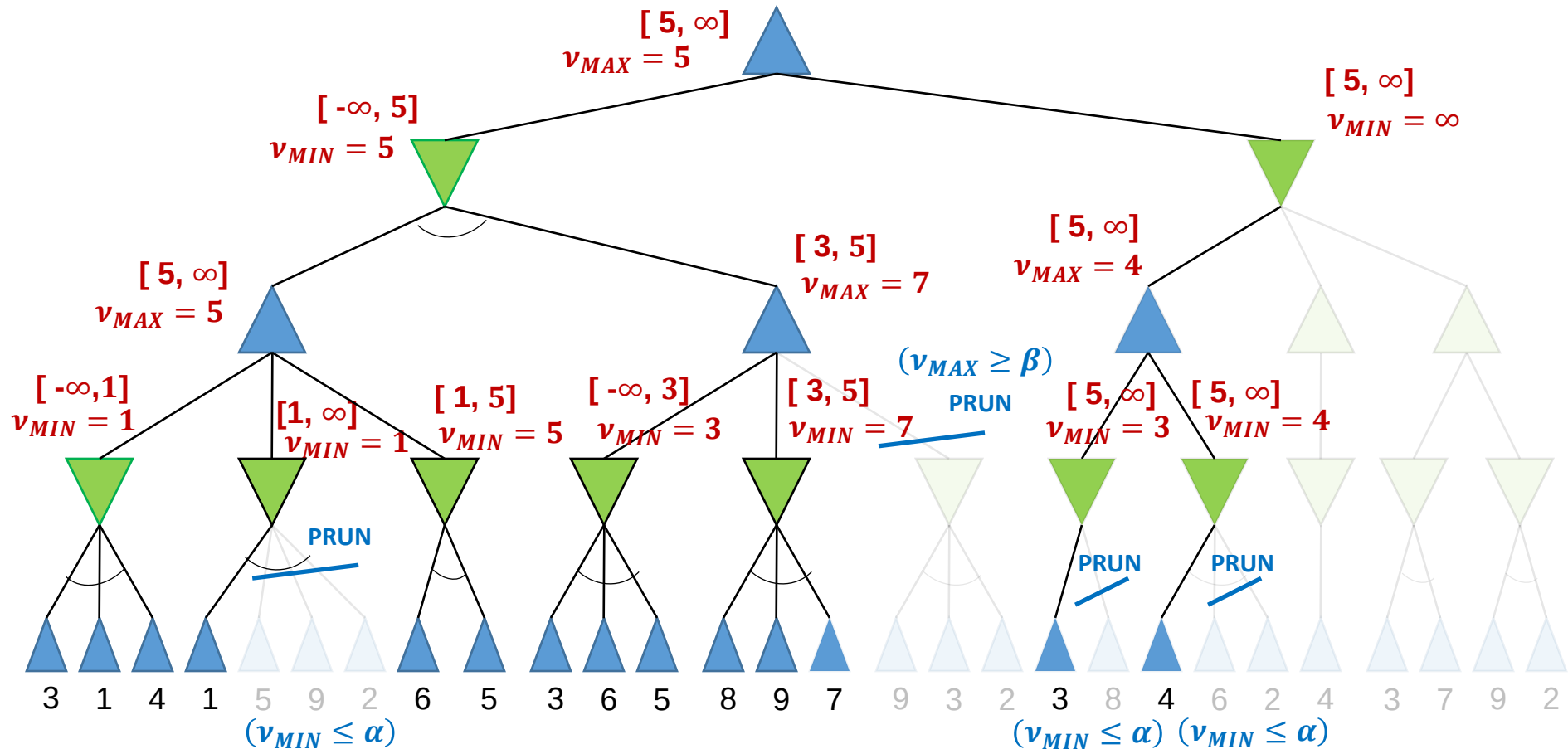
Ex: alpha-beta pruning



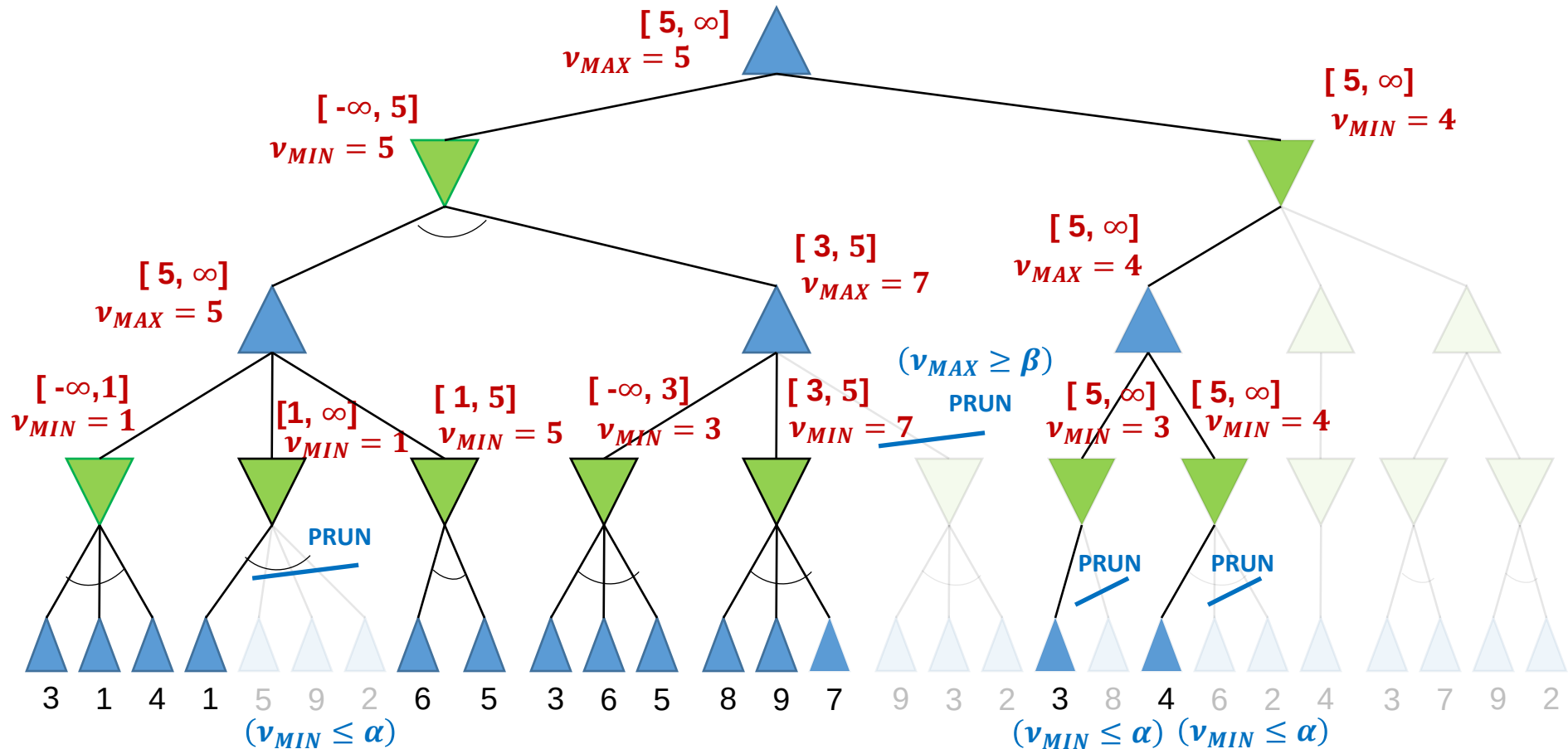
Ex: alpha-beta pruning



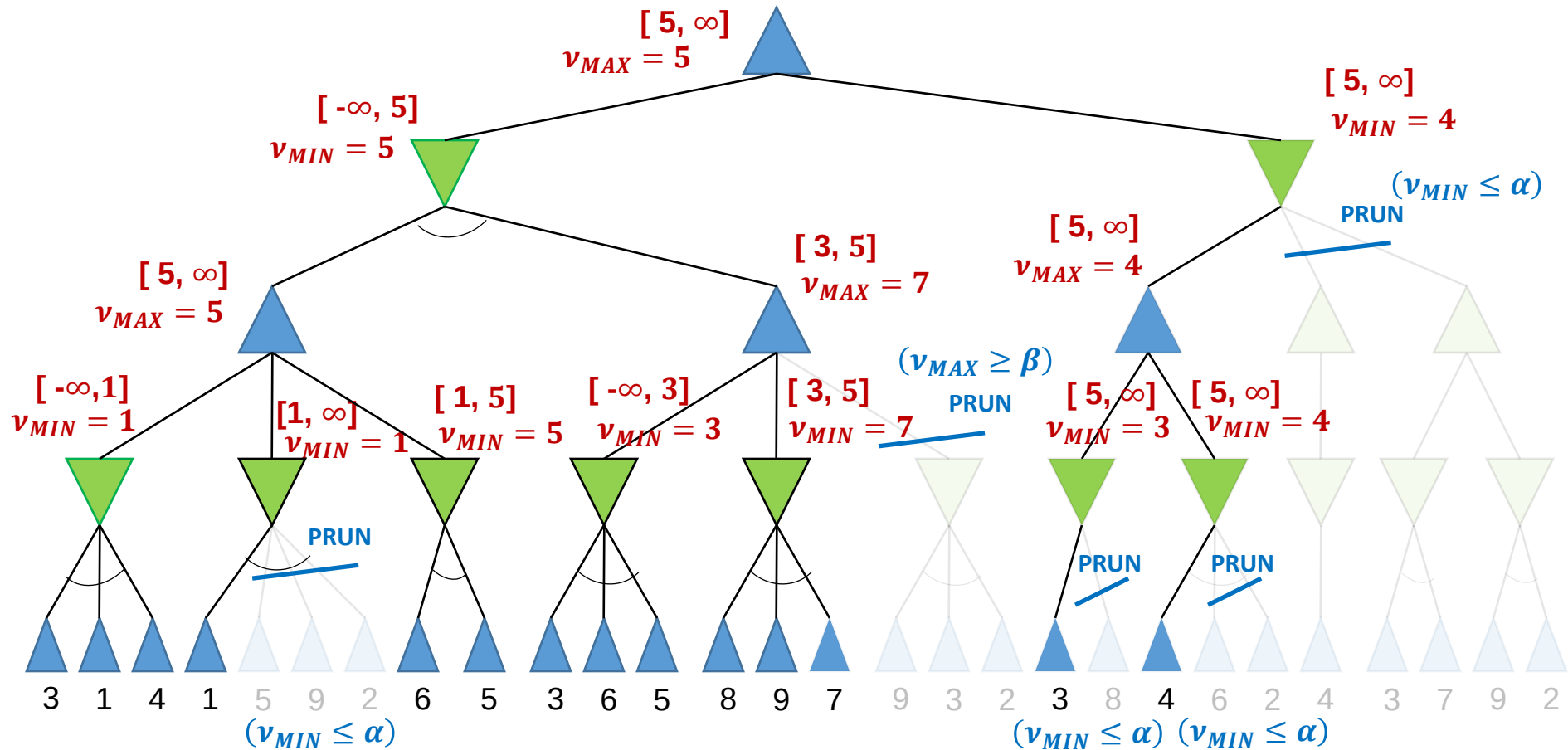
Ex: alpha-beta pruning



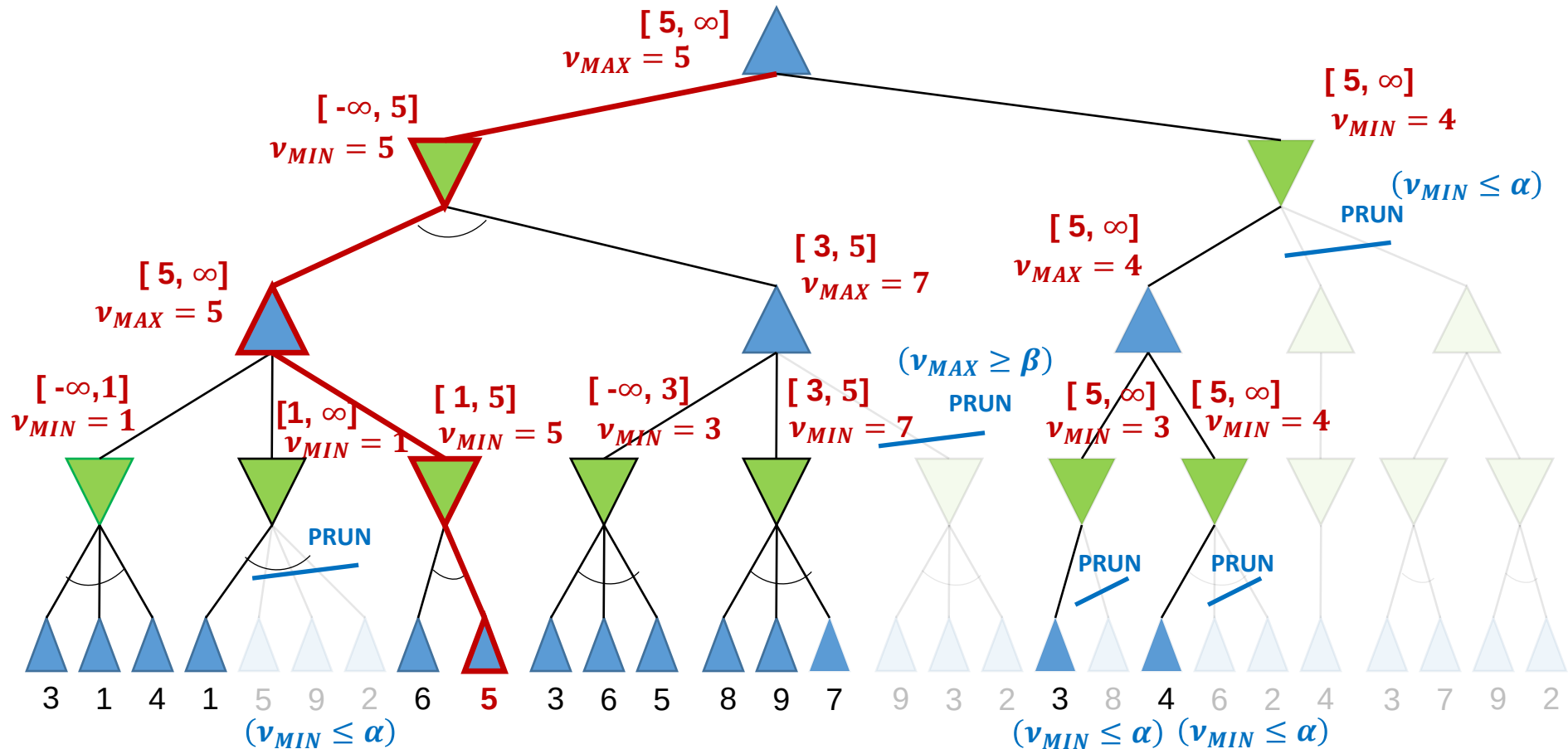
Ex: alpha-beta pruning



Ex: alpha-beta pruning



Ex: alpha-beta pruning



Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Ex: Tic-Tac-Toe

Consider the tic-tac-toe game.

- Define an appropriate **utility function**.
- Construct its **search tree with one move** (two plies) deep.
- Search for the optimal solution by using the **minimax algorithm with alpha-beta pruning**.

Ignore the symmetric states having the same minimax value.

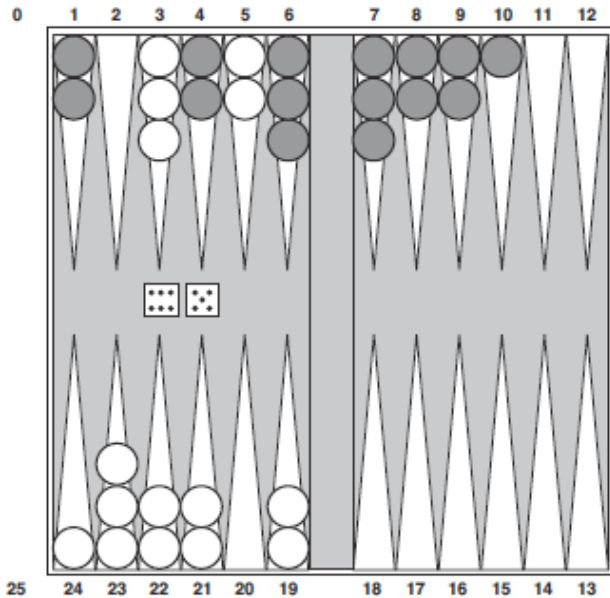
Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENT



Stochastic Games

For **stochastic games**, such as backgammon, we can not construct a standard game tree as in tic-tac-toe. In this case, the **stochastic game tree** must include **chance** nodes in addition to **MAX** and **MIN** nodes.



The goal of the game backgammon is to move all one's pieces off the board. White moves clockwise toward 25, and Black moves counterclockwise toward 0. A piece can move to any position unless multiple opponent pieces are there; if there is one opponent, it is captured and must start at the beginning. In the position shown, **White has rolled 6-5 and must choose among four legal moves:** (5-10,5-11), (5-11,19-24), (5-10,10-16), and (5-11,11-16), where the notation (5-11,11-16) means to move one piece from position 5 to 11 and then to move a piece from 11 to 16.

Adversarial Search

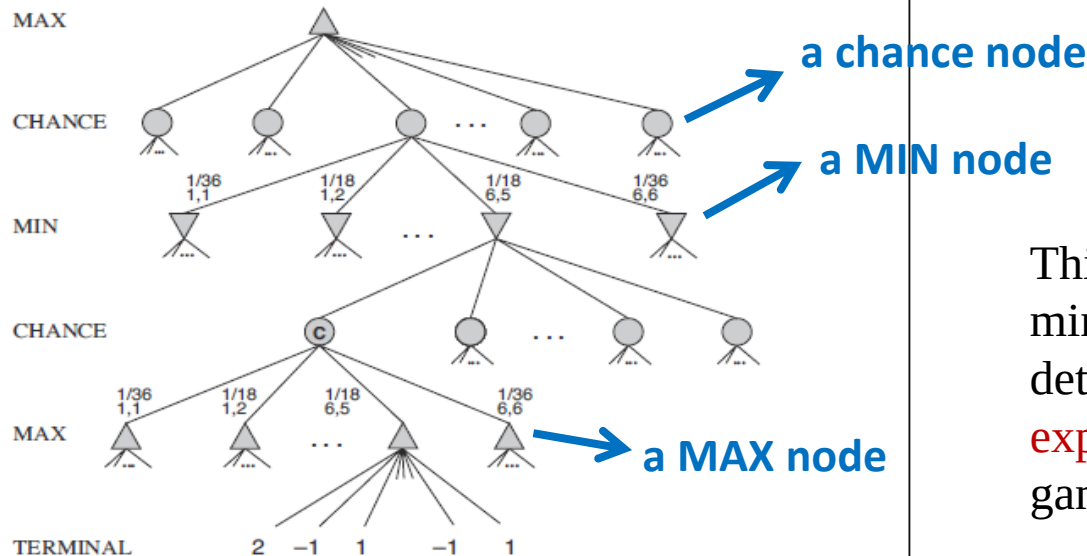
INTRODUCTION TO ARTIFICIAL INTELLIGENT



Stochastic Games

We can not exactly determine the minimax value of the **chance** nodes. Instead of this, we calculate the **expected value** of a position (**chance node**). Let r represent the index of a possible outcome of the change node, $P(r)$ be its probability and X_r be its minimax value.

$$E(X) = \sum_r X_r P(r)$$



Schematic game tree for a backgammon position.

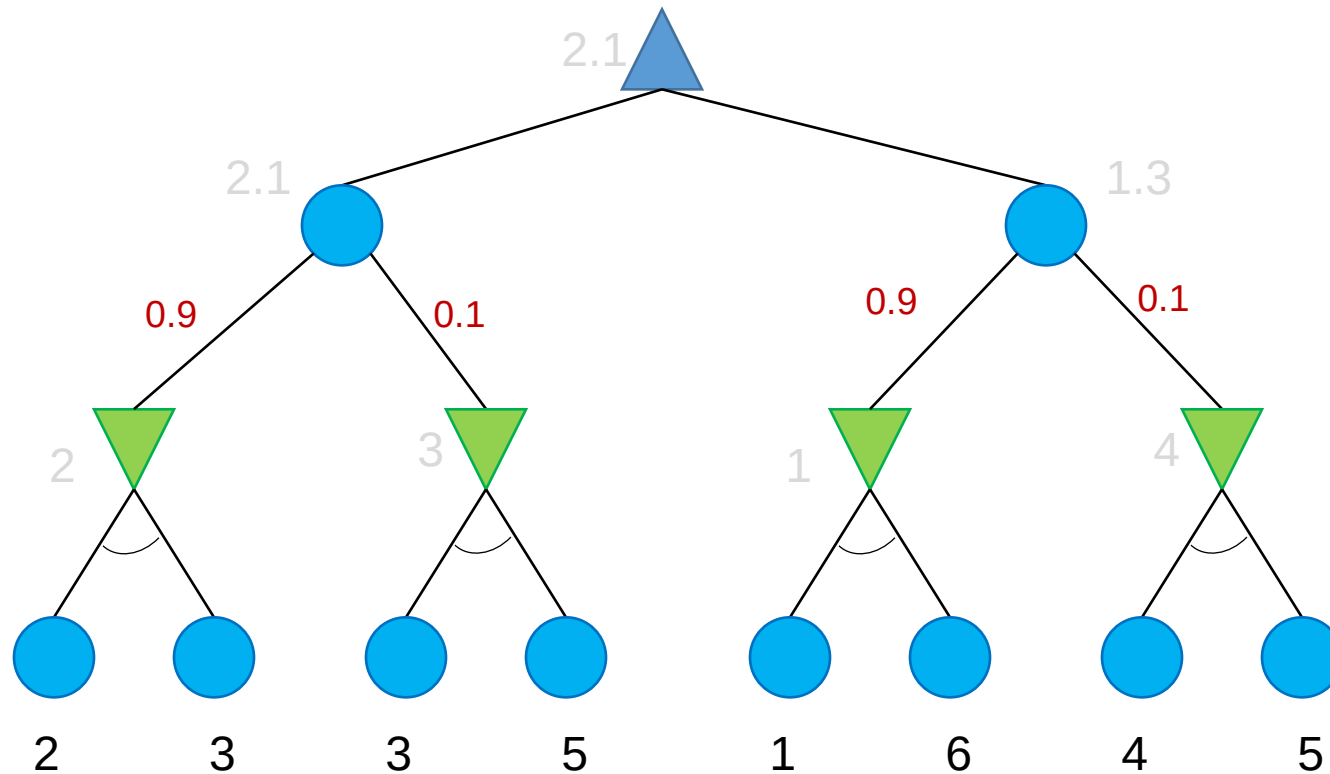
This leads us to generalize the minimax value for deterministic games to an **expecti-minimax value** for games with chance nodes.

Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Simple Example

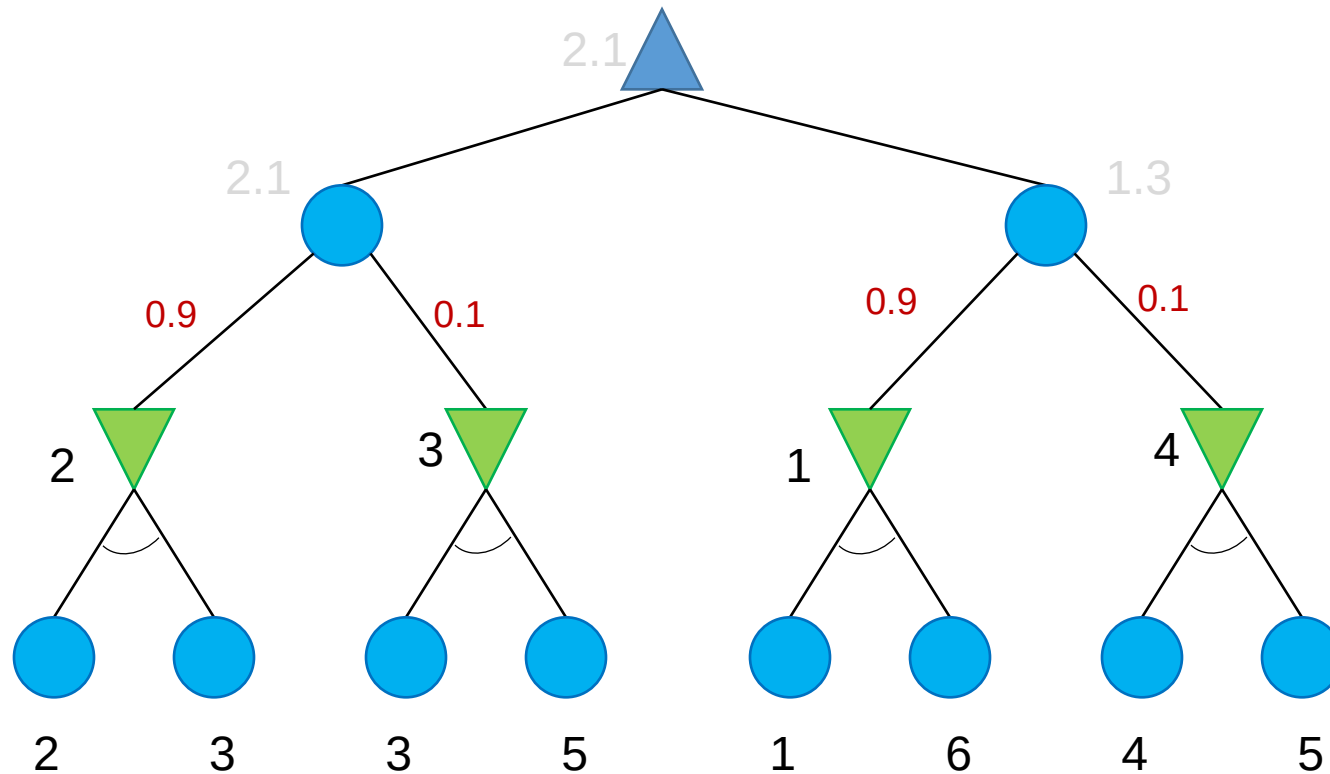


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Simple Example

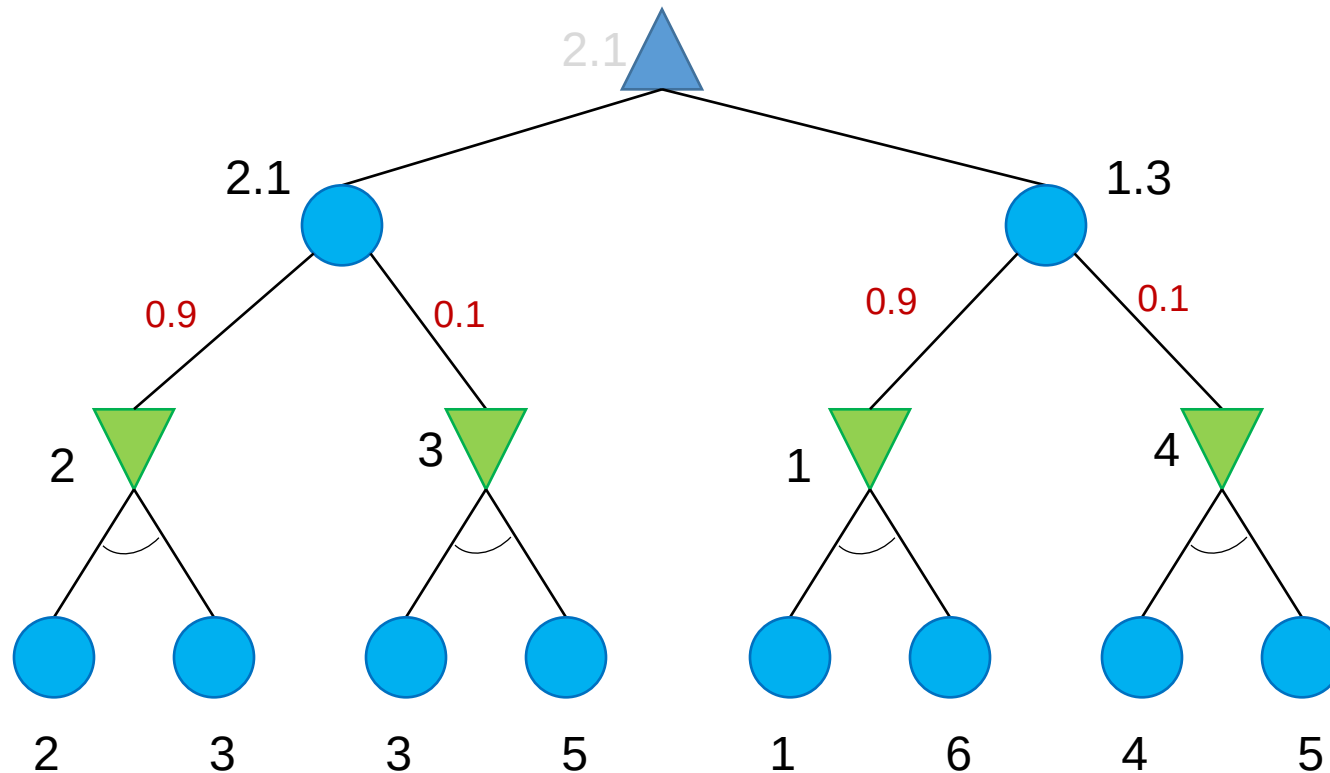


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Simple Example

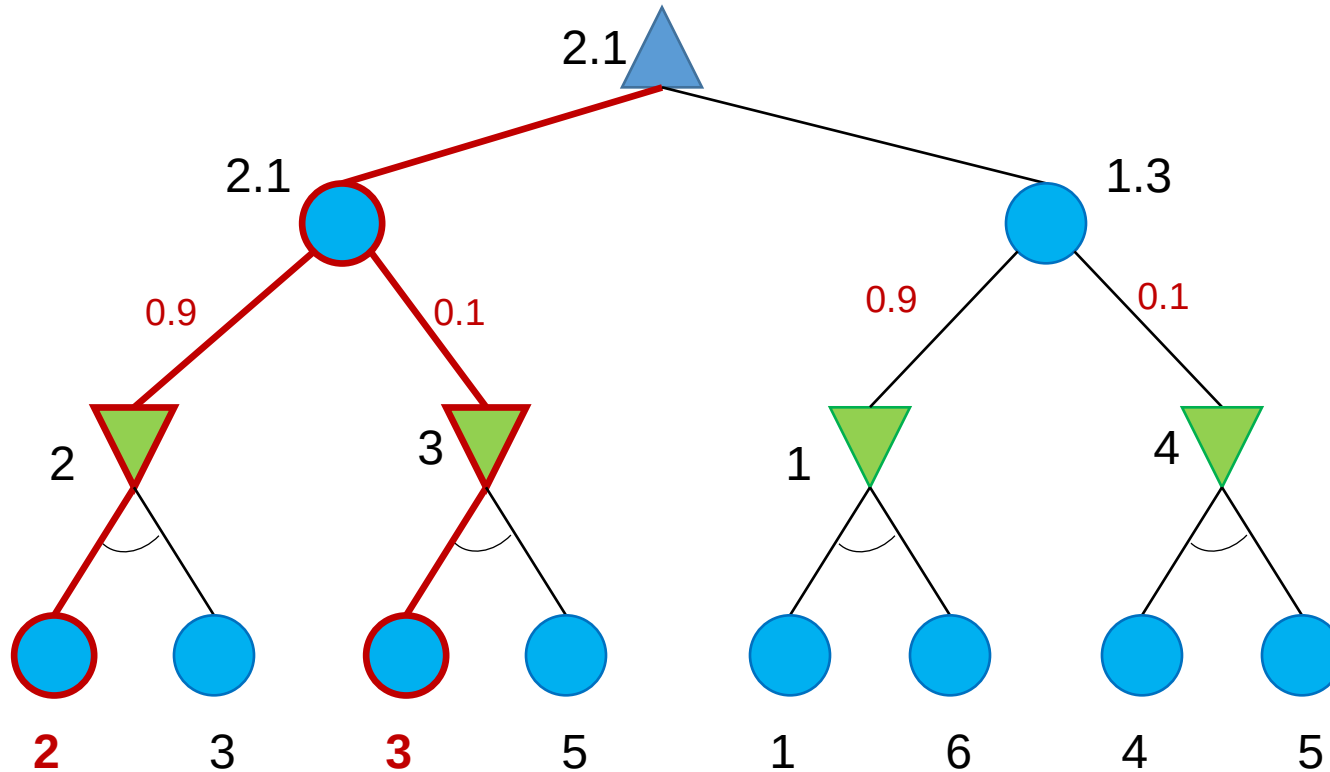


Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



A Simple Example



Adversarial Search

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



Main Reference

Stuart Russell, Peter Norvig; “Artificial Intelligence A Modern Approach”, Pearson, 3rd Edition, 2013.